

ProxGQP

Reference

Contents

1	Introduction	9
I	Background	11
2	Numerical linear algebra recalled	13
2.1	Conditioning	13
2.2	Factorisation	14
2.3	Sparse matrices	17
2.4	Schur complement	19
2.5	Iterative refinement	19
3	Convex analytic background	21
3.1	Convex sets and cones	21
3.2	Generalised inequalities	22
3.3	Dual and polar, $\text{Aut}(K)$ and symmetric cones	22
3.4	Cone catalogue	23
3.5	Jordan algebra and symmetric cones	24
3.6	Convex functions	28
3.7	Differentiability	29
3.8	Projection	30
4	Convex program, Lagrangian and KKT	43
4.1	Convex program and Lagrangian	43
4.2	KKT conditions	46
5	Monotone operator and proximal representation	49
5.1	Set-valued operators	49
5.2	The resolvent	51
5.3	Variable metric proximal point algorithm	53
5.4	Proximal operator and Moreau envelope	58
6	Newton method	63
6.1	Classic Newton method	63
6.2	Semismooth Newton method	65
6.3	Merit functions and line searches	68
6.4	Self-concordant functions and damped Newton methods	69
6.5	Smoothing families	73

7	Interior-point method basics	75
7.1	Barrier smoothing	75
7.2	Short-step path following	77
7.3	Logarithmically homogeneous barriers	78
7.4	Barriers for the catalogue	79
7.5	Primal-dual Newton systems	81
7.5.1	Exact barrier Newton system	82
7.5.2	Averaged Newton system	82
7.5.3	Jordan Newton system	83
7.5.4	Nesterov-Todd scaling	84
7.6	Predictor-corrector methods	85
8	Augmented Lagrangian method basics	89
8.1	Augmented Lagrangian smoothing family	89
8.2	The augmented Lagrangian method as a proximal point algorithm	90
8.3	The proximal point method on the KKT operator	92
8.4	Primal formulation of a proximal step	93
8.5	Infeasibility certificates	96
8.6	Asymptotic displacement and certificates	98
II	Method construction	101
9	Getting a Newton	103
9.1	First systems	103
9.2	A family of merit functions	104
9.3	Changing the representation	107
10	Smoothing point of view and interior point version	111
10.1	The proximal step as a smoothing of the slack	111
10.2	General theory for a general smoothing	111
III	Implementation detail	115
11	The map	117
11.1	What is where	117
11.2	A reading order	118
12	One solve, end to end	119
12.1	Setup	119
12.2	The starting point	119
12.3	The outer loop	119
12.4	How the metrics are represented	120
12.5	The inner loop	120
12.6	What leaves the solver	121

13 Equilibration	123
13.1 Where it happens	123
13.2 What is produced	123
13.3 The hypothesis on the row scaling	124
13.4 Ruiz's iteration	124
13.5 The cost scaling	125
14 Initialisation	127
14.1 Where it happens	127
14.2 The semismooth start	127
14.3 The interior start	128
14.3.1 A reference point	129
14.3.2 A seeded point	129
14.3.3 Pushing the point inside	130
14.4 The centres	132
15 Termination	133
15.1 Where it happens	133
15.2 What is measured	133
15.3 Termination of the inner loop	134
15.4 The certificates	134
15.5 The verdict, and the rescue	136
16 The Newton systems	137
16.1 Where it happens	137
16.2 Choosing a road	137
16.3 The three by three road	139
16.3.1 Three ways to hold a block	139
16.3.2 Solving	141
16.4 The Schur road	141
16.4.1 The weight	141
16.4.2 The reduction	142
17 Backends	145
17.1 Where it happens	145
17.2 The pattern contract	146
17.3 The five backends	146
17.4 Choosing one	147
17.5 Threads	147
18 Globalisation	149
18.1 Where it happens	149
18.2 The merit along the ray	149
18.2.1 What can be asked of it, and at what price	149
18.2.2 Why a bracket exists	150
18.3 Fraction to the boundary	151
18.4 The exact search	151
18.5 The Armijo search	152
18.6 The decrease search	152
18.7 The unit step	153

19 The parameter schedule	155
19.1 Where it happens	155
19.2 What the schedule is trying to balance	155
19.3 The parameters, once	156
19.4 <code>bcl</code> , one penalty for the whole problem	156
19.5 <code>gbcl</code> , one penalty per row	157
19.6 <code>interior</code> , for the barrier road	159
20 The cone kernel	161
20.1 Where it happens	161
20.2 The zero cone	162
20.3 The nonnegative orthant	162
20.4 The second order cone	162
20.5 The semidefinite cone	164
20.6 The exponential and power cones	167
20.6.1 The exponential cone	167
20.6.2 The power cone	169
21 The bindings and the Python interface	173
21.1 Where it happens	173
21.2 The binding is one function	173
21.3 The cones	174
21.4 <code>Problem</code> , which is where the checking is	174
21.5 Settings, and the knob names	174
21.6 Solving	175
22 Building it	177
22.1 What has to be present	177
22.2 The environment	177
22.3 Building	177
22.4 Building without the dependencies present	178
22.5 Installing	178
IV Benchmark	179
23 The harness	181
23.1 What it is	181
23.2 Getting the data	181
23.3 Running it	182
23.3.1 One core per solve	182
23.3.2 Sequential sets are different	182
23.4 Where a run goes	182
23.5 Grading	183
23.5.1 What a solver is not blamed for	183
23.6 Statistics	183

24 The benchmark sets	185
24.1 The generated sets	185
24.2 The collected archives	187
24.2.1 Maros and Mészáros, 138 instances	187
24.2.2 Netlib, 109 instances	187
24.2.3 CBLIB, 134 instances of 934	187
24.3 The portfolio sets	188
24.4 The sequential sets	189
25 Results	191
25.1 The conditions	191
25.2 The quadratic sets	191
25.3 All fifteen sets	192
26 Closing remark	195

Chapter 1

Introduction

This document serves as a reference for the solver proxgqp. It contains the theory needed to maintain the codebase, together with the implementation details.

It is separated into four parts: the first introduces the theoretical tools used to develop the two methods, the second develops the two methods on that basis, the third give some details on the implementatin and the last give some benchmark against other solvers. A closing chapter give the way the solver was developped and possible direction for the futur.

Part I

Background

Chapter 2

Numerical linear algebra recalled

The objective of this part is to introduce the notions of numerical linear algebra used in the solver.

2.1 Conditioning

We first recall what a submultiplicative norm is.

Definition 2.1 (Submultiplicative matrix norm). A norm $\|\cdot\|$ on square matrices is *submultiplicative* if

$$\|AB\| \leq \|A\| \|B\| \quad \text{for all } A, B. \quad (2.1)$$

The spectral norm and the Frobenius norm are submultiplicative; the entrywise maximum is not.

Definition 2.2 (Conditioning). Let $A \in \mathbb{R}^{n \times n}$ be invertible and $\|\cdot\|$ a submultiplicative matrix norm. The *condition number* of A is

$$\kappa(A) = \|A\| \|A^{-1}\|. \quad (2.2)$$

Generally we have in any submultiplicative norm:

$$\kappa(A^T A) \leq \kappa(A)^2$$

Definition 2.3 (Singular values and spectral norm). Let $A \in \mathbb{R}^{m \times n}$. The *singular values* of A are the nonnegative square roots of the eigenvalues of $A^T A$, written

$$\sigma_1(A) \geq \dots \geq \sigma_n(A) \geq 0.$$

The largest singular value is denoted by $\sigma_{\max}(A)$. When A has full column rank, all these singular values are positive, and the smallest one is denoted by $\sigma_{\min}(A)$.

The *spectral norm*, or matrix 2-norm, is

$$\|A\|_2 = \max_{\|x\|_2=1} \|Ax\|_2 = \sigma_{\max}(A).$$

For a rectangular matrix of full column rank, its spectral condition number is defined by

$$\kappa_2(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}.$$

And if A symmetric definite positive:

$$\kappa_2(A) = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)}.$$

Proposition 2.4 (Conditioning of a normal product). *Let $A \in \mathbb{R}^{m \times n}$ have full column rank. In the spectral norm*

$$\kappa_2(A^\top A) = \kappa_2(A)^2. \quad (2.3)$$

Proof. The eigenvalues of $A^\top A$ are the squares of the singular values of A , which are positive by the rank assumption, so $\kappa(A^\top A) = \sigma_{\max}(A)^2 / \sigma_{\min}(A)^2 = \kappa(A)^2$. $\square$

We now define backward stability. An algorithm is backward stable when its output for a problem (A, b) is the exact solution of a nearby problem in the data space, whence the name.

Definition 2.5 (Backward error). Let $\hat{x}$ be a computed solution from an algorithm of $Ax = b$. Its *backward error* is

$$\eta(\hat{x}) = \min\{\varepsilon \geq 0 : (A + \Delta A)\hat{x} = b + \Delta b, \|\Delta A\| \leq \varepsilon \|A\|, \|\Delta b\| \leq \varepsilon \|b\|\}. \quad (2.4)$$

An algorithm is *backward stable* when $\eta(\hat{x})$ is of the order of the unit roundoff for every input.

Definition 2.6 (Forward error). With x the exact solution of $Ax = b$ and $\hat{x}$ a computed one, the *forward error* is $\|\hat{x} - x\| / \|x\|$.

The forward error is bounded by a combination of the stability of the algorithm, through the backward error, and the intrinsic difficulty of the problem, through the conditioning.

Proposition 2.7 (Forward error from the condition number, [10]). *If $\kappa(A) \eta(\hat{x}) < 1$ then*

$$\frac{\|\hat{x} - x\|}{\|x\|} \leq \frac{2 \kappa(A) \eta(\hat{x})}{1 - \kappa(A) \eta(\hat{x})}, \quad (2.5)$$

which to first order is $\kappa(A) \eta(\hat{x})$.

2.2 Factorisation

In numerical linear algebra one does not generally form A^{-1} to solve $Ax = b$, mainly because the associated algorithms are not stable enough or are too costly. A factorisation is used instead, of the general form $A = R_1 \times \cdots \times R_n$, where each R_i makes $R_i^{-1}x$ easy to obtain, as for a diagonal or a triangular matrix. Only symmetric A concerns us here, since those are the ones appearing in the program. But first we define the unit in which the cost of an algorithm is measured.

Definition 2.8 (Flop). A *flop* is one floating point arithmetic operation. The cost of an algorithm is its flop count.

Definition 2.9 (Dense kernel). A *dense kernel* is an operation on matrices stored entry by entry, in contrast with the sparse matrix (defined later), so that its cost depends on the dimensions alone.

Definition 2.10 (BLAS levels). Dense kernels are graded by the ratio of arithmetic to memory traffic. *Level 1* kernels act on vectors, performing $O(n)$ flops on $O(n)$ data; *level 2* kernels are matrix–vector, performing $O(n^2)$ flops on $O(n^2)$ data; *level 3* kernels are matrix–matrix, performing $O(n^3)$ flops on $O(n^2)$ data. Only level 3 kernels perform substantially more computation per data element fetched, and hence, since fetching data is often the bottleneck, it is better to use level 3 operations whenever possible.

Definition 2.11 (Blocked algorithm). An algorithm is *blocked* when it is organised so that its inner loop is a level 3 kernel: the matrix is partitioned into submatrices, and each step updates a submatrix by a matrix–matrix product rather than by a sequence of scalar or vector operations.

Definition 2.12 (Cholesky factorisation). For $A \in \mathcal{S}^n$ a *Cholesky factorisation* is $A = LL^\top$ with L lower triangular of positive diagonal. It costs $\frac{1}{3}n^3 + O(n^2)$ flops and is blocked, its inner kernel being a level 3 update.

Proposition 2.13 (Existence of the Cholesky factorisation). $A \in \mathcal{S}^n$ admits a Cholesky factorisation if and only if $A \succ 0$, in which case the factor is unique. The factorisation is then backward stable and needs no pivoting.

Definition 2.14 ($LDL^\top$ factorisation). For $A \in \mathcal{S}^n$ an $LDL^\top$ factorisation is $PAP^\top = LDL^\top$ with P a permutation, L unit lower triangular and D block diagonal with blocks of size 1 and 2. It costs $\frac{1}{3}n^3 + O(n^2)$ flops and is blocked.

Proposition 2.15 (Existence of the $LDL^\top$ factorisation, [4]). Every $A \in \mathcal{S}^n$ admits an $LDL^\top$ factorisation with D block diagonal of blocks of size at most 2, and under the Bunch–Kaufman pivoting rule the factorisation is backward stable with a growth factor bounded independently of A .

Both factorisations are the same procedure, and it is worth writing out once, because every later statement about cost, fill and ordering is a statement about one line of it. The form below is the *right looking* one: at each step a column of L is produced and the whole remaining submatrix is updated at once.

Algorithm 2.1 Cholesky, right looking

Input: $A \in \mathcal{S}^n$ with $A \succ 0$

Output: L lower triangular with $A = LL^\top$

```

1: for  $k = 1, \dots, n$  do
2:    $L_{kk} \leftarrow \sqrt{A_{kk}}$ 
3:    $L_{k+1:n, k} \leftarrow A_{k+1:n, k} / L_{kk}$  ▷ the  $k$ th column of the factor
4:    $A_{k+1:n, k+1:n} \leftarrow A_{k+1:n, k+1:n} - L_{k+1:n, k} L_{k+1:n, k}^\top$  ▷ the rank one update
5: end for
```

Algorithm 2.2 $LDL^\top$, right looking, with D diagonal

Input: $A \in \mathcal{S}^n$ for which the factorisation exists without pivoting

Output: L unit lower triangular and D diagonal with $A = LDL^\top$

```

1: for  $k = 1, \dots, n$  do
2:    $D_{kk} \leftarrow A_{kk}$ 
3:    $L_{kk} \leftarrow 1, \quad L_{k+1:n, k} \leftarrow A_{k+1:n, k} / D_{kk}$ 
4:    $A_{k+1:n, k+1:n} \leftarrow A_{k+1:n, k+1:n} - D_{kk} L_{k+1:n, k} L_{k+1:n, k}^\top$ 
5: end for
```

The two differ in one place only. Cholesky puts $\sqrt{A_{kk}}$ on the diagonal and so requires $A_{kk} > 0$ at every step, which is why it exists exactly for $A \succ 0$. The $LDL^\top$ form keeps A_{kk} itself in D and takes no square root, so a negative pivot is admissible and the factorisation can proceed on an indefinite matrix; what it cannot tolerate is a pivot that is zero or, in floating point, small relative to the entries it must divide. That is what pivoting exists

to avoid, and what the class of the next definitions avoids by construction. The loop order $1, \dots, n$ is written that way only for readability; any symmetric permutation may be applied first.

Definition 2.16 (Inertia). The *inertia* of $A \in \mathcal{S}^n$ is the triple $(n_+(A), n_-(A), n_0(A))$ counting its positive, negative and zero eigenvalues.

Proposition 2.17 (Sylvester's law of inertia). *For any invertible X , the matrices A and $XAX^\top$ have the same inertia. In particular a symmetric permutation does not change it, and if $A = LDL^\top$ then the inertia of A is that of D , which for a diagonal D is read off the signs of its entries.*

Definition 2.18 (Quasi-definite matrix). A symmetric matrix is *quasi-definite* when it can be written in blocks as

$$\begin{pmatrix} H & B^\top \\ B & -C \end{pmatrix}, \quad H \succ 0, \quad C \succ 0, \quad (2.6)$$

with B arbitrary. Such a matrix is indefinite, of inertia $(n_H, n_C, 0)$ where n_H and n_C are the sizes of the two diagonal blocks.

Proposition 2.19 (Vanderbei). *A quasi-definite matrix is invertible, and for every symmetric permutation P the matrix $PAP^\top$ admits a factorisation $LDL^\top$ with D diagonal and no pivoting required. The signs of D are those of the inertia, by Proposition 2.17.*

This is the property that separates the two systems of the solver from a general indefinite one. Because the factorisation exists for every symmetric permutation, the permutation may be chosen for sparsity alone, by Definition 2.31 and its refinements, without any of it being revisited for stability once numbers arrive. A general indefinite matrix requires pivoting, which is decided from the values and therefore changes the pattern between iterations, defeating the split between the symbolic and the numeric factorisation which will be defined later in this chapter.

Definition 2.20 (Low-rank update of a factorisation). Let $A \in \mathcal{S}^n$ be factorised, $U \in \mathbb{R}^{n \times p}$ and $C \in \mathcal{S}^p$ with $p \ll n$. A *rank- p update* is the passage from a factorisation of A to one of

$$\tilde{A} = A + UCU^\top. \quad (2.7)$$

Proposition 2.21 (Woodbury formula). *If A , C and $C^{-1} + U^\top A^{-1}U$ are invertible then*

$$(A + UCU^\top)^{-1} = A^{-1} - A^{-1}U(C^{-1} + U^\top A^{-1}U)^{-1}U^\top A^{-1}. \quad (2.8)$$

Proposition 2.22 (Cost of a dense low-rank update, [9]). *A Cholesky or $LDL^\top$ factorisation of a dense $A \in \mathcal{S}^n$ can be updated to one of $A + UCU^\top$ with $\text{rank } U = p$ in $O(pn^2)$ flops, against $O(n^3)$ for recomputation.*

Proposition 2.23 (Symmetric eigendecomposition). *Every $A \in \mathcal{S}^n$ admits $A = Q\Lambda Q^\top$ with Q orthogonal and Λ real diagonal. It is computed backward stably in $O(n^3)$ flops, with a constant several times that of a Cholesky factorisation.*

2.3 Sparse matrices

When the matrix has many zero entries, as happens for specifically structured data, storing and computing with every entry of A is wasteful. This leads to the notion of a sparse matrix.

Definition 2.24 (Sparse matrix). A matrix is *sparse* when its number of nonzero entries is $O(n)$ rather than $O(n^2)$, and is represented so that the cost of an operation is proportional to that number rather than to n^2 .

Definition 2.25 (Pattern). The *pattern* of A is $\mathcal{P}(A) = \{(i, j) : A_{ij} \neq 0\}$. Two matrices have the same pattern when these sets agree. The *fill* of a factorisation is $\mathcal{P}(L) \setminus \mathcal{P}(A)$, the entries that vanish in A and not in its factor.

The sparsity pattern determines which entries of a matrix are nonzero, while a sparse storage format determines how those entries are represented in memory. The three standard formats are coordinate (COO), compressed sparse row (CSR), and compressed sparse column (CSC).

Definition 2.26 (Coordinate format). For a matrix $A \in \mathbb{R}^{m \times n}$ with $\text{nnz}(A)$ nonzero entries, the *coordinate format* stores three arrays:

$$\text{row}[k], \quad \text{col}[k], \quad \text{val}[k], \quad k = 1, \dots, \text{nnz}(A),$$

where $\text{val}[k] = A_{\text{row}[k], \text{col}[k]}$. Thus, each nonzero is stored together with its row and column indices.

The trouble with the previous representation is that it is not minimal in size. If the values are laid out in row order, left to right and top to bottom, it is enough to record where each row starts in **val** to recover the row index.

Definition 2.27 (Compressed sparse row format). The *compressed sparse row* (CSR) format stores three arrays:

$$\text{val}, \quad \text{col_ind}, \quad \text{row_ptr}.$$

The arrays **val** and **col_ind** have length $\text{nnz}(A)$, while **row_ptr** has length $m + 1$. The nonzeros in row i are stored in positions

$$\text{row_ptr}[i], \dots, \text{row_ptr}[i + 1] - 1$$

of **val** and **col_ind**. Hence,

$$A_{ij} = \text{val}[k] \quad \text{when} \quad k \in [\text{row_ptr}[i], \text{row_ptr}[i + 1]) \text{ and } \text{col_ind}[k] = j.$$

CSC is the same idea, with the entries of **val** laid out in column order, top to bottom and left to right.

Definition 2.28 (Compressed sparse column format). The *compressed sparse column* (CSC) format is the column-oriented analogue of CSR. It stores three arrays:

$$\text{val}, \quad \text{row_ind}, \quad \text{col_ptr}.$$

The arrays **val** and **row_ind** have length $\text{nnz}(A)$, while **col_ptr** has length $n + 1$. The nonzeros in column j are stored in positions

$$\text{col_ptr}[j], \dots, \text{col_ptr}[j + 1] - 1$$

of **val** and **row_ind**.

For a symmetric matrix only the upper or the lower triangular part need be kept, which halves the memory required and leads to the upper and lower CSR and CSC representations.

For a sparse matrix the factorisation has two phases, the first being the symbolic factorisation, which is the costlier.

Definition 2.29 (Fill-reducing permutation). A *fill-reducing permutation* is a permutation P chosen so that the factor of $PAP^\top$ has fewer nonzeros than that of A . It is computed from $\mathcal{P}(A)$ alone by a heuristic, minimising the fill exactly being NP-hard.

The factorisations above are computed one column at a time, and the cost of each step is what decides the cost of the whole.

Read the third line of either factorisation algorithm again, this time on a sparse matrix. The column $L_{k+1:n,k}$ has nonzeros only in the rows where $A_{k+1:n,k}$ did, so the outer product touches the submatrix indexed by exactly those rows and nothing else. How many there are is the quantity that governs everything.

Definition 2.30 (Elimination step and degree). Let $A \in \mathcal{S}^n$ be factored in the order $1, \dots, n$. The *elimination* of column k computes that column of L and then updates the remaining submatrix by the rank one term $-\ell_k d_k \ell_k^\top$. The *elimination degree* d_k is the number of nonzeros below the diagonal in ℓ_k , that is the number of rows still to be eliminated that column k touches.

Three consequences follow from the update being an outer product on those d_k rows. The step costs about $d_k^2/2$ flops, the update being symmetric, so the whole factorisation costs

$$\text{flops} \approx \sum_{k=1}^n \frac{d_k^2}{2}. \quad (2.9)$$

The step makes those d_k rows pairwise adjacent, so any pair among them that was not already an entry of A becomes fill. And a column with $d_k = 1$ updates a single diagonal entry, so it costs one flop and creates no fill at all.

A dense matrix has $d_k = n - k$, and (2.9) then gives $n^3/6$, which is the familiar count. Everything a fill-reducing permutation does is an attempt to keep the d_k small.

Definition 2.31 (Minimum degree ordering). The *minimum degree* heuristic chooses, at each step, a column of least current degree among those not yet eliminated, updating the degrees after each elimination. It is the heuristic behind the orderings used in practice, of which AMD is the standard refinement.

A column of degree one is therefore eliminated first by such an ordering, and by Definition 2.30 that costs one flop and creates no fill.

Definition 2.32 (Elimination tree). Let $A \in \mathcal{S}^n$ and suppose that, after applying a symmetric permutation, it has a factorisation

$$PAP^\top = LL^\top \quad \text{or} \quad PAP^\top = LDL^\top,$$

where L is lower triangular.

The *elimination tree* describes the dependencies between the columns of the factor. For each column j , its parent is

$$\text{parent}(j) = \min\{i > j : L_{ij} \neq 0\},$$

Definition 2.33 (Symbolic factorisation). The *symbolic factorisation* of A is the computation, from $\mathcal{P}(A)$ alone, of a fill-reducing permutation, of the elimination tree, of $\mathcal{P}(L)$ and of the storage for L . It reads no numerical value, and is therefore common to all matrices of the same pattern.

Once this is done for concrete value a numeric factorisation can be done much faster.

Definition 2.34 (Numeric factorisation). The *numeric factorisation* is the computation of the values of L and D into the structure produced by the symbolic factorisation.

Consequently, if the sparsity pattern of A is fixed even as its values change, the symbolic factorisation need be performed only once, which is an important gain.

Two numeric factorisations of the same pattern can differ greatly in speed, according to whether they treat columns one at a time or in groups.

Definition 2.35 (Simplicial and supernodal factorisation). A *simplicial* factorisation eliminates one column at a time, and its inner loop is a sparse vector operation. A *supernodal* factorisation first groups columns with identical or nearly identical patterns into *supernodes*, and eliminates each group as a unit; its inner loop is then a dense kernel on a small dense block, and it can call the level three routines of Definition 2.10.

The choice is not free. Supernodes only exist when many columns share a pattern, which is to say when the factor is locally dense, so a supernodal algorithm gains on a dense factor and loses on a very sparse one, where the grouping finds nothing and only adds overhead. A backend that offers both usually decides between them from the predicted density of L .

2.4 Schur complement

Given a block matrix, it is possible to reduce its size and obtain an equivalent smaller system. It is obtained by expressing one variable directly in terms of the others and substituting it into the remaining relations. What results is generally worse conditioned, but smaller, and sometimes gains a property such as definiteness.

Proposition 2.36 (Schur complement). Let $K = \begin{pmatrix} A & B \\ B^\top & D \end{pmatrix}$ with D invertible, and put $S = A - BD^{-1}B^\top$. Then K is invertible if and only if S is, the system $K \begin{pmatrix} u \\ v \end{pmatrix} = \begin{pmatrix} a \\ b \end{pmatrix}$ is equivalent to

$$S u = a - BD^{-1}b, \quad v = D^{-1}(b - B^\top u), \quad (2.10)$$

2.5 Iterative refinement

Given a factorisation, it is sometimes possible to reduce the error by a process named iterative refinement.

Definition 2.37 (Iterative refinement). Given a factorisation of A and an approximate solution $\hat{x}^{(0)}$ of $Ax = b$, *iterative refinement* computes, for $k = 0, 1, \dots$,

$$\rho^{(k)} = b - A\hat{x}^{(k)}, \quad A\delta^{(k)} = \rho^{(k)}, \quad \hat{x}^{(k+1)} = \hat{x}^{(k)} + \delta^{(k)}, \quad (2.11)$$

the correction being solved with the existing factorisation.

Chapter 3

Convex analytic background

3.1 Convex sets and cones

We recall some basic notions connected with convexity.

Definition 3.1 (Convex set). $C \subseteq \mathbb{R}^m$ is *convex* if $\lambda v + (1 - \lambda)w \in C$ for all $v, w \in C$ and all $\lambda \in [0, 1]$.

Definition 3.2 (Convex hull). A *convex combination* of $v_1, \dots, v_r$ is $\sum_i \lambda_i v_i$ with $\lambda_i \geq 0$ and $\sum_i \lambda_i = 1$. The *convex hull* $\text{conv } C$ is the set of convex combinations of finitely many points of C , equivalently the smallest convex set containing C .

We first define what a cone is. Cones appear naturally in optimisation problems, mainly because most constraints posed in practice are invariant under scaling by a positive number.

Definition 3.3 (Cone). $K \subseteq \mathbb{R}^m$ is a *cone* if $v \in K$ and $t \geq 0$ imply $tv \in K$. It is a *convex cone* if it is in addition convex, equivalently if $v, w \in K$ and $s, t \geq 0$ imply $sv + tw \in K$.

Sometimes we want to say that an element lies strictly inside a cone, but the cone is often contained in an affine space of strictly lower dimension, because of equality constraints. We therefore prefer the notion of relative interior.

Definition 3.4 (Affine hull). The *affine hull* $\text{aff } C$ is the smallest affine subspace of $\mathbb{R}^m$ containing C .

Definition 3.5 (Relative interior).

$$\text{relint } C = \{v \in C : \exists \varepsilon > 0, B(v, \varepsilon) \cap \text{aff } C \subseteq C\}. \quad (3.1)$$

Definition 3.6 (Relative boundary). $\text{rbd } C = \text{cl } C \setminus \text{relint } C$.

Definition 3.7 (Pointed cone). A convex cone K is *pointed* if $K \cap (-K) = \{0\}$.

Definition 3.8 (Solid cone). A convex cone $K \subseteq \mathbb{R}^m$ is *solid* if $\text{int } K \neq \emptyset$.

Definition 3.9 (Proper cone). A convex cone is *proper* if it is closed, pointed and solid.

3.2 Generalised inequalities

The simplest constraints after the scalar inequalities are memberships of a half space, $x \leq 0$ for instance. For a general cone, membership can be read in the same fashion once the notion of a generalised inequality is defined.

Definition 3.10 (Generalised inequality). For $K \subseteq \mathbb{R}^m$ a closed convex cone,

$$x \preceq_K y \iff y - x \in K, \quad x \prec_K y \iff y - x \in \text{int } K. \quad (3.2)$$

Proposition 3.11 (Order properties of $\preceq_K$). $\preceq_K$ is reflexive and transitive, and if $x \preceq_K y$ and $x' \preceq_K y'$ then $x + x' \preceq_K y + y'$ and $tx \preceq_K ty$ for every $t \geq 0$.

Proposition 3.12 (Antisymmetry). $\preceq_K$ is antisymmetric, and hence a partial order, if and only if K is pointed.

Proposition 3.13 (Emptiness of the strict relation). There exist x, y with $x \prec_K y$ if and only if K is solid.

Definition 3.14 (The two classical instances). For $K = \mathbb{R}_+^n$, $x \preceq_K y$ is $x_i \leq y_i$ for every i and $x \prec_K y$ is $x_i < y_i$ for every i . For $K = \mathcal{S}_+^n$, $X \preceq_K Y$ is $Y - X$ positive semidefinite and $X \prec_K Y$ is $Y - X$ positive definite; both are written without the subscript, as $X \preceq Y$ and $X \prec Y$.

3.3 Dual and polar, $\text{Aut}(K)$ and symmetric cones

Two notions are natural here. The first is the dual cone, the set of elements of the space forming an angle of at most $\frac{\pi}{2}$ with every element of the cone.

Definition 3.15 (Dual cone). $K^* = \{z \in \mathbb{R}^m : \langle z, v \rangle \geq 0 \text{ for all } v \in K\}$.

The second is its opposite, the set of all elements forming an angle of at least $\frac{\pi}{2}$ with every element of the cone.

Definition 3.16 (Polar cone). $K^\circ = \{z \in \mathbb{R}^m : \langle z, v \rangle \leq 0 \text{ for all } v \in K\} = -K^*$.

No cone equals its polar: $K \subseteq K^\circ$ forces $\langle v, v \rangle \leq 0$ on K , hence $K = \{0\}$, and $\{0\}^\circ = \mathbb{R}^m$. The question is worth asking of the dual instead, where it happens often enough to be worth a name, and leads to the properties discussed below.

Definition 3.17 (Self-dual cone). K is *self-dual* if $K = K^*$.

As the cone get bigger the dual get smaller self-duality is hence a first idea of symmetry for a cone.

Topologically the dual is always the better behaved of the two, even when the cone itself is not, being the set of its supporting hyperplanes.

Proposition 3.18 (The dual is a closed convex cone). For any $K \subseteq \mathbb{R}^m$, K^* is a closed convex cone.

Proof. $K^* = \bigcap_{v \in K} \{z : \langle z, v \rangle \geq 0\}$ is an intersection of closed convex cone. And all three properties are preserved by arbitrary intersections. $\square$

Proposition 3.19 (Inverse monotonicity of duality). $K_1 \subseteq K_2$ implies $K_2^* \subseteq K_1^*$.

Proposition 3.20 (Bipolar theorem, [17]). *K is a closed convex cone if and only if $K^{**} = K$.*

When constraints of different kinds are present, the cone is written as a product of smaller cones, one per constraint. The following proposition states that the dual is compatible with that product.

Proposition 3.21 (Duality and Cartesian products). *If $K = K_1 \times \cdots \times K_p$ then $K^* = K_1^* \times \cdots \times K_p^*$.*

Proof. Write $z = (z_1, \dots, z_p)$ and $v = (v_1, \dots, v_p)$, so that $\langle z, v \rangle = \sum_j \langle z_j, v_j \rangle$. If every $z_j \in K_j^*$ then each term is nonnegative for $v \in K$, so $z \in K^*$. Conversely let $z \in K^*$ and fix j and $v_j \in K_j$. Taking v with that j th block and zero elsewhere, which lies in K , gives $\langle z_j, v_j \rangle \geq 0$, so $z_j \in K_j^*$. $\square$

Once we have a cone a natural question what change of coordinate that preserve it :

Definition 3.22 (Automorphism group). For $K \subseteq \mathbb{R}^m$ a proper cone,

$$\text{Aut}(K) = \{A \in GL(m, \mathbb{R}) : AK = K\}. \quad (3.3)$$

Another natural question is whether some points are more special than others, in the sense that no automorphism carries one to another while keeping the cone intact. An $A \in \text{Aut}(K)$ is a linear homeomorphism of $\mathbb{R}^m$ carrying K onto K , so it carries $\text{int } K$ onto $\text{int } K$ and hence ∂K onto ∂K : the boundary is preserved, and it is enough to ask the question on the interior.

Definition 3.23 (Homogeneous cone). A proper cone K is *homogeneous* if $\text{Aut}(K)$ acts transitively on $\text{int } K$: for all $u, v \in \text{int } K$ there is $A \in \text{Aut}(K)$ with $Au = v$.

This gives us a second symmetry property which together with self-duality give the following definition:

Definition 3.24 (Symmetric cone). A proper cone is *symmetric* if it is homogeneous and self-dual.

Symmetric cones have far better properties than non symmetric ones, and this is reflected in the methods.

3.4 Cone catalogue

This section gives the definition of the six cones implemented in the solver, together with some of their properties.

Definition 3.25 (Zero cone). $\{0\} \subseteq \mathbb{R}^d$ is a closed convex cone with $\{0\}^* = \mathbb{R}^d$. It is not self-dual and not solid.

Definition 3.26 (Nonnegative orthant). $\mathbb{R}_+^d = \{x \in \mathbb{R}^d : x_i \geq 0\}$ is a symmetric cone, and $(\mathbb{R}_+^d)^* = \mathbb{R}_+^d$.

Definition 3.27 (Lorentz cone). $\mathcal{L}^d = \{(t, a) \in \mathbb{R} \times \mathbb{R}^{d-1} : \|a\| \leq t\}$ is a symmetric cone, and $(\mathcal{L}^d)^* = \mathcal{L}^d$.

Definition 3.28 (Semidefinite cone). $\mathcal{S}_+^k = \{X \in \mathcal{S}^k : X \succeq 0\}$ is a symmetric cone for the trace inner product $\langle X, Y \rangle = \text{tr}(XY)$, and $(\mathcal{S}_+^k)^* = \mathcal{S}_+^k$.

Definition 3.29 (Exponential cone).

$$\mathcal{K}_{\text{exp}} = \text{cl}\{(x_1, x_2, x_3) \in \mathbb{R}^3 : x_2 e^{x_1/x_2} \leq x_3, x_2 > 0\}, \quad (3.4)$$

a proper cone that is not self-dual, with

$$\mathcal{K}_{\text{exp}}^* = \text{cl}\{(z_1, z_2, z_3) \in \mathbb{R}^3 : -z_1 e^{z_2/z_1} \leq z_3, z_1 < 0\}. \quad (3.5)$$

Definition 3.30 (Power cone). For $\alpha \in (0, 1)$,

$$\mathcal{K}_\alpha = \{(x_1, x_2, x_3) \in \mathbb{R}^3 : x_1^\alpha x_2^{1-\alpha} \geq |x_3|, x_1 \geq 0, x_2 \geq 0\}, \quad (3.6)$$

a proper cone that is not self-dual, with

$$\mathcal{K}_\alpha^* = \{(z_1, z_2, z_3) : \left(\frac{z_1}{\alpha}\right)^\alpha \left(\frac{z_2}{1-\alpha}\right)^{1-\alpha} \geq |z_3|, z_1 \geq 0, z_2 \geq 0\}. \quad (3.7)$$

3.5 Jordan algebra and symmetric cones

Symmetric cones enjoy a great many properties, owing to their links with Jordan algebras.

Definition 3.31 (Euclidean Jordan algebra). A *Euclidean Jordan algebra* (EJA) is a finite dimensional real vector space V with an inner product $\langle \cdot, \cdot \rangle$ and a bilinear product $\circ$ such that, for all $u, v, w \in V$ and with $u^2 = u \circ u$,

$$u \circ v = v \circ u, \quad u \circ (u^2 \circ v) = u^2 \circ (u \circ v), \quad \langle u \circ v, w \rangle = \langle v, u \circ w \rangle. \quad (3.8)$$

It is with an unit e satisfying $e \circ u = u$.

Fixing a basis and writing $x \circ y$ in it shows that $\circ$ is C^∞ , being a bilinear map.

Example 3.32 (Three Jordan algebras). $V = \mathbb{R}^d$ with $u \circ v = u \odot v$ the componentwise product and $e = (1, \dots, 1)$. $V = \mathbb{R} \times \mathbb{R}^{d-1}$ with $(t, a) \circ (s, b) = (ts + \langle a, b \rangle, tb + sa)$ and $e = (1, 0)$. $V = \mathcal{S}^k$ with $U \circ W = \frac{1}{2}(UW + WU)$ and $e = I$.

The following theorems link symmetric with EJA:

Definition 3.33 (Cone of squares). The *cone of squares* of V is $K_V = \{u \circ u : u \in V\}$.

Proposition 3.34 (Jordan, von Neumann and Wigner, [8]). *A proper cone is symmetric if and only if it is the cone of squares of a Euclidean Jordan algebra.*

We have the fundamental theorem:

Proposition 3.35 (Spectral decomposition, [8]). *Let V be a Euclidean Jordan algebra (EJA). There is an unique $r \in \mathbb{N}$ named the rank of V such that every $x \in V$ admits*

$$x = \sum_{i=1}^r \lambda_i c_i, \quad (3.9)$$

with $\lambda_1, \dots, \lambda_r \in \mathbb{R}$ the eigenvalues of x and $c_1, \dots, c_r$ a Jordan frame, that is idempotents with $c_i \circ c_j = 0$ for $i \neq j$, $c_i \circ c_i = c_i$ and $\sum_i c_i = e$. The eigenvalues are unique up to their order, the frame need not be.

It is then immediate that:

Corollary 3.36 (Trace, determinant, inverse, and powers). *Let V be a Euclidean Jordan algebra of rank r , and let*

$$x = \sum_{i=1}^r \lambda_i c_i$$

be a spectral decomposition of x . The trace and determinant of x are defined by

$$\operatorname{tr}(x) = \sum_{i=1}^r \lambda_i, \quad \det(x) = \prod_{i=1}^r \lambda_i.$$

These quantities are independent of the choice of Jordan frame because the eigenvalues are unique up to permutation.

Moreover,

$$x \in K_V \iff \lambda_i \geq 0 \text{ for every } i,$$

and

$$x \in \operatorname{int} K_V \iff \lambda_i > 0 \text{ for every } i.$$

The element x is invertible if and only if

$$\det(x) \neq 0,$$

or equivalently, if and only if $\lambda_i \neq 0$ for every i . In this case,

$$x^{-1} = \sum_{i=1}^r \lambda_i^{-1} c_i.$$

More generally, whenever the scalar powers λ_i^p are defined,

$$x^p = \sum_{i=1}^r \lambda_i^p c_i.$$

Finally, every primitive idempotent c_i in a Jordan frame satisfies

$$\operatorname{tr}(c_i) = 1.$$

If $r > 1$, then

$$\det(c_i) = 0.$$

Definition 3.37 (Ideal of a Jordan algebra). Let $(J, \circ)$ be a Jordan algebra. A linear subspace $I \subseteq J$ is called a *Jordan ideal* if

$$x \circ y \in I \quad \text{for all } x \in J \text{ and } y \in I.$$

Equivalently,

$$J \circ I \subseteq I.$$

Definition 3.38 (Simple Jordan algebra). A Jordan algebra J is called *simple* if its only Jordan ideals are the trivial ideals

$$\{0\} \quad \text{and} \quad J.$$

Equivalently, if $I \subseteq J$ is a linear subspace satisfying

$$x \circ y \in I \quad \text{for all } x \in J \text{ and } y \in I,$$

then either $I = \{0\}$ or $I = J$.

Simple Euclidean Jordan algebras are the building blocks of all the others.

Theorem 3.39 (Decomposition of Euclidean Jordan algebras). *Every finite-dimensional Euclidean Jordan algebra V is an orthogonal direct sum of simple Euclidean Jordan algebras. More precisely, there exist simple Jordan ideals $V_1, \dots, V_p$ such that*

$$V = V_1 \oplus \cdots \oplus V_p,$$

$$V_i \circ V_j = \{0\} \quad \text{for } i \neq j,$$

and

$$\langle V_i, V_j \rangle = 0 \quad \text{for } i \neq j.$$

Proposition 3.40 (Form of the inner product). *Let V be a finite-dimensional Euclidean Jordan algebra with orthogonal decomposition into simple ideals*

$$V = V_1 \oplus \cdots \oplus V_r.$$

Then :

$$\langle x, y \rangle = \sum_{i=1}^r c_i \operatorname{tr}_i(x_i \circ y_i), \quad c_i > 0,$$

where

$$x = (x_1, \dots, x_r), \quad y = (y_1, \dots, y_r),$$

and tr_i denotes the Jordan trace on the simple component V_i . In particular, the canonical trace inner product is obtained by taking $c_i = 1$ for every i :

$$\langle x, y \rangle_{\text{can}} = \operatorname{tr}(x \circ y) = \sum_{i=1}^r \operatorname{tr}_i(x_i \circ y_i).$$

Some property that we will use:

Theorem 3.41 (Orthogonality in the cone of squares). *Let V be a Euclidean Jordan algebra, let K denote its cone of squares, and equip V with the trace inner product*

$$\langle x, y \rangle = \operatorname{tr}(x \circ y).$$

Then, for all $x, y \in K$,

$$x \circ y = 0 \quad \Longleftrightarrow \quad \langle x, y \rangle = 0.$$

Definition 3.42 (Left multiplication operator). Let V be a Jordan algebra. For each $x \in V$, the *left multiplication operator* associated with x is the linear map

$$L(x) : V \rightarrow V, \quad L(x)y = x \circ y.$$

Definition 3.43 (Quadratic representation). For $x \in V$, the *quadratic representation* of x is the linear map

$$Q(x) : V \rightarrow V$$

defined by

$$Q(x) = 2L(x)^2 - L(x^2).$$

Equivalently, for $y \in V$,

$$Q(x)y = 2x \circ (x \circ y) - x^2 \circ y.$$

Theorem 3.44 (Derivatives of the log-determinant). *Let V be a finite-dimensional Euclidean Jordan algebra with cone of squares K , and equip V with the trace inner product*

$$\langle x, y \rangle = \text{tr}(x \circ y).$$

Define

$$f : \text{int } K \rightarrow \mathbb{R}, \quad f(x) = \log \det(x).$$

Then

$$\nabla f(x) = x^{-1}$$

and

$$\nabla^2 f(x) = -Q(x)^{-1} = -Q(x^{-1}).$$

Theorem 3.45. *Let V be a Euclidean Jordan algebra with unit element e , and let $K \subset V$ be its symmetric cone. Then*

$$\{A \in \text{Aut}(K) : A = A^\top, A \succ 0\} = \{Q(w) : w \in \text{int } K\}.$$

Equivalently, an automorphism A of K is self-adjoint and positive definite if and only if there exists $w \in \text{int } K$ such that

$$A = Q(w).$$

Theorem 3.46. *Let V be a Euclidean Jordan algebra with unit element e , and let $K \subset V$ be its symmetric cone. For every $s, x \in \text{int } K$, there exists a unique operator*

$$W \in \text{Aut}(K)$$

such that for the canonical inner product:

$$W = W^\top, \quad W \succ 0, \quad Ws = x.$$

Moreover, W is a quadratic representation:

$$W = Q(w),$$

where the unique element $w \in \text{int } K$ is given by

$$w = Q(s^{-1/2}) \left(Q(s^{1/2}) x \right)^{1/2}$$

For each symmetric cone considered here, the canonical trace inner product is a positive scalar multiple of the standard inner product:

$$\langle x, y \rangle_{\text{tr}} = \gamma_K \langle x, y \rangle_{\text{std}},$$

where

$$\gamma_K = \begin{cases} 1, & K = \mathbb{R}_+^n, \\ 2, & K = \mathcal{L}^n, \\ 1, & K = \mathbb{S}_+^n. \end{cases}$$

For the semidefinite cone, the standard inner product is the Frobenius inner product

$$\langle X, Y \rangle_F = \text{Tr}(XY).$$

3.6 Convex functions

In this section we recall the definition of convex functions and some of their property.

Definition 3.47 (Extended real valued convex function). $f : \mathbb{R}^N \rightarrow \mathbb{R} \cup \{+\infty\}$ is *convex* if $f(\lambda v + (1-\lambda)w) \leq \lambda f(v) + (1-\lambda)f(w)$ for all v, w and all $\lambda \in (0, 1)$, with the convention that the inequality holds whenever the right side is $+\infty$. The function f is *concave* if $-f$ is convex.

Convexity is stable under several operations, among them addition and multiplication by a positive scalar. Two further facts are used .

Proposition 3.48 (Supremum and partial minimization). *Let $(f_\theta)_\theta$ be a family of convex functions. Then $h(v) = \sup_\theta f_\theta(v)$ is convex. If g is convex jointly in its two arguments, then the partial minimum $e(v) = \inf_u g(v, u)$ is convex. A pointwise infimum $v \mapsto \inf_\theta f_\theta(v)$ is not convex in general.*

Definition 3.49 (Domain and properness). $\text{dom } f = \{v : f(v) < +\infty\}$, and f is *proper* if $\text{dom } f \neq \emptyset$.

Definition 3.50 (Epigraph and lower semicontinuity). $\text{epi } f = \{(v, t) \in \mathbb{R}^N \times \mathbb{R} : f(v) \leq t\}$, and f is *lower semicontinuous* at v if $\liminf_{w \rightarrow v} f(w) \geq f(v)$.

Definition 3.51 (Closed function). f is *closed* if $\text{epi } f$ is closed, equivalently if f is lower semicontinuous everywhere.

Definition 3.52 (Coercivity). f is *coercive* if $f(v) \rightarrow +\infty$ whenever $\|v\| \rightarrow +\infty$.

Proposition 3.53 (Attainment under coercivity). *A closed proper coercive $f : \mathbb{R}^N \rightarrow \mathbb{R} \cup \{+\infty\}$ attains its infimum.*

Definition 3.54 (Strong convexity). f is *strongly convex with modulus $\sigma > 0$* in the metric $\mathcal{M} \succ 0$ if $f - \frac{\sigma}{2} \|\cdot\|_{\mathcal{M}}^2$ is convex, where $\|v\|_{\mathcal{M}}^2 = \langle v, \mathcal{M}v \rangle$. For $\mathcal{M} = I$ the metric is omitted.

Proposition 3.55 (Attainment under strong convexity). *A closed proper strongly convex f attains its infimum at exactly one point.*

Definition 3.56 (Indicator). $\delta_C(v) = 0$ for $v \in C$ and $\delta_C(v) = +\infty$ otherwise.

Given $v \in C$, the normal cone at v collects the directions making an obtuse angle with every direction pointing from v into C .

Definition 3.57 (Normal cone). For C convex and $v \in C$, $N_C(v) = \{z : \langle z, w - v \rangle \leq 0 \text{ for all } w \in C\}$.

Definition 3.58 (Subdifferential). For f convex,

$$\partial f(v) = \{z : f(w) \geq f(v) + \langle z, w - v \rangle \text{ for all } w\}. \quad (3.10)$$

Proposition 3.59 (Normal for a convex cone). *For $v \in K$, K convex cone, the normal cone at v is given by:*

$$N_K(v) = \{z \in K^\circ : \langle z, v \rangle = 0\} \quad (3.11)$$

Proof. Let $z \in N_K(v)$. For every $u \in K$, the conic property gives $v + u \in K$. Therefore,

$$\langle z, u \rangle = \langle z, (v + u) - v \rangle \leq 0.$$

Since this holds for every $u \in K$, we have $z \in K^\circ$.

Taking $w = 0$ in the definition of $N_K(v)$ gives

$$\langle z, -v \rangle \leq 0, \quad \text{and hence} \quad \langle z, v \rangle \geq 0.$$

On the other hand, $z \in K^\circ$ and $v \in K$ imply

$$\langle z, v \rangle \leq 0.$$

Consequently,

$$\langle z, v \rangle = 0.$$

Conversely, suppose that

$$z \in K^\circ \quad \text{and} \quad \langle z, v \rangle = 0.$$

Then, for every $w \in K$,

$$\langle z, w - v \rangle = \langle z, w \rangle - \langle z, v \rangle = \langle z, w \rangle \leq 0.$$

Thus, $z \in N_K(v)$. □

Proposition 3.60 (Singleton subdifferential). *For f convex, if $\partial f(v)$ is a singleton then f is differentiable at v .*

Proposition 3.61 (Subdifferential of the indicator). *For C closed convex and $v \in C$, $\partial \delta_C(v) = N_C(v)$, and $\partial \delta_C(v) = \emptyset$ for $v \notin C$.*

From the definition of the subdifferential it is immediate that

Proposition 3.62 (Fermat). *p minimises a convex f over $\mathbb{R}^N$ if and only if $0 \in \partial f(p)$.*

3.7 Differentiability

Definition 3.63 (Lipschitz). $\mathcal{R} : \mathbb{R}^m \rightarrow \mathbb{R}^p$ is *Lipschitz with constant L* if

$$\|\mathcal{R}(v) - \mathcal{R}(w)\| \leq L \|v - w\| \quad \text{for all } v, w, \quad (3.12)$$

and *locally Lipschitz* if every point has a neighbourhood on which it is Lipschitz with some constant.

Proposition 3.64 (Rademacher, [7]). *A locally Lipschitz $\mathcal{R} : \mathbb{R}^m \rightarrow \mathbb{R}^p$ is differentiable outside a set of Lebesgue measure zero.*

It is then natural to extend the definition of differentiability in the following way:

Definition 3.65 (Bouligand and Clarke Jacobians). Let $\mathcal{R} : \mathbb{R}^m \rightarrow \mathbb{R}^p$ be locally Lipschitz, and let $D_{\mathcal{R}}$ denote the set of points at which $\mathcal{R}$ is differentiable. The *Bouligand Jacobian* of $\mathcal{R}$ at v is

$$\partial_B \mathcal{R}(v) = \left\{ \lim_{k \rightarrow \infty} \mathcal{R}'(v_k) : v_k \rightarrow v, \quad v_k \in D_{\mathcal{R}} \right\}. \quad (3.13)$$

The *Clarke Jacobian* of $\mathcal{R}$ at v is the convex hull of its Bouligand Jacobian:

$$\partial_C \mathcal{R}(v) = \text{conv}(\partial_B \mathcal{R}(v)). \quad (3.14)$$

We write $\partial \mathcal{R}(v) = \partial_C \mathcal{R}(v)$.

3.8 Projection

Definition 3.66 (Projection). For $C \subseteq \mathbb{R}^m$ nonempty closed convex and $v \in \mathbb{R}^m$, $\Pi_C(v)$ is a $w \in C$ minimising $\|v - w\|$ over C .

Proposition 3.67 (Existence, uniqueness and characterisation). *For C nonempty closed convex the minimiser exists, is unique, and $p = \Pi_C(v)$ is characterised by*

$$p \in C, \quad \langle v - p, w - p \rangle \leq 0 \quad \text{for all } w \in C. \quad (3.15)$$

Proof. Consider the function

$$w \mapsto \frac{1}{2} \|v - w\|^2 + \delta_C(w).$$

Since C is nonempty, closed and convex, this function is closed, proper and strongly convex with modulus 1. The preceding attainment result therefore shows that it has a unique minimiser. Thus, $\Pi_C(v)$ exists and is unique.

Let $p = \Pi_C(v)$, and fix $w \in C$. By convexity of C ,

$$p + t(w - p) \in C \quad \text{for every } t \in [0, 1].$$

Consequently, the function

$$\varphi(t) = \frac{1}{2} \|v - p - t(w - p)\|^2$$

has a minimum over $[0, 1]$ at $t = 0$. Its right derivative at zero therefore satisfies

$$0 \leq \varphi'(0) = -\langle v - p, w - p \rangle.$$

Hence,

$$\langle v - p, w - p \rangle \leq 0 \quad \text{for every } w \in C.$$

Conversely, suppose that $p \in C$ satisfies

$$\langle v - p, w - p \rangle \leq 0 \quad \text{for every } w \in C.$$

Then, for every $w \in C$,

$$\begin{aligned} \|v - w\|^2 &= \|(v - p) - (w - p)\|^2 \\ &= \|v - p\|^2 - 2\langle v - p, w - p \rangle + \|w - p\|^2 \\ &\geq \|v - p\|^2. \end{aligned}$$

Thus, p minimises the distance from v to C , and therefore $p = \Pi_C(v)$. $\square$

Definition 3.68 (Firm nonexpansiveness). $T : \mathbb{R}^m \rightarrow \mathbb{R}^m$ is *firmly nonexpansive* if

$$\|T(v) - T(w)\|^2 \leq \langle T(v) - T(w), v - w \rangle \quad \text{for all } v, w. \quad (3.16)$$

Definition 3.69 (Monotone single-valued map). $T : \mathbb{R}^m \rightarrow \mathbb{R}^m$ is *monotone* if

$$\langle T(v) - T(w), v - w \rangle \geq 0 \quad \text{for all } v, w. \quad (3.17)$$

Proposition 3.70 (Firm nonexpansiveness implies Lipschitz and monotone). *A firmly nonexpansive T is Lipschitz with constant 1 and monotone.*

Proof. Monotonicity is immediate, the right side of the defining inequality being at least the left, which is nonnegative. For the Lipschitz bound, Cauchy–Schwarz gives $\|T(v) - T(w)\|^2 \leq \|T(v) - T(w)\| \|v - w\|$, and dividing by $\|T(v) - T(w)\|$ when it is nonzero gives the claim; when it is zero the claim is trivial. $\square$

Proposition 3.71 (Firm nonexpansiveness of the projection). $\Pi_C(\cdot)$ is firmly nonexpansive for every nonempty closed convex C .

Proof. Write $p = \Pi_C(v)$ and $q = \Pi_C(w)$. The characterisation at v with the test point q gives $\langle v - p, q - p \rangle \leq 0$, and at w with the test point p gives $\langle w - q, p - q \rangle \leq 0$. Adding them, $\langle (v - p) - (w - q), q - p \rangle \leq 0$, that is $\langle v - w, q - p \rangle \leq \langle p - q, q - p \rangle = -\|p - q\|^2$, which on negating is the required inequality. $\square$

Rademacher’s theorem therefore applies and the Clarke Jacobian is well defined. The following theorem links the projection on K with the projection on its polar, generalising the orthogonal decomposition along a subspace.

Proposition 3.72 (Moreau). *Let K be a closed convex cone. Every $v \in \mathbb{R}^m$ decomposes uniquely as*

$$v = \Pi_K(v) + \Pi_{K^\circ}(v), \quad \langle \Pi_K(v), \Pi_{K^\circ}(v) \rangle = 0. \quad (3.18)$$

Proof. Set

$$p = \Pi_K(v), \quad q = v - p.$$

By the characterisation of the projection,

$$q \in N_K(p) = K^\circ \cap p^\perp.$$

Consequently,

$$q \in K^\circ \quad \text{and} \quad \langle p, q \rangle = 0.$$

Since K is a closed convex cone, the bipolar theorem gives $(K^\circ)^\circ = K$. Therefore,

$$v - q = p \in K \cap q^\perp = (K^\circ)^\circ \cap q^\perp = N_{K^\circ}(q).$$

The characterisation of the projection now gives

$$q = \Pi_{K^\circ}(v).$$

Hence,

$$v = \Pi_K(v) + \Pi_{K^\circ}(v), \quad \langle \Pi_K(v), \Pi_{K^\circ}(v) \rangle = 0.$$

For uniqueness, suppose that

$$v = p' + q', \quad p' \in K, \quad q' \in K^\circ, \quad \langle p', q' \rangle = 0.$$

Then

$$v - p' = q' \in K^\circ \cap (p')^\perp = N_K(p').$$

Thus,

$$p' = \Pi_K(v), \quad q' = v - p' = \Pi_{K^\circ}(v).$$

$\square$

Proposition 3.73. *Let K be a nonempty closed convex cone, and define*

$$f(x) = \frac{1}{2} \|\Pi_K(x)\|^2.$$

Then f is continuously differentiable and

$$\nabla f(x) = \Pi_K(x).$$

Proof. Set $p = \Pi_K(x)$. The Moreau decomposition gives $\langle x, p \rangle = \|p\|^2$. Moreover, by completing the square, p is the unique maximiser over $y \in K$ of $\langle x, y \rangle - \frac{1}{2}\|y\|^2$. Consequently, $f(x) = \max_{y \in K} \{\langle x, y \rangle - \frac{1}{2}\|y\|^2\}$.

Now let $p_h = \Pi_K(x+h)$. Using p as a candidate at $x+h$ gives $f(x+h) \geq f(x) + \langle p, h \rangle$. On the other hand, using the optimality of p at x gives $f(x+h) \leq f(x) + \langle p_h, h \rangle$. Therefore,

$$0 \leq f(x+h) - f(x) - \langle p, h \rangle \leq \langle p_h - p, h \rangle \leq \|p_h - p\| \|h\| \leq \|h\|^2,$$

where the last inequality follows from the 1-Lipschitz continuity of the projection. Hence $f(x+h) - f(x) - \langle p, h \rangle = o(\|h\|)$, which proves that $\nabla f(x) = p = \Pi_K(x)$. $\square$

Proposition 3.74 (Distance in terms of a projection). *For K a closed convex cone,*

$$\text{dist}(v, K) = \|\Pi_{K^\circ}(v)\| = \|\Pi_{K^*}(-v)\|, \quad (3.19)$$

so $v \in K$ if and only if $\Pi_{K^}(-v) = 0$.*

Proposition 3.75 (The Clarke Jacobian of a projection). *Let K be a closed convex cone and $F \in \partial \Pi_{K^*}(\cdot)(\sigma)$. Then $F = F^\top$ and $0 \preceq F \preceq I$.*

Proof. Set $P = \Pi_{K^*}(\cdot)$. From the preceding proposition,

$$P = \nabla \phi, \quad \phi(x) = \frac{1}{2} \|P(x)\|^2,$$

and ϕ is convex. Moreover, the Moreau decomposition gives

$$I - P = \Pi_{(K^*)^\circ}(\cdot) = \Pi_{-K}(\cdot).$$

Therefore,

$$I - P = \nabla \psi, \quad \psi(x) = \frac{1}{2} \|\Pi_{-K}(x)\|^2,$$

where ψ is also convex.

Consider a point x at which P is differentiable, and write $J = DP(x)$. Since $P = \nabla \phi$, the matrix $J = \nabla^2 \phi(x)$ is symmetric and positive semidefinite. Hence

$$J = J^\top, \quad J \succeq 0.$$

Similarly, $I - P = \nabla \psi$ is differentiable at x , with derivative $I - J$. Since ψ is convex,

$$I - J \succeq 0.$$

Thus, at every point where P is differentiable,

$$J = J^\top, \quad 0 \preceq J \preceq I.$$

The projection P is Lipschitz and is therefore differentiable almost everywhere. By definition, every matrix in the Clarke generalized Jacobian $\partial P(\sigma)$ is a convex combination of limits of matrices $DP(x_k)$, where $x_k \rightarrow \sigma$ and P is differentiable at every x_k . Since the set

$$\{A : A = A^\top, 0 \preceq A \preceq I\}$$

is closed and convex, these properties are preserved under limits and convex combinations. Consequently, every $F \in \partial P(\sigma)$ satisfies

$$F = F^\top, \quad 0 \preceq F \preceq I.$$

□

Proposition 3.76 (Action on the argument). *$F\sigma = \Pi_K(\sigma)$ for every $F \in \partial \Pi_K(\cdot)(\sigma)$ and every σ .*

Proof. Set $P = \Pi_K(\cdot)$. Since K is a cone, its projection is positively homogeneous:

$$P(tx) = tP(x) \quad \text{for every } t \geq 0.$$

At any point x where P is differentiable, differentiating this identity with respect to t at $t = 1$ gives

$$DP(x)x = P(x).$$

Now let J be a limiting Jacobian of P at σ . There is a sequence $x_k \rightarrow \sigma$ such that P is differentiable at every x_k and $DP(x_k) \rightarrow J$. Since P is 1-Lipschitz, $\|DP(x_k)\| \leq 1$. Therefore,

$$\begin{aligned} \|DP(x_k)\sigma - P(\sigma)\| &\leq \|DP(x_k)(\sigma - x_k)\| + \|DP(x_k)x_k - P(\sigma)\| \\ &= \|DP(x_k)(\sigma - x_k)\| + \|P(x_k) - P(\sigma)\| \\ &\leq 2\|x_k - \sigma\|. \end{aligned}$$

Letting $k \rightarrow \infty$ gives

$$J\sigma = P(\sigma).$$

Finally, the Clarke generalized Jacobian is the convex hull of such limiting Jacobians. Since the identity $J\sigma = P(\sigma)$ is preserved under limits and convex combinations, every $F \in \partial P(\sigma)$ satisfies

$$F\sigma = P(\sigma) = \Pi_K(\sigma).$$

□

Replacing σ by his projection on K and using the fact that $\Pi_K \circ \Pi_K = \Pi_K$, we deduce that:

Proposition 3.77 (Fixed point formulation). *$p = \Pi_K(\sigma)$ then:*

$$Fp = p$$

A natural question is which metrics leave the Euclidean projection onto K unchanged.

Definition 3.78 (Admissible metric). Let $K \subseteq \mathbb{R}^n$ be a nonempty closed convex cone, and let $M \in \mathbb{S}_{++}^n$. Define

$$\langle x, y \rangle_M = \langle x, My \rangle, \quad \|x\|_M^2 = \langle x, Mx \rangle.$$

The projection onto K in the metric M is

$$\Pi_K(v)^M = \operatorname{argmin}_{p \in K} \frac{1}{2} \|v - p\|_M^2.$$

The metric M is said to be *admissible for K* if

$$\Pi_K(v)^M = \Pi_K(v) \quad \text{for every } v \in \mathbb{R}^n.$$

Definition 3.79 (Polar cone in the metric M). The polar cone of K in the metric M is

$$K^{\circ_M} = \{z \in \mathbb{R}^n : \langle z, w \rangle_M \leq 0 \text{ for every } w \in K\}.$$

Equivalently,

$$K^{\circ_M} = M^{-1}K^\circ.$$

Proposition 3.80 (Characterisation of the metric projection). Let $p_M \in \mathbb{R}^n$. Then

$$p_M = \Pi_K(v)^M$$

if and only if

$$p_M \in K, \quad v - p_M \in K^{\circ_M}, \quad \langle p_M, v - p_M \rangle_M = 0.$$

Equivalently,

$$p_M \in K, \quad M(v - p_M) \in K^\circ, \quad \langle p_M, M(v - p_M) \rangle = 0.$$

There is a simple characterisation of admissibility.

Theorem 3.81 (Commutation characterisation of admissibility). Let $K \subseteq \mathbb{R}^n$ be a nonempty closed convex cone and let $M \in \mathbb{S}_{++}^n$. Then M is admissible for K if and only if

$$M\Pi_K(v) = \Pi_K(Mv) \quad \text{for every } v \in \mathbb{R}^n.$$

Proof. Write $P = \Pi_K(\cdot)$. Recall that the metric projection is characterised by

$$p = \Pi_K(v)^M \iff p \in K, \quad M(v - p) \in K^\circ, \quad \langle p, M(v - p) \rangle = 0.$$

Suppose first that M is admissible. Let $p \in K$ and $r \in N_K(p)$. Since

$$p = P(p + r) = \Pi_K(p + r)^M,$$

the characterisation of the metric projection gives

$$Mr \in N_K(p).$$

Thus,

$$MN_K(p) \subseteq N_K(p) \quad \text{for every } p \in K.$$

In particular, taking $p = 0$ gives

$$MK^\circ \subseteq K^\circ.$$

Since M is symmetric, this also implies $MK \subseteq K$. Indeed, for $x \in K$ and $z \in K^\circ$,

$$\langle Mx, z \rangle = \langle x, Mz \rangle \leq 0.$$

Now let

$$p = P(v), \quad q = v - p.$$

Then $q \in N_K(p)$. Consequently,

$$Mq \in N_K(p) \quad \text{and} \quad M^2q \in N_K(p).$$

It follows that $Mq \in K^\circ$ and

$$\langle Mp, Mq \rangle = \langle p, M^2q \rangle = 0.$$

Since $Mp \in K$ and

$$Mv = Mp + Mq,$$

the Moreau characterisation gives

$$P(Mv) = Mp = MP(v).$$

Conversely, suppose that

$$P(Mv) = MP(v) \quad \text{for every } v.$$

Since $\Pi_{K^\circ}(v) = v - P(v)$, we also have

$$\Pi_{K^\circ}(Mv) = M\Pi_{K^\circ}(v).$$

Fix v , and set

$$p = P(v), \quad q = \Pi_{K^\circ}(v) = v - p.$$

By repeated application of the commutation identity, for every integer $k \geq 0$,

$$P(M^k v) = M^k p, \quad \Pi_{K^\circ}(M^k v) = M^k q.$$

The Moreau decomposition therefore gives

$$0 = \langle M^k p, M^k q \rangle = \langle p, M^{2k} q \rangle.$$

Let $\mu_1, \dots, \mu_s$ be the distinct eigenvalues of M , and let $E_1, \dots, E_s$ be the corresponding orthogonal spectral projectors. Writing

$$a_j = \langle p, E_j q \rangle,$$

the preceding identities for $k = 0, \dots, s-1$ become

$$\sum_{j=1}^s \mu_j^{2k} a_j = 0.$$

Since the numbers μ_j^2 are distinct, the corresponding Vandermonde matrix is invertible. Hence $a_j = 0$ for every j , and therefore

$$\langle p, Mq \rangle = \sum_{j=1}^s \mu_j a_j = 0.$$

Moreover,

$$Mq = \Pi_{K^\circ}(Mv) \in K^\circ.$$

Thus,

$$p \in K, \quad M(v - p) = Mq \in K^\circ, \quad \langle p, M(v - p) \rangle = 0.$$

The characterisation of the metric projection gives

$$p = \Pi_K(v)^M.$$

Since $p = P(v)$, the metric M is admissible. $\square$

By Moreau we have immediately that:

Proposition 3.82 (The three forms of admissibility). *For $M \succ 0$ the three conditions $M\Pi_K(\cdot) = \Pi_K(M\cdot)$, $M\Pi_{K^*}(\cdot) = \Pi_{K^*}(M\cdot)$ and $M\Pi_{K^\circ}(\cdot) = \Pi_{K^\circ}(M\cdot)$ are equivalent.*

Below we give some property of admissible metric we will use.

Proposition 3.83 (Structure and closure of admissible metrics). *Let K be a closed convex cone and let $M \in \mathcal{A}(K)$. Then:*

1. *The cones are invariant under M :*

$$MK = K$$

In particular, $M \in \text{Aut}(K)$.

2. *For every σ and every $F \in \partial\Pi_{K^*}(\cdot)(\sigma)$,*

$$MF = FM.$$

Consequently,

$$M^{-1}F = (M^{-1}F)^\top, \quad 0 \preceq M^{-1}F \preceq M^{-1}.$$

3. *The set $\mathcal{A}(K)$ is closed under positive linear combinations, inversion and positive-definite limits. If $M, N \in \mathcal{A}(K)$ and $MN = NM$, then*

$$MN \in \mathcal{A}(K).$$

Thus, $\mathcal{A}(K) \cup \{0\}$ is a convex cone.

4. *For every $\alpha \in \mathbb{R}$,*

$$M^\alpha \in \mathcal{A}(K).$$

Proof. We first record a useful characterisation. For every $p \in K$,

$$M \in \mathcal{A}(K) \iff MN_K(p) \subseteq N_K(p).$$

Indeed, if $q \in N_K(p)$, then

$$p = \Pi_K(p + q).$$

Admissibility and the characterisation of the metric projection imply

$$Mq \in N_K(p).$$

Conversely, if $MN_K(p) \subseteq N_K(p)$ for every $p \in K$, then, for $p = \Pi_K(v)$ and $q = v - p \in N_K(p)$, one has $Mq \in N_K(p)$. Hence $p = \Pi_K(v)^M$, so M is admissible.

Convexity. Let $M, N \in \mathcal{A}(K)$ and let $\alpha, \beta > 0$. For every $p \in K$ and $q \in N_K(p)$,

$$Mq \in N_K(p), \quad Nq \in N_K(p).$$

Since $N_K(p)$ is a convex cone,

$$(\alpha M + \beta N)q = \alpha Mq + \beta Nq \in N_K(p).$$

Therefore,

$$\alpha M + \beta N \in \mathcal{A}(K).$$

Thus, $\mathcal{A}(K)$ is closed under positive linear combinations.

Inversion and invariance of the cones. Write $P = \Pi_K(\cdot)$. By the commutation characterisation,

$$MP(v) = P(Mv).$$

Replacing v by $M^{-1}v$ gives

$$P(M^{-1}v) = M^{-1}P(v),$$

so

$$M^{-1} \in \mathcal{A}(K).$$

If $v \in K$, then $P(v) = v$, and therefore

$$P(Mv) = MP(v) = Mv.$$

Thus, $Mv \in K$, which gives $MK \subseteq K$. Applying the same argument to M^{-1} gives the reverse inclusion, so

$$MK = K.$$

Consequently, $M \in \text{Aut}(K)$.

Products and limits. Let $M, N \in \mathcal{A}(K)$ and suppose that $MN = NM$. Then MN is symmetric positive definite, and

$$MNP(v) = MP(Nv) = P(MNv).$$

Hence,

$$MN \in \mathcal{A}(K).$$

Now suppose that $M_j \in \mathcal{A}(K)$ and

$$M_j \longrightarrow M \succ 0.$$

Using the continuity of the projection,

$$MP(v) = \lim_{j \rightarrow \infty} M_j P(v) = \lim_{j \rightarrow \infty} P(M_j v) = P(Mv).$$

Therefore, $M \in \mathcal{A}(K)$.

Commutation with generalized Jacobians. Set

$$P_* = \Pi_{K^*}(\cdot).$$

Let x be a point at which P_* is differentiable, and write

$$J = DP_*(x).$$

Since

$$P_*(y) = MP_*(M^{-1}y),$$

the projection is differentiable at Mx , with

$$J' = DP_*(Mx) = MJM^{-1}.$$

Both J and J' are symmetric. Therefore,

$$MJM^{-1} = J' = (J')^\top = M^{-1}JM,$$

and hence

$$M^2J = JM^2.$$

Since $M \succ 0$, the matrices M and M^2 have the same eigenspaces. It follows that

$$MJ = JM.$$

This relation is preserved under limits and convex combinations. Therefore, for every σ and every $F \in \partial P_*(\sigma)$,

$$MF = FM.$$

Since $F = F^\top$ and $0 \preceq F \preceq I$, simultaneous diagonalisation gives

$$M^{-1}F = (M^{-1}F)^\top, \quad 0 \preceq M^{-1}F \preceq M^{-1}.$$

Square roots and real powers. We first prove that $M^{1/2}$ is admissible. Consider the Newton iteration

$$X_0 = I, \quad X_{j+1} = \frac{1}{2} (X_j + MX_j^{-1}).$$

Every X_j is a positive-definite rational function of M and therefore commutes with M . Suppose that X_j is admissible. Then X_j^{-1} is admissible, and the commuting-product property gives

$$MX_j^{-1} \in \mathcal{A}(K).$$

Closure under positive linear combinations then gives

$$X_{j+1} \in \mathcal{A}(K).$$

Since $X_0 = I$ is admissible, every X_j is admissible.

Diagonalising M reduces this iteration to the scalar Newton iteration for the positive square root. Hence,

$$X_j \longrightarrow M^{1/2}.$$

Closure under positive-definite limits gives

$$M^{1/2} \in \mathcal{A}(K).$$

Repeated square roots, commuting products and inversion now give

$$M^{m/2^r} \in \mathcal{A}(K)$$

for every $m \in \mathbb{Z}$ and every nonnegative integer r .

Let $\alpha \in \mathbb{R}$, and choose dyadic rational numbers α_j such that $\alpha_j \rightarrow \alpha$. We then have,

$$M^{\alpha_j} \rightarrow M^\alpha.$$

Therefore,

$$M^\alpha \in \mathcal{A}(K).$$

□

Proposition 3.84 (Weighted squared norm of the projection). *Let $K \subseteq \mathbb{R}^n$ be a closed convex cone and let $M \in \mathcal{A}(K)$. Define*

$$\phi_M(x) = \frac{1}{2} \|\Pi_K(x)\|_M^2.$$

Then ϕ_M is convex and continuously differentiable, with

$$\nabla \phi_M(x) = M\Pi_K(x).$$

Equivalently,

$$\partial \phi_M(x) = \{M\Pi_K(x)\}.$$

Proof. Since M is admissible,

$$\Pi_K(x)^M = \Pi_K(x).$$

Completing the square gives

$$\langle Mx, y \rangle - \frac{1}{2} \|y\|_M^2 = \frac{1}{2} \|x\|_M^2 - \frac{1}{2} \|x - y\|_M^2.$$

Consequently,

$$\Pi_K(x) = \operatorname{argmax}_{y \in K} \left\{ \langle Mx, y \rangle - \frac{1}{2} \|y\|_M^2 \right\}.$$

If $p = \Pi_K(x)$, metric complementarity gives

$$\langle Mp, x - p \rangle = 0,$$

and hence

$$\phi_M(x) = \max_{y \in K} \left\{ \langle Mx, y \rangle - \frac{1}{2} \|y\|_M^2 \right\}.$$

This representation also shows that ϕ_M is convex.

Now let

$$p = \Pi_K(x), \quad p_h = \Pi_K(x + h).$$

Using p as a candidate at $x + h$ gives

$$\phi_M(x + h) \geq \phi_M(x) + \langle Mp, h \rangle.$$

Using the optimality of p at x and the maximiser p_h at $x + h$ gives

$$\phi_M(x + h) \leq \phi_M(x) + \langle Mp_h, h \rangle.$$

Therefore,

$$\begin{aligned}
0 &\leq \phi_M(x+h) - \phi_M(x) - \langle Mp, h \rangle \\
&\leq \langle M(p_h - p), h \rangle \\
&\leq \|p_h - p\|_M \|h\|_M \\
&\leq \|h\|_M^2,
\end{aligned}$$

where the last inequality follows from the nonexpansiveness of the metric projection. Thus,

$$\phi_M(x+h) - \phi_M(x) - \langle Mp, h \rangle = o(\|h\|),$$

which proves

$$\nabla \phi_M(x) = Mp = M\Pi_K(x).$$

Continuity of the gradient follows from continuity of the projection. $\square$

Proposition 3.85 (contraction formula). *Let $P = \Pi_{K^*}$, let $M \in \mathcal{A}(K^*)$, and suppose that P is twice differentiable at σ . Set*

$$p = P(\sigma) \quad \text{and} \quad F = DP(\sigma).$$

Then

$$D^2P(\sigma)[h, Mp] = MF(I - F)h \quad \text{for every } h \in \mathbb{R}^m. \quad (3.20)$$

Equivalently,

$$D^2P(\sigma)[\cdot, Mp] = MF(I - F). \quad (3.21)$$

Proof. At every point x at which P is differentiable, the derivative of the projection satisfies

$$DP(x)P(x) = P(x). \quad (3.22)$$

Differentiating this identity at $x = \sigma$ in the direction h gives

$$D^2P(\sigma)[h, p] + F^2h = Fh.$$

It follows that

$$D^2P(\sigma)[h, p] = F(I - F)h. \quad (3.23)$$

Since M is admissible, it commutes with the derivative of P :

$$DP(x)M = MDP(x)$$

at every point where P is differentiable. Differentiating this identity at σ in the direction h , and then applying the resulting identity to p , gives

$$D^2P(\sigma)[h, Mp] = MD^2P(\sigma)[h, p].$$

Using the preceding formula, we obtain

$$D^2P(\sigma)[h, Mp] = MF(I - F)h.$$

$\square$

Proposition 3.86 (Diagonal admissible metrics on a product). *Let*

$$K = K_1 \times \cdots \times K_p$$

be a product of cones from the catalogue, and let

$$M = \text{diag}(D_1, \dots, D_p) \succ 0,$$

where each D_j is diagonal in the coordinates of the corresponding block. Then $M \in \mathcal{A}(K)$ if and only if D_j is a positive multiple of the identity on every block, except on zero-cone and nonnegative-orthant blocks, where any positive diagonal matrix is allowed.

Proof. Since

$$K^* = K_1^* \times \cdots \times K_p^*$$

and the projection onto K^* acts blockwise, we have

$$\Pi_{K^*}(v_1, \dots, v_p) = (\Pi_{K_1^*}(v_1), \dots, \Pi_{K_p^*}(v_p)).$$

Consequently, M is admissible for K if and only if D_j is admissible for K_j for every j . It is therefore enough to consider one block.

For a zero-cone block, the projection is constant, while the projection onto its dual is the identity. Hence every positive diagonal matrix is admissible.

For a nonnegative-orthant block,

$$\Pi_{\mathbb{R}_+^d}(v) = \max(v, 0)$$

componentwise. If $D \succ 0$ is diagonal, then

$$\Pi_{\mathbb{R}_+^d}(Dv) = \max(Dv, 0) = D \max(v, 0) = D \Pi_{\mathbb{R}_+^d}(v).$$

Thus every positive diagonal matrix is admissible.

Now consider any other cone C from the catalogue. Such a cone admits a boundary point x whose normal cone is a ray

$$N_C(x) = \mathbb{R}_+ n,$$

where every coordinate of n is nonzero. If D is admissible, the normal-cone characterization of admissibility gives

$$DN_C(x) \subseteq N_C(x).$$

In particular, $Dn \in \mathbb{R}_+ n$, so there exists $\lambda > 0$ such that

$$Dn = \lambda n.$$

Writing $D = \text{diag}(d_1, \dots, d_d)$, we obtain

$$d_i n_i = \lambda n_i \quad \text{for every } i.$$

Since every n_i is nonzero,

$$d_i = \lambda \quad \text{for every } i.$$

Therefore,

$$D = \lambda I.$$

Such a full-support normal can be chosen on every Lorentz, semidefinite, exponential, and power-cone block. For example, on a Lorentz block one may take

$$x = (1, u), \quad \|u\| = 1,$$

with every coordinate of u nonzero. Then

$$N_C(x) = \mathbb{R}_+(-1, u),$$

whose generator has no zero coordinate. The other blocks are handled by choosing a generic smooth boundary point, whose normal likewise has full support.

Conversely, every positive multiple of the identity commutes with a positively homogeneous projection:

$$\Pi_{C^*}(\lambda v) = \lambda \Pi_{C^*}(v), \quad \lambda > 0.$$

Hence every block matrix of the stated form is admissible, and therefore so is their block-diagonal product. $\square$

Chapter 4

Convex program, Lagrangian and KKT

This chapter introduces what are convex program and the KKT conditions: a set of relations equivalent to optimality that guides the design of every algorithm that follows.

4.1 Convex program and Lagrangian

Definition 4.1 (Convex program). Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be convex, let $c : \mathbb{R}^n \rightarrow \mathbb{R}^m$ be affine, written $c(x) = Cx + d$, and let $K \subseteq \mathbb{R}^m$ be a closed convex cone. The problem is

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to} \quad c(x) \preceq_K 0. \quad (4.1)$$

Its optimal value is written p^* , and a point satisfying the constraint is called *feasible*.

The problem the solver addresses is a conic quadratic program.

Definition 4.2 (Conic quadratic program). The *conic quadratic program* is the particular case in which f is the convex quadratic

$$f(x) = \frac{1}{2} \langle x, Px \rangle + \langle q, x \rangle, \quad P \in \mathcal{S}^n, P \succeq 0, q \in \mathbb{R}^n, \quad (4.2)$$

so that f is differentiable with $\nabla f(x) = Px + q$. This is the problem carried by the rest of the document.

Definition 4.3 (Lagrangian).

$$L(x, z) = f(x) + \langle z, c(x) \rangle, \quad z \in K^*. \quad (4.3)$$

Proposition 4.4 (Dual function). *Let*

$$g(z) = \inf_x L(x, z), \quad z \in K^*.$$

Then g is concave and upper semicontinuous. In the conic quadratic case, $g(z) > -\infty$ if and only if

$$Px + q + C^\top z = 0 \quad (4.4)$$

admits a solution x_z . In that case,

$$g(z) = -\frac{1}{2} \langle x_z, Px_z \rangle + \langle z, d \rangle, \quad (4.5)$$

and this value is independent of the choice of x_z .

Proof. For every fixed x , the mapping

$$z \mapsto L(x, z) = f(x) + \langle z, Cx + d \rangle$$

is affine and continuous. Since a pointwise infimum of affine functions is concave, g is concave. Moreover, for every $\alpha \in \mathbb{R}$,

$$\{z : g(z) < \alpha\} = \bigcup_x \{z : L(x, z) < \alpha\},$$

which is open. Hence g is upper semicontinuous.

In the conic quadratic case,

$$L(x, z) = \frac{1}{2} \langle x, Px \rangle + \langle q + C^\top z, x \rangle + \langle z, d \rangle,$$

where $P \succeq 0$. Set

$$b_z = q + C^\top z.$$

If there exists x_z satisfying

$$Px_z + b_z = 0,$$

then, for every h ,

$$L(x_z + h, z) = L(x_z, z) + \frac{1}{2} \langle h, Ph \rangle.$$

Since $P \succeq 0$, this shows that x_z is a global minimizer and that $g(z) = L(x_z, z) > -\infty$.

Conversely, suppose that (4.4) has no solution. Since P is symmetric,

$$\text{range}(P) = \ker(P)^\perp,$$

so $b_z \notin \text{range}(P)$. There therefore exists $h \in \ker(P)$ such that

$$\langle b_z, h \rangle \neq 0.$$

Replacing h by $-h$, if necessary, we may assume that $\langle b_z, h \rangle < 0$. Then

$$L(th, z) = t \langle b_z, h \rangle + \langle z, d \rangle \longrightarrow -\infty \quad \text{as } t \rightarrow +\infty.$$

Thus $g(z) = -\infty$.

Finally, if x_z satisfies $Px_z + b_z = 0$, then

$$g(z) = L(x_z, z) \tag{4.6}$$

$$= \frac{1}{2} \langle x_z, Px_z \rangle + \langle b_z, x_z \rangle + \langle z, d \rangle \tag{4.7}$$

$$= -\frac{1}{2} \langle x_z, Px_z \rangle + \langle z, d \rangle. \tag{4.8}$$

If x'_z is another solution, then

$$P(x_z - x'_z) = 0.$$

Consequently,

$$Px_z = Px'_z$$

and

$$\langle x_z, Px_z \rangle = \langle x'_z, Px'_z \rangle.$$

Therefore, the displayed value of $g(z)$ does not depend on the chosen solution. $\square$

Definition 4.5 (Dual problem).

$$\max_{z \in K^*} g(z), \quad (4.9)$$

with optimal value g^* . A $z \in K^*$ at which $g(z)$ is finite is *dual feasible*.

Proposition 4.6 (Weak duality). *For every primal feasible point x and every dual feasible point z ,*

$$g(z) \leq f(x).$$

Consequently,

$$g^* \leq p^*.$$

Proof. Let x be primal feasible and z be dual feasible. Then

$$-c(x) \in K \quad \text{and} \quad z \in K^*.$$

By the definition of the dual cone,

$$\langle z, -c(x) \rangle \geq 0,$$

and hence

$$\langle z, c(x) \rangle \leq 0.$$

Using the definition of the dual function, we obtain

$$g(z) = \inf_u L(u, z) \quad (4.10)$$

$$\leq L(x, z) \quad (4.11)$$

$$= f(x) + \langle z, c(x) \rangle \quad (4.12)$$

$$\leq f(x). \quad (4.13)$$

Taking the supremum over all dual feasible z , and then the infimum over all primal feasible x , gives

$$g^* = \sup_{z \in K^*} g(z) \leq \inf_{-c(x) \in K} f(x) = p^*.$$

□

Definition 4.7 (Duality gap). The *duality gap* of a feasible x and a dual feasible z is $f(x) - g(z) \geq 0$. The *optimal duality gap* is $p^* - g^* \geq 0$.

Definition 4.8 (Strong duality). *Strong duality* holds when $p^* = g^*$.

Strong duality does not hold unconditionally; it requires a qualification condition. A classical one follows.

Definition 4.9 (Slater condition). The problem satisfies the *Slater condition* if there is x with $-c(x) \in \text{relint } K$.

Proposition 4.10 (Slater implies strong duality). *If the problem is bounded below and satisfies the Slater condition, then $p^* = g^*$ and the supremum defining g^* is attained.*

4.2 KKT conditions

In what follows the problem is solved through a system of relations equivalent to optimality whenever strong duality holds.

Proposition 4.11 (Karush, Kuhn and Tucker conditions). *Assume that p^* is finite, that strong duality holds,*

$$p^* = g^*,$$

and that the dual supremum is attained. Then a feasible point x^ is optimal if and only if there exists z^* satisfying*

$$0 \in \partial f(x^*) + C^\top z^*, \quad \text{stationarity,} \quad (4.14)$$

$$-c(x^*) \in K, \quad \text{primal feasibility,} \quad (4.15)$$

$$z^* \in K^*, \quad \text{dual feasibility,} \quad (4.16)$$

$$\langle z^*, c(x^*) \rangle = 0, \quad \text{complementarity.} \quad (4.17)$$

For $z^* \in K^*$, primal feasibility and complementarity are equivalent to

$$c(x^*) \in N_{K^*}(z^*). \quad (4.18)$$

Thus the KKT conditions can equivalently be written as

$$0 \in \partial f(x^*) + C^\top z^*, \quad c(x^*) \in N_{K^*}(z^*). \quad (4.19)$$

Proof. Suppose first that x^* is primal optimal, and let z^* be a dual optimal point. Strong duality gives

$$f(x^*) = p^* = g^* = g(z^*).$$

By the definition of the dual function,

$$g(z^*) \leq L(x^*, z^*).$$

We have:

$$f(x^*) = g(z^*) \leq L(x^*, z^*) \leq f(x^*).$$

Hence all the inequalities are equalities. In particular,

$$\langle z^*, c(x^*) \rangle = 0.$$

The equality

$$g(z^*) = L(x^*, z^*)$$

also shows that x^* minimizes the convex function $L(\cdot, z^*)$. Fermat's rule therefore gives

$$0 \in \partial_x L(x^*, z^*).$$

Since

$$L(x, z^*) = f(x) + \langle z^*, Cx + d \rangle,$$

the subdifferential sum rule with an affine function gives

$$\partial_x L(x^*, z^*) = \partial f(x^*) + C^\top z^*.$$

Consequently,

$$0 \in \partial f(x^*) + C^\top z^*.$$

The other 2 conditions are simply admissibility. Thus all four KKT conditions hold.

Conversely, suppose that x^* and z^* satisfy the four KKT conditions. The stationarity condition and the affine subdifferential rule give

$$0 \in \partial_x L(x^*, z^*).$$

Since $L(\cdot, z^*)$ is convex, Fermat's rule implies that x^* minimizes $L(\cdot, z^*)$. Therefore,

$$g(z^*) = L(x^*, z^*).$$

Using complementarity, we obtain

$$g(z^*) = f(x^*) + \langle z^*, c(x^*) \rangle = f(x^*).$$

Since x^* is primal feasible and z^* is dual feasible, weak duality gives

$$g(z^*) \leq g^* \leq p^* \leq f(x^*).$$

The two endpoints are equal, so equality holds throughout:

$$g(z^*) = g^* = p^* = f(x^*).$$

Therefore, x^* is primal optimal and z^* is dual optimal.

It remains to verify the normal-cone formulation. Since K is a closed convex cone,

$$(K^*)^\circ = -K.$$

For $z^* \in K^*$, the normal cone to K^* is therefore

$$N_{K^*}(z^*) = (K^*)^\circ \cap (z^*)^\perp = -K \cap (z^*)^\perp.$$

Consequently,

$$c(x^*) \in N_{K^*}(z^*)$$

is equivalent to

$$-c(x^*) \in K \quad \text{and} \quad \langle z^*, c(x^*) \rangle = 0.$$

This proves the equivalent normal-cone formulation. $\square$

Corollary 4.12 (The differentiable and the quadratic cases). *If f is differentiable at x^* then $\partial f(x^*) = \{\nabla f(x^*)\}$ and the first condition reads*

$$\nabla f(x^*) + C^\top z^* = 0, \tag{4.20}$$

which for the conic quadratic program is

$$Px^* + q + C^\top z^* = 0. \tag{4.21}$$

Chapter 5

Monotone operator and proximal representation

This chapter introduces monotone operators and proves some of their properties. They are useful because they supply a theory from which the convergence of a great many algorithms follows directly, by reading each as a realisation of the abstract proximal point algorithm applied to finding their zeros.

5.1 Set-valued operators

Definition 5.1 (Set-valued operator). A *set-valued operator* on $\mathbb{R}^N$ is a subset $T \subseteq \mathbb{R}^N \times \mathbb{R}^N$. One writes

$$T(u) = \{v : (u, v) \in T\}, \quad \text{gph } T = T, \quad T^{-1} = \{(v, u) : (u, v) \in T\}, \quad (5.1)$$

and $\text{dom } T = \{u : T(u) \neq \emptyset\}$.

For such an operator, monotonicity is the natural generalisation of a nondecreasing map on the line, and of the gradient of a convex function.

Definition 5.2 (Monotone operator). T is *monotone* if

$$\langle v - v', u - u' \rangle \geq 0 \quad \text{whenever } (u, v) \in T \text{ and } (u', v') \in T. \quad (5.2)$$

Definition 5.3 (Order on operators). $T_1 \subseteq T_2$ means containment of the graphs.

Definition 5.4 (Maximal monotone operator). A monotone T is *maximal monotone* if the only monotone T' with $T \subseteq T'$ is T itself.

Proposition 5.5 (Closed graph). *Let $T : \mathbb{R}^n \rightrightarrows \mathbb{R}^n$ be maximal monotone. Then $\text{gra } T$ is closed.*

Proof. Let (u_k, v_k) be a sequence in $\text{gra } T$ such that

$$(u_k, v_k) \longrightarrow (u, v).$$

For every $(u', v') \in \text{gra } T$, the monotonicity of T gives

$$\langle v_k - v', u_k - u' \rangle \geq 0.$$

Passing to the limit as $k \rightarrow \infty$, we obtain

$$\langle v - v', u - u' \rangle \geq 0 \quad \text{for every } (u', v') \in \text{gra } T.$$

Thus the pair (u, v) is monotonically related to every point of $\text{gra } T$. Consequently,

$$\text{gra } T \cup \{(u, v)\}$$

is the graph of a monotone extension of T . Since T is maximal monotone, it admits no proper monotone extension. Therefore,

$$(u, v) \in \text{gra } T.$$

Hence $\text{gra } T$ is closed. □

Proposition 5.6 (Affine operators with positive semidefinite symmetric part). *Let $Q \in \mathbb{R}^{N \times N}$ satisfy*

$$\langle Qu, u \rangle \geq 0 \quad \text{for every } u \in \mathbb{R}^N,$$

and let $b \in \mathbb{R}^N$. Then the affine operator

$$T(u) = Qu + b$$

is maximal monotone and has domain $\mathbb{R}^N$.

Proof. For any $u, u' \in \mathbb{R}^N$,

$$\langle T(u) - T(u'), u - u' \rangle = \langle Q(u - u'), u - u' \rangle \tag{5.3}$$

$$\geq 0. \tag{5.4}$$

Thus T is monotone.

To prove maximality, suppose that $(\bar{u}, \bar{v})$ is monotonically related to $\text{gra } T$. This means that

$$\langle \bar{v} - T(u), \bar{u} - u \rangle \geq 0 \quad \text{for every } u \in \mathbb{R}^N.$$

Fix $h \in \mathbb{R}^N$ and $t > 0$, and choose

$$u = \bar{u} - th.$$

The preceding inequality becomes

$$\langle \bar{v} - Q(\bar{u} - th) - b, th \rangle \geq 0,$$

or equivalently,

$$\langle \bar{v} - Q\bar{u} - b + tQh, h \rangle \geq 0.$$

Letting $t \downarrow 0$, we obtain

$$\langle \bar{v} - Q\bar{u} - b, h \rangle \geq 0 \quad \text{for every } h \in \mathbb{R}^N.$$

Applying the same inequality to $-h$ shows that

$$\langle \bar{v} - Q\bar{u} - b, h \rangle = 0 \quad \text{for every } h \in \mathbb{R}^N.$$

Therefore,

$$\bar{v} = Q\bar{u} + b,$$

so $(\bar{u}, \bar{v}) \in \text{gra } T$. Hence no point can be added to the graph while preserving monotonicity, and T is maximal monotone. □

Proposition 5.7 (Rockafellar, [17]). *If $\phi : \mathbb{R}^N \rightarrow \mathbb{R} \cup \{+\infty\}$ is closed proper convex then $\partial\phi$ is maximal monotone.*

Proposition 5.8 (Sum of maximal monotone operators, [17]). *If T_1 and T_2 are maximal monotone and $\text{relint dom } T_1 \cap \text{relint dom } T_2 \neq \emptyset$, then $T_1 + T_2$ is maximal monotone. In particular this holds when one of them has domain $\mathbb{R}^N$.*

5.2 The resolvent

Definition 5.9 (Metric resolvent). Let $T : \mathbb{R}^N \rightrightarrows \mathbb{R}^N$ be set-valued and let $\mathcal{M} \succ 0$. The metric resolvent of T is

$$J_T^{\mathcal{M}} = (\mathcal{M} + T)^{-1}\mathcal{M}. \quad (5.5)$$

Equivalently,

$$p \in J_T^{\mathcal{M}}(v) \iff \mathcal{M}(v - p) \in T(p). \quad (5.6)$$

Proposition 5.10 (Uniqueness on the domain). *If T is monotone and $\mathcal{M} \succ 0$, then $J_T^{\mathcal{M}}$ is single-valued on its domain.*

Proof. Suppose that $p, p' \in J_T^{\mathcal{M}}(v)$. Then

$$\mathcal{M}(v - p) \in T(p), \quad \mathcal{M}(v - p') \in T(p').$$

The monotonicity of T gives

$$\langle \mathcal{M}(v - p) - \mathcal{M}(v - p'), p - p' \rangle \geq 0.$$

The left-hand side is

$$-\langle \mathcal{M}(p - p'), p - p' \rangle = -\|p - p'\|_{\mathcal{M}}^2.$$

Hence

$$\|p - p'\|_{\mathcal{M}}^2 \leq 0,$$

and therefore $p = p'$. □

Proposition 5.11 (Firm nonexpansiveness on the domain). *Let T be monotone and let $\mathcal{M} \succ 0$. For every $v, v' \in \text{dom } J_T^{\mathcal{M}}$,*

$$\|J_T^{\mathcal{M}}(v) - J_T^{\mathcal{M}}(v')\|_{\mathcal{M}}^2 \leq \langle J_T^{\mathcal{M}}(v) - J_T^{\mathcal{M}}(v'), v - v' \rangle_{\mathcal{M}}. \quad (5.7)$$

In particular,

$$\|J_T^{\mathcal{M}}(v) - J_T^{\mathcal{M}}(v')\|_{\mathcal{M}} \leq \|v - v'\|_{\mathcal{M}}. \quad (5.8)$$

Proof. Set

$$p = J_T^{\mathcal{M}}(v), \quad p' = J_T^{\mathcal{M}}(v').$$

By the definition of the resolvent,

$$\mathcal{M}(v - p) \in T(p), \quad \mathcal{M}(v' - p') \in T(p').$$

Monotonicity gives

$$\langle \mathcal{M}(v - p) - \mathcal{M}(v' - p'), p - p' \rangle \geq 0.$$

Rearranging, we obtain

$$\|p - p'\|_{\mathcal{M}}^2 \leq \langle p - p', v - v' \rangle_{\mathcal{M}},$$

which proves firm nonexpansiveness. The Cauchy-Schwarz inequality then gives

$$\|p - p'\|_{\mathcal{M}}^2 \leq \|p - p'\|_{\mathcal{M}} \|v - v'\|_{\mathcal{M}},$$

and hence

$$\|p - p'\|_{\mathcal{M}} \leq \|v - v'\|_{\mathcal{M}}.$$

□

Proposition 5.12 (Minty's theorem, [12]). *A monotone operator $T : \mathbb{R}^N \rightrightarrows \mathbb{R}^N$ is maximal monotone if and only if*

$$\text{range}(I + T) = \mathbb{R}^N.$$

Proposition 5.13 (Minty's theorem in a metric). *Let $T : \mathbb{R}^N \rightrightarrows \mathbb{R}^N$ be monotone. Then T is maximal monotone if and only if*

$$\text{range}(\mathcal{M} + T) = \mathbb{R}^N$$

for one, and consequently for every, $\mathcal{M} \succ 0$.

Proof. Let $R = \mathcal{M}^{1/2}$, and define

$$\tilde{T}(y) = R^{-1}T(R^{-1}y).$$

If

$$u \in T(x), \quad u' \in T(x'),$$

and

$$y = Rx, \quad y' = Rx', \quad w = R^{-1}u, \quad w' = R^{-1}u',$$

then

$$\langle w - w', y - y' \rangle = \langle u - u', x - x' \rangle.$$

Thus the change of variables preserves monotonicity and maximal monotonicity. Consequently, T is maximal monotone if and only if $\tilde{T}$ is maximal monotone.

Moreover,

$$u \in \mathcal{M}x + T(x)$$

is equivalent, after multiplication by R^{-1} , to

$$R^{-1}u \in y + \tilde{T}(y).$$

Since R^{-1} is bijective,

$$\text{range}(\mathcal{M} + T) = \mathbb{R}^N \iff \text{range}(I + \tilde{T}) = \mathbb{R}^N.$$

The result follows from Minty's theorem applied to $\tilde{T}$. □

Proposition 5.14 (Resolvent of a maximal monotone operator). *Let T be maximal monotone and let $\mathcal{M} \succ 0$. Then $J_T^{\mathcal{M}}$ is single-valued and defined on all of $\mathbb{R}^N$. Moreover,*

$$\text{Fix}(J_T^{\mathcal{M}}) = T^{-1}(0). \quad (5.9)$$

Proof. Single-valuedness follows from monotonicity. The metric form of Minty's theorem gives

$$\text{range}(\mathcal{M} + T) = \mathbb{R}^N.$$

Since $\mathcal{M}$ is bijective, $J_T^{\mathcal{M}}$ is defined on all of $\mathbb{R}^N$.

Finally,

$$p = J_T^{\mathcal{M}}(p) \iff \mathcal{M}(p - p) \in T(p) \quad (5.10)$$

$$\iff 0 \in T(p). \quad (5.11)$$

Thus

$$\text{Fix}(J_T^{\mathcal{M}}) = T^{-1}(0).$$

□

Proposition 5.15 (One-step estimate). *Let T be maximal monotone, let $\mathcal{M} \succ 0$, and let $v^* \in T^{-1}(0)$. If*

$$p = J_T^{\mathcal{M}}(v),$$

then

$$\|p - v^*\|_{\mathcal{M}}^2 \leq \|v - v^*\|_{\mathcal{M}}^2 - \|p - v\|_{\mathcal{M}}^2. \quad (5.12)$$

Proof. Since $v^* \in T^{-1}(0)$, it is a fixed point of the resolvent:

$$J_T^{\mathcal{M}}(v^*) = v^*.$$

Firm nonexpansiveness therefore gives

$$\|p - v^*\|_{\mathcal{M}}^2 \leq \langle p - v^*, v - v^* \rangle_{\mathcal{M}}.$$

Using

$$v - v^* = (v - p) + (p - v^*),$$

we obtain

$$\langle p - v^*, v - p \rangle_{\mathcal{M}} \geq 0.$$

Expanding the squared distance gives

$$\|v - v^*\|_{\mathcal{M}}^2 = \|v - p\|_{\mathcal{M}}^2 + \|p - v^*\|_{\mathcal{M}}^2 + 2 \langle v - p, p - v^* \rangle_{\mathcal{M}} \quad (5.13)$$

$$\geq \|v - p\|_{\mathcal{M}}^2 + \|p - v^*\|_{\mathcal{M}}^2. \quad (5.14)$$

Rearranging proves the result. □

5.3 Variable metric proximal point algorithm

Definition 5.16 (Variable-metric quasi-Fejér sequence). Let $S \subseteq \mathbb{R}^N$ be nonempty, and let $(\mathcal{M}_k)$ be a sequence of positive definite matrices. A sequence (v^k) is *variable-metric quasi-Fejér monotone* with respect to S if, for every $v^* \in S$, there exists a summable sequence (δ_k) of nonnegative numbers such that

$$\|v^{k+1} - v^*\|_{\mathcal{M}_{k+1}}^2 \leq (1 + \eta_k) \|v^k - v^*\|_{\mathcal{M}_k}^2 + \delta_k, \quad (5.15)$$

where $\eta_k \geq 0$ and

$$\sum_{k=0}^{\infty} \eta_k < \infty.$$

When $\mathcal{M}_k = I$, $\eta_k = 0$, and $\delta_k = 0$, this reduces to the usual notion of Fejér monotonicity.

Lemma 5.17 (Variable-metric quasi-Fejér convergence). *Let $S \subseteq \mathbb{R}^N$ be nonempty, and suppose that*

$$\mu I \preceq \mathcal{M}_k \preceq \nu I$$

for some $0 < \mu \leq \nu < \infty$. Suppose that, for every $v^ \in S$,*

$$d_{k+1}(v^*) \leq (1 + \eta_k)d_k(v^*) - s_k + \delta_k, \quad (5.16)$$

where

$$d_k(v^*) = \|v^k - v^*\|_{\mathcal{M}_k}^2,$$

the sequences (η_k) , (δ_k) , and (s_k) are nonnegative, and

$$\sum_{k=0}^{\infty} \eta_k < \infty, \quad \sum_{k=0}^{\infty} \delta_k < \infty.$$

Then:

1. (v^k) is bounded;
2. $d_k(v^*)$ converges for every $v^* \in S$;
3. $\sum_{k=0}^{\infty} s_k < \infty$;
4. if every cluster point of (v^k) belongs to S , then (v^k) converges to a point of S .

Proof. Define

$$\Lambda_0 = 1, \quad \Lambda_k = \prod_{j=0}^{k-1} (1 + \eta_j).$$

Since

$$\log \Lambda_k \leq \sum_{j=0}^{k-1} \eta_j,$$

the sequence (Λ_k) converges to a finite positive limit.

Fix $v^* \in S$, write $d_k = d_k(v^*)$, and divide (5.16) by $\Lambda_{k+1} = (1 + \eta_k)\Lambda_k$. Dropping the nonnegative term s_k gives

$$\frac{d_{k+1}}{\Lambda_{k+1}} \leq \frac{d_k}{\Lambda_k} + \frac{\delta_k}{\Lambda_{k+1}}.$$

Set

$$b_k = \frac{d_k}{\Lambda_k}, \quad r_k = \frac{\delta_k}{\Lambda_{k+1}}.$$

Then

$$b_{k+1} \leq b_k + r_k, \quad \sum_{k=0}^{\infty} r_k < \infty.$$

The sequence

$$\tilde{b}_k = b_k + \sum_{j=k}^{\infty} r_j$$

is nonnegative and nonincreasing. It therefore converges. Since the tail sum tends to zero, (b_k) converges. As (Λ_k) also converges, (d_k) converges.

In particular, (d_k) is bounded. Since

$$\mu \|v^k - v^*\|^2 \leq d_k,$$

the sequence (v^k) is bounded.

Rearranging (5.16) yields

$$s_k \leq d_k - d_{k+1} + \eta_k d_k + \delta_k.$$

Summing from $k = 0$ to n , we obtain

$$\sum_{k=0}^n s_k \leq d_0 - d_{n+1} + \sum_{k=0}^n \eta_k d_k + \sum_{k=0}^n \delta_k.$$

The right-hand side is bounded uniformly in n , so

$$\sum_{k=0}^{\infty} s_k < \infty.$$

Finally, suppose that every cluster point of (v^k) belongs to S . Since (v^k) is bounded, it has a cluster point $\bar{v} \in S$. Choose a subsequence such that

$$v^{k_i} \longrightarrow \bar{v}.$$

Applying the preceding argument with $v^* = \bar{v}$ shows that

$$d_k(\bar{v}) = \|v^k - \bar{v}\|_{\mathcal{M}_k}^2$$

has a limit. Along the subsequence,

$$d_{k_i}(\bar{v}) \leq \nu \|v^{k_i} - \bar{v}\|^2 \longrightarrow 0.$$

Hence $d_k(\bar{v}) \rightarrow 0$. Since

$$\mu \|v^k - \bar{v}\|^2 \leq d_k(\bar{v}),$$

we conclude that $v^k \rightarrow \bar{v} \in S$. □

Proposition 5.18 (Convergence in the exact case). *Let T be maximal monotone with $T^{-1}(0) \neq \emptyset$. Suppose that $(\mathcal{M}_k)$ satisfies*

$$\mu I \preceq \mathcal{M}_k \preceq \nu I, \quad \mathcal{M}_{k+1} \preceq (1 + \eta_k) \mathcal{M}_k, \quad (5.17)$$

where $0 < \mu \leq \nu < \infty$, $\eta_k \geq 0$, and

$$\sum_{k=0}^{\infty} \eta_k < \infty.$$

Then the sequence

$$v^{k+1} = J_T^{\mathcal{M}_k}(v^k) \quad (5.18)$$

converges, for every initial point v^0 , to an element of $T^{-1}(0)$.

Proof. Set

$$S = T^{-1}(0),$$

and fix $v^* \in S$. The one-step estimate gives

$$\|v^{k+1} - v^*\|_{\mathcal{M}_k}^2 \leq \|v^k - v^*\|_{\mathcal{M}_k}^2 - \|v^{k+1} - v^k\|_{\mathcal{M}_k}^2.$$

Using the metric variation condition, we obtain

$$\|v^{k+1} - v^*\|_{\mathcal{M}_{k+1}}^2 \leq (1 + \eta_k) \|v^k - v^*\|_{\mathcal{M}_k}^2 - \|v^{k+1} - v^k\|_{\mathcal{M}_k}^2.$$

Thus the quasi-Fejér lemma applies with

$$s_k = \|v^{k+1} - v^k\|_{\mathcal{M}_k}^2, \quad \delta_k = 0.$$

It follows that (v^k) is bounded and

$$\sum_{k=0}^{\infty} \|v^{k+1} - v^k\|_{\mathcal{M}_k}^2 < \infty.$$

Since $\mathcal{M}_k \succeq \mu I$,

$$v^{k+1} - v^k \longrightarrow 0.$$

Let $\bar{v}$ be a cluster point, and choose a subsequence

$$v^{k_i} \longrightarrow \bar{v}.$$

The resolvent relation gives

$$u^{k_i} := \mathcal{M}_{k_i-1}(v^{k_i-1} - v^{k_i}) \in T(v^{k_i}).$$

Moreover,

$$\|u^{k_i}\| \leq \nu \|v^{k_i-1} - v^{k_i}\| \longrightarrow 0.$$

Since the graph of T is closed,

$$0 \in T(\bar{v}).$$

Thus every cluster point belongs to S . The quasi-Fejér lemma now gives

$$v^k \longrightarrow \bar{v} \in T^{-1}(0).$$

□

Proposition 5.19 (Convergence in the inexact case, [18]). *Under the assumptions of the preceding proposition, suppose instead that v^{k+1} satisfies*

$$\|v^{k+1} - J_T^{\mathcal{M}_k}(v^k)\|_{\mathcal{M}_k} \leq \varepsilon_k, \quad \sum_{k=0}^{\infty} \varepsilon_k < \infty. \quad (5.19)$$

Then (v^k) converges to a point of $T^{-1}(0)$.

Proof. Set

$$\widehat{v}^{k+1} = J_T^{\mathcal{M}_k}(v^k), \quad e_k = v^{k+1} - \widehat{v}^{k+1}.$$

Then

$$\|e_k\|_{\mathcal{M}_k} \leq \varepsilon_k.$$

Fix $v^* \in T^{-1}(0)$, and write

$$a_k = \|v^k - v^*\|_{\mathcal{M}_k}, \quad t_k = \|\widehat{v}^{k+1} - v^k\|_{\mathcal{M}_k}.$$

The one-step estimate gives

$$\|\widehat{v}^{k+1} - v^*\|_{\mathcal{M}_k}^2 \leq a_k^2 - t_k^2.$$

In particular,

$$\|\widehat{v}^{k+1} - v^*\|_{\mathcal{M}_k} \leq a_k.$$

The triangle inequality and the metric variation condition imply

$$a_{k+1} \leq (1 + \eta_k)^{1/2}(a_k + \varepsilon_k).$$

Iterating this inequality and using

$$\sum_k \eta_k < \infty, \quad \sum_k \varepsilon_k < \infty,$$

shows that (a_k) is bounded. Choose $A > 0$ such that

$$a_k \leq A \quad \text{for every } k.$$

We also have

$$\|v^{k+1} - v^*\|_{\mathcal{M}_k}^2 = \|\widehat{v}^{k+1} - v^* + e_k\|_{\mathcal{M}_k}^2 \tag{5.20}$$

$$\leq \|\widehat{v}^{k+1} - v^*\|_{\mathcal{M}_k}^2 + 2\|\widehat{v}^{k+1} - v^*\|_{\mathcal{M}_k} \|e_k\|_{\mathcal{M}_k} + \|e_k\|_{\mathcal{M}_k}^2 \tag{5.21}$$

$$\leq a_k^2 - t_k^2 + 2A\varepsilon_k + \varepsilon_k^2. \tag{5.22}$$

Using the metric variation condition gives

$$a_{k+1}^2 \leq (1 + \eta_k)a_k^2 - (1 + \eta_k)t_k^2 + (1 + \eta_k)(2A\varepsilon_k + \varepsilon_k^2).$$

The quasi-Fejér lemma applies with

$$s_k = (1 + \eta_k)t_k^2$$

and

$$\delta_k = (1 + \eta_k)(2A\varepsilon_k + \varepsilon_k^2).$$

The sequence (δ_k) is summable. Consequently,

$$\sum_{k=0}^{\infty} t_k^2 < \infty,$$

and hence

$$\widehat{v}^{k+1} - v^k \longrightarrow 0.$$

Moreover, $\varepsilon_k \rightarrow 0$ and $\mathcal{M}_k \succeq \mu I$ imply

$$\|e_k\| \leq \frac{\varepsilon_k}{\sqrt{\mu}} \rightarrow 0.$$

Thus

$$v^{k+1} - v^k = \hat{v}^{k+1} - v^k + e_k \rightarrow 0.$$

The resolvent relation gives

$$u^{k+1} = \mathcal{M}_k(v^k - \hat{v}^{k+1}) \in T(\hat{v}^{k+1}).$$

Since $\mathcal{M}_k \preceq \nu I$,

$$\|u^{k+1}\| \leq \nu \|v^k - \hat{v}^{k+1}\| \rightarrow 0.$$

Let $\bar{v}$ be a cluster point of (v^k) , and choose a subsequence $v^{k_i} \rightarrow \bar{v}$. Since

$$\hat{v}^{k_i+1} - v^{k_i} \rightarrow 0,$$

we also have

$$\hat{v}^{k_i+1} \rightarrow \bar{v}.$$

The graph of T is closed, so

$$0 \in T(\bar{v}).$$

Thus every cluster point of (v^k) belongs to $T^{-1}(0)$. The quasi-Fejér lemma now implies that the entire sequence converges to a point of $T^{-1}(0)$. $\square$

Algorithm 5.1 Variable metric proximal point algorithm

Input: T maximal monotone, $v^0 \in \mathbb{R}^N$, metrics $(\mathcal{M}_k)$, tolerances (ε_k) summable

Output: a point of $T^{-1}(0)$

1: **for** $k = 0, 1, 2, \dots$ **do**

2: compute v^{k+1} with $\|v^{k+1} - J_T^{\mathcal{M}_k}(v^k)\|_{\mathcal{M}_k} \leq \varepsilon_k$, that is solve $\mathcal{M}_k(v^k - v^{k+1}) \in T(v^{k+1})$ approximately

3: choose $\mathcal{M}_{k+1}$ satisfying the bounds of the convergence hypotheses

4: **end for**

5.4 Proximal operator and Moreau envelope

Definition 5.20 (Proximal operator and Moreau envelope). For ϕ closed proper convex and $\mathcal{M} \succ 0$,

$$\text{prox}_{\phi}^{\mathcal{M}}(v) = \underset{w}{\operatorname{argmin}} \left\{ \phi(w) + \frac{1}{2} \|w - v\|_{\mathcal{M}}^2 \right\}, \quad e_{\phi}^{\mathcal{M}}(v) = \min_w \left\{ \phi(w) + \frac{1}{2} \|w - v\|_{\mathcal{M}}^2 \right\}. \quad (5.23)$$

The Moreau envelope selects, for each v , the point striking the best trade off between minimising ϕ and staying close to v . One advantage is the regularity it gains, as the properties below show; and as $\mathcal{M}$ grows the envelope increases to ϕ itself.

Proposition 5.21 (Well-posedness). *Let $\phi : \mathbb{R}^N \rightarrow \mathbb{R} \cup \{+\infty\}$ be closed, proper, and convex, and let $\mathcal{M} \succ 0$. Then, for every $v \in \mathbb{R}^N$, the problem*

$$\min_{w \in \mathbb{R}^N} \left\{ \phi(w) + \frac{1}{2} \|w - v\|_{\mathcal{M}}^2 \right\}$$

has a unique solution. Consequently, $\text{prox}_{\phi}^{\mathcal{M}}$ is single-valued and defined on all of $\mathbb{R}^N$, and $e_{\phi}^{\mathcal{M}}$ is finite everywhere.

Proof. For fixed v , the function

$$w \mapsto \phi(w) + \frac{1}{2} \|w - v\|_{\mathcal{M}}^2$$

is closed and proper. Since $\mathcal{M} \succ 0$, its quadratic term is strongly convex and coercive. The entire function is therefore coercive and strongly convex. Coercivity and closedness give existence of a minimizer, while strong convexity gives uniqueness. $\square$

Proposition 5.22 (Resolvent form). *For $v, p \in \mathbb{R}^N$,*

$$p = \text{prox}_{\phi}^{\mathcal{M}}(v)$$

if and only if

$$\mathcal{M}(v - p) \in \partial\phi(p).$$

Equivalently,

$$\text{prox}_{\phi}^{\mathcal{M}} = (I + \mathcal{M}^{-1}\partial\phi)^{-1}.$$

Proof. By Fermat's rule,

$$p = \text{prox}_{\phi}^{\mathcal{M}}(v) \iff 0 \in \partial\phi(p) + \mathcal{M}(p - v) \quad (5.24)$$

$$\iff \mathcal{M}(v - p) \in \partial\phi(p). \quad (5.25)$$

Rearranging the last inclusion gives

$$v \in p + \mathcal{M}^{-1}\partial\phi(p),$$

which is precisely the resolvent formulation. $\square$

As a consequence of the general monotone operator case:

Proposition 5.23 (Firm nonexpansiveness). *The mapping $\text{prox}_{\phi}^{\mathcal{M}}$ is firmly nonexpansive in the metric induced by $\mathcal{M}$. More precisely, for every $v, v' \in \mathbb{R}^N$,*

$$\left\| \text{prox}_{\phi}^{\mathcal{M}}(v) - \text{prox}_{\phi}^{\mathcal{M}}(v') \right\|_{\mathcal{M}}^2 \leq \left\langle \text{prox}_{\phi}^{\mathcal{M}}(v) - \text{prox}_{\phi}^{\mathcal{M}}(v'), v - v' \right\rangle_{\mathcal{M}}. \quad (5.26)$$

In particular,

$$\left\| \text{prox}_{\phi}^{\mathcal{M}}(v) - \text{prox}_{\phi}^{\mathcal{M}}(v') \right\|_{\mathcal{M}} \leq \|v - v'\|_{\mathcal{M}}.$$

Proposition 5.24 (Regularity of the envelope). *The Moreau envelope $e_{\phi}^{\mathcal{M}}$ is convex, finite everywhere, and continuously differentiable. Its gradient is*

$$\nabla e_{\phi}^{\mathcal{M}}(v) = \mathcal{M}(v - \text{prox}_{\phi}^{\mathcal{M}}(v)). \quad (5.27)$$

Proof. Finiteness follows from the well-posedness proposition. The function

$$(v, w) \mapsto \phi(w) + \frac{1}{2} \|w - v\|_{\mathcal{M}}^2$$

is jointly convex in (v, w) . Its partial minimization over w is therefore convex, so $e_{\phi}^{\mathcal{M}}$ is convex.

Fix v , and write

$$p = \text{prox}_{\phi}^{\mathcal{M}}(v), \quad r = \mathcal{M}(v - p).$$

For $h \in \mathbb{R}^N$, evaluating the minimization problem defining $e_{\phi}^{\mathcal{M}}(v + h)$ at p gives

$$e_{\phi}^{\mathcal{M}}(v + h) \leq \phi(p) + \frac{1}{2} \|p - v - h\|_{\mathcal{M}}^2 \quad (5.28)$$

$$= e_{\phi}^{\mathcal{M}}(v) + \langle r, h \rangle + \frac{1}{2} \|h\|_{\mathcal{M}}^2. \quad (5.29)$$

Thus

$$e_{\phi}^{\mathcal{M}}(v + h) - e_{\phi}^{\mathcal{M}}(v) \leq \langle r, h \rangle + \frac{1}{2} \|h\|_{\mathcal{M}}^2. \quad (5.30)$$

Now set

$$p_h = \text{prox}_{\phi}^{\mathcal{M}}(v + h), \quad r_h = \mathcal{M}(v + h - p_h).$$

Applying (5.30) with $v + h$ in place of v and with increment $-h$ gives

$$e_{\phi}^{\mathcal{M}}(v) - e_{\phi}^{\mathcal{M}}(v + h) \leq -\langle r_h, h \rangle + \frac{1}{2} \|h\|_{\mathcal{M}}^2.$$

Equivalently,

$$e_{\phi}^{\mathcal{M}}(v + h) - e_{\phi}^{\mathcal{M}}(v) \geq \langle r_h, h \rangle - \frac{1}{2} \|h\|_{\mathcal{M}}^2.$$

The proximal mapping is Lipschitz, so $p_h \rightarrow p$ as $h \rightarrow 0$. Hence

$$r_h = \mathcal{M}(v + h - p_h) \longrightarrow \mathcal{M}(v - p) = r.$$

Combining the upper and lower estimates yields

$$e_{\phi}^{\mathcal{M}}(v + h) = e_{\phi}^{\mathcal{M}}(v) + \langle r, h \rangle + o(\|h\|).$$

Therefore,

$$\nabla e_{\phi}^{\mathcal{M}}(v) = r = \mathcal{M}(v - \text{prox}_{\phi}^{\mathcal{M}}(v)).$$

Finally, the proximal mapping is continuous, so the displayed gradient is continuous. Thus $e_{\phi}^{\mathcal{M}}$ is continuously differentiable on $\mathbb{R}^N$. $\square$

Proposition 5.25 (Envelope of an indicator). *Let $C \subseteq \mathbb{R}^N$ be nonempty, closed, and convex, and let $\mathcal{M} \succ 0$. Then*

$$\text{prox}_{\delta_C}^{\mathcal{M}}(v) = \Pi_C(v)^{\mathcal{M}}, \quad e_{\delta_C}^{\mathcal{M}}(v) = \frac{1}{2} \text{dist}_{\mathcal{M}}(v, C)^2, \quad (5.31)$$

where $\Pi_C(\cdot)^{\mathcal{M}}$ denotes the projection onto C in the metric induced by $\mathcal{M}$. In particular, if $C = K^*$ and $\mathcal{M} \in \mathcal{A}(K)$, then

$$\Pi_{K^*}(v)^{\mathcal{M}} = \Pi_{K^*}(v),$$

so

$$\text{prox}_{\delta_{K^*}}^{\mathcal{M}}(v) = \Pi_{K^*}(v).$$

Proof. By the definition of the proximal mapping,

$$\text{prox}_{\delta_C}^{\mathcal{M}}(v) = \underset{w \in \mathbb{R}^N}{\operatorname{argmin}} \left\{ \delta_C(w) + \frac{1}{2} \|w - v\|_{\mathcal{M}}^2 \right\}.$$

Since $\delta_C(w) = 0$ for $w \in C$ and $+\infty$ otherwise, this reduces to

$$\text{prox}_{\delta_C}^{\mathcal{M}}(v) = \underset{w \in C}{\operatorname{argmin}} \frac{1}{2} \|w - v\|_{\mathcal{M}}^2 = \Pi_C(v)^{\mathcal{M}}.$$

Similarly, the definition of the Moreau envelope gives

$$e_{\delta_C}^{\mathcal{M}}(v) = \inf_{w \in \mathbb{R}^N} \left\{ \delta_C(w) + \frac{1}{2} \|w - v\|_{\mathcal{M}}^2 \right\} \quad (5.32)$$

$$= \inf_{w \in C} \frac{1}{2} \|w - v\|_{\mathcal{M}}^2 \quad (5.33)$$

$$= \frac{1}{2} \operatorname{dist}_{\mathcal{M}}(v, C)^2. \quad (5.34)$$

Finally, if $C = K^*$ and $\mathcal{M} \in \mathcal{A}(K)$, admissibility ensures that the metric projection onto K^* agrees with the Euclidean projection. Therefore,

$$\Pi_{K^*}(v)^{\mathcal{M}} = \Pi_{K^*}(v).$$

□

Chapter 6

Newton method

6.1 Classic Newton method

Definition 6.1 (Equality constrained minimisation). Given $f : \mathbb{R}^n \rightarrow \mathbb{R}$ twice continuously differentiable and convex, $A \in \mathbb{R}^{q \times n}$ and $b \in \mathbb{R}^q$, the problem is

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to} \quad Ax = b. \quad (6.1)$$

Definition 6.2 (Zero finding). Given $\mathcal{R} : \mathbb{R}^N \rightarrow \mathbb{R}^N$, the problem is to find v with $\mathcal{R}(v) = 0$.

Algorithm 6.1 Newton's method for $\mathcal{R}(v) = 0$

Input: $\mathcal{R}$ differentiable, $v^0 \in \mathbb{R}^N$, tolerance $\varepsilon > 0$

Output: v with $\|\mathcal{R}(v)\| \leq \varepsilon$

```
1: for  $k = 0, 1, 2, \dots$  do
2:   if  $\|\mathcal{R}(v^k)\| \leq \varepsilon$  then
3:     return  $v^k$ 
4:   end if
5:   solve  $\mathcal{R}'(v^k) \Delta v^k = -\mathcal{R}(v^k)$ 
6:    $v^{k+1} \leftarrow v^k + \Delta v^k$ 
7: end for
```

Algorithm 6.2 Newton's method on the optimality conditions of the equality constrained problem

Input: f, A, b , a starting pair (x^0, y^0) , tolerance $\varepsilon > 0$

Output: (x, y) with $\|\mathcal{R}(x, y)\| \leq \varepsilon$, where $\mathcal{R}(x, y) = (\nabla f(x) + A^\top y, Ax - b)$

```
1: for  $k = 0, 1, 2, \dots$  do
2:   if  $\|\mathcal{R}(x^k, y^k)\| \leq \varepsilon$  then
3:     return  $(x^k, y^k)$ 
4:   end if
5:   solve  $\begin{pmatrix} \nabla^2 f(x^k) & A^\top \\ A & 0 \end{pmatrix} \begin{pmatrix} \Delta x^k \\ \Delta y^k \end{pmatrix} = - \begin{pmatrix} \nabla f(x^k) + A^\top y^k \\ Ax^k - b \end{pmatrix}$ 
6:    $(x^{k+1}, y^{k+1}) \leftarrow (x^k + \Delta x^k, y^k + \Delta y^k)$ 
7: end for
```

Proposition 6.3 (Local quadratic convergence). *Let $\mathcal{R}(v^*) = 0$. Suppose that $\mathcal{R}$ is continuously differentiable in a neighborhood of v^* , that $\mathcal{R}'$ is Lipschitz continuous there with constant L , and that $\mathcal{R}'(v^*)$ is invertible with*

$$\|\mathcal{R}'(v^*)^{-1}\| \leq \beta.$$

Then there exists $\varepsilon > 0$ such that, for every v^0 satisfying

$$\|v^0 - v^*\| \leq \varepsilon,$$

Newton's iteration

$$v^{k+1} = v^k - \mathcal{R}'(v^k)^{-1} \mathcal{R}(v^k)$$

is well defined, remains in the ball $\overline{B}(v^, \varepsilon)$, and satisfies*

$$\|v^{k+1} - v^*\| \leq \beta L \|v^k - v^*\|^2. \quad (6.2)$$

In particular, $v^k \rightarrow v^$ quadratically.*

Proof. Since $\mathcal{R}'(v^*)$ is invertible and $\mathcal{R}'$ is continuous, there exists $\varepsilon > 0$ such that $\mathcal{R}'(v)$ is invertible and

$$\|\mathcal{R}'(v)^{-1}\| \leq 2\beta \quad (6.3)$$

whenever

$$\|v - v^*\| \leq \varepsilon.$$

Shrinking ε , if necessary, we may also assume that $\overline{B}(v^*, \varepsilon)$ lies in the neighborhood where $\mathcal{R}'$ is L -Lipschitz and that

$$\beta L \varepsilon < 1.$$

Let $v \in \overline{B}(v^*, \varepsilon)$, and set

$$e = v - v^*.$$

Since $\mathcal{R}(v^*) = 0$:

$$\mathcal{R}(v) = \int_0^1 \mathcal{R}'(v^* + te) e \, dt.$$

Consequently,

$$\mathcal{R}'(v)e - \mathcal{R}(v) = \int_0^1 (\mathcal{R}'(v) - \mathcal{R}'(v^* + te)) e \, dt.$$

The Lipschitz continuity of $\mathcal{R}'$ yields

$$\|\mathcal{R}'(v)e - \mathcal{R}(v)\| \leq \int_0^1 L \|v - (v^* + te)\| \|e\| \, dt \quad (6.4)$$

$$= L \|e\|^2 \int_0^1 (1-t) \, dt \quad (6.5)$$

$$= \frac{L}{2} \|e\|^2. \quad (6.6)$$

Let

$$v^+ = v - \mathcal{R}'(v)^{-1} \mathcal{R}(v)$$

be the Newton iterate. Then

$$v^+ - v^* = \mathcal{R}'(v)^{-1} (\mathcal{R}'(v)e - \mathcal{R}(v)).$$

Using (6.3), we obtain

$$\|v^+ - v^*\| \leq 2\beta \frac{L}{2} \|e\|^2 = \beta L \|v - v^*\|^2.$$

Since $\|v - v^*\| \leq \varepsilon$,

$$\|v^+ - v^*\| \leq \beta L \varepsilon \|v - v^*\| < \|v - v^*\| \leq \varepsilon.$$

Thus the Newton iterate remains in $\overline{B}(v^*, \varepsilon)$. The argument can therefore be applied inductively to every iterate, giving

$$\|v^{k+1} - v^*\| \leq \beta L \|v^k - v^*\|^2$$

for every k . Hence $v^k \rightarrow v^*$ quadratically. $\square$

6.2 Semismooth Newton method

Definition 6.4 (Semismoothness). Let $\mathcal{R} : \mathbb{R}^N \rightarrow \mathbb{R}^N$ be locally Lipschitz. The mapping $\mathcal{R}$ is *semismooth* at v if it is directionally differentiable at v and

$$\sup_{V \in \partial_C \mathcal{R}(v+h)} \|\mathcal{R}(v+h) - \mathcal{R}(v) - Vh\| = o(\|h\|) \quad \text{as } h \rightarrow 0. \quad (6.7)$$

It is *semismooth of order $\gamma > 0$* at v if

$$\sup_{V \in \partial_C \mathcal{R}(v+h)} \|\mathcal{R}(v+h) - \mathcal{R}(v) - Vh\| = O(\|h\|^{1+\gamma}). \quad (6.8)$$

The case $\gamma = 1$ is called *strong semismoothness*.

Definition 6.5 (CD-regularity). A point v^* is *CD-regular* for $\mathcal{R}$ if every matrix in the Clarke Jacobian

$$\partial \mathcal{R}(v^*)$$

is nonsingular.

Algorithm 6.3 Semismooth Newton method

Input: A locally Lipschitz and semismooth mapping $\mathcal{R} : \mathbb{R}^N \rightarrow \mathbb{R}^N$, an initial point v^0 , and a tolerance $\varepsilon > 0$

Output: A point v satisfying $\|\mathcal{R}(v)\| \leq \varepsilon$

1: **for** $k = 0, 1, 2, \dots$ **do**

2: **if** $\|\mathcal{R}(v^k)\| \leq \varepsilon$ **then**

3: **return** v^k

4: **end if**

5: Choose $V_k \in \partial_C \mathcal{R}(v^k)$

6: Solve

$$V_k \Delta v^k = -\mathcal{R}(v^k)$$

7: Set $v^{k+1} \leftarrow v^k + \Delta v^k$

8: **end for**

Definition 6.6 (Q-convergence). Let $v^k \rightarrow v^*$.

1. The convergence is *Q-linear* with rate $\rho \in (0, 1)$ if

$$\|v^{k+1} - v^*\| \leq \rho \|v^k - v^*\|$$

for all sufficiently large k .

2. The convergence is *Q-superlinear* if

$$\|v^{k+1} - v^*\| = o(\|v^k - v^*\|).$$

3. The convergence has *Q-order at least* $1 + \gamma$ if

$$\|v^{k+1} - v^*\| = O(\|v^k - v^*\|^{1+\gamma}).$$

Proposition 6.7 (Closed graph of the Clarke Jacobian). *Let $\mathcal{R} : \mathbb{R}^N \rightarrow \mathbb{R}^N$ be locally Lipschitz. Then the set-valued mapping*

$$v \mapsto \partial\mathcal{R}(v)$$

has a closed graph. More precisely, if

$$v_k \rightarrow v, \quad V_k \rightarrow V, \quad V_k \in \partial\mathcal{R}(v_k),$$

then

$$V \in \partial\mathcal{R}(v).$$

Lemma 6.8 (Uniform invertibility near a CD-regular point). *Suppose that $\mathcal{R}$ is locally Lipschitz near v^* and that every matrix in $\partial\mathcal{R}(v^*)$ is nonsingular. Then there exist a neighborhood U of v^* and a constant $\beta > 0$ such that every*

$$V \in \partial\mathcal{R}(v), \quad v \in U,$$

is nonsingular and satisfies

$$\|V^{-1}\| \leq \beta.$$

Proof. Suppose that the conclusion were false. Then there would exist sequences

$$v_j \rightarrow v^*, \quad V_j \in \partial\mathcal{R}(v_j),$$

such that V_j is singular or

$$\|V_j^{-1}\| \rightarrow +\infty.$$

Since $\mathcal{R}$ is locally Lipschitz, its Clarke Jacobians are locally bounded. After passing to a subsequence, we may assume that

$$V_j \rightarrow V.$$

In either case, we can choose $x_j \in \mathbb{R}^N$ such that

$$\|x_j\| = 1 \quad \text{and} \quad \|V_j x_j\| \rightarrow 0.$$

The unit sphere is compact, so, after passing to another subsequence,

$$x_j \rightarrow x \quad \text{with} \quad \|x\| = 1.$$

Since the Clarke Jacobian has a closed graph,

$$V \in \partial\mathcal{R}(v^*).$$

Moreover,

$$Vx = \lim_{j \rightarrow \infty} V_j x_j = 0.$$

Because $x \neq 0$, the matrix V is singular. This contradicts the CD-regularity of v^* . Therefore, the asserted neighborhood and uniform inverse bound exist. $\square$

Proposition 6.9 (Local convergence of the semismooth Newton method, [16]). *Suppose that*

$$\mathcal{R}(v^*) = 0,$$

that $\mathcal{R}$ is semismooth at v^ , and that v^* is CD-regular. Then there exists $\varepsilon > 0$ such that the semismooth Newton method is well defined for every initial point satisfying*

$$\|v^0 - v^*\| \leq \varepsilon.$$

The iterates remain in $\overline{B}(v^, \varepsilon)$ and converge to v^* Q -superlinearly.*

If $\mathcal{R}$ is semismooth of order $\gamma > 0$ at v^ , then the convergence has Q -order at least $1 + \gamma$:*

$$\|v^{k+1} - v^*\| = O(\|v^k - v^*\|^{1+\gamma}).$$

In particular, strong semismoothness gives local Q -quadratic convergence.

Proof. By the uniform invertibility lemma, there exist a neighborhood U of v^* and a constant $\beta > 0$ such that

$$\|V^{-1}\| \leq \beta$$

for every $v \in U$ and every $V \in \partial_C \mathcal{R}(v)$.

For v near v^* , set

$$e = v - v^*$$

and choose any $V \in \partial\mathcal{R}(v)$. Since $\mathcal{R}(v^*) = 0$, semismoothness gives

$$\mathcal{R}(v) - Ve = o(\|e\|),$$

uniformly over all $V \in \partial\mathcal{R}(v)$.

The corresponding semismooth Newton iterate is

$$v^+ = v - V^{-1}\mathcal{R}(v).$$

Its error satisfies

$$v^+ - v^* = e - V^{-1}\mathcal{R}(v) \tag{6.9}$$

$$= -V^{-1}(\mathcal{R}(v) - Ve). \tag{6.10}$$

Consequently,

$$\|v^+ - v^*\| \leq \|V^{-1}\| \|\mathcal{R}(v) - Ve\| \quad (6.11)$$

$$\leq \beta \|\mathcal{R}(v) - Ve\| \quad (6.12)$$

$$= o(\|v - v^*\|). \quad (6.13)$$

Therefore, there exists $\varepsilon > 0$ such that $\overline{B}(v^*, \varepsilon) \subseteq U$ and

$$\|v^+ - v^*\| \leq \frac{1}{2} \|v - v^*\|$$

whenever

$$0 < \|v - v^*\| \leq \varepsilon.$$

It follows inductively that every matrix V_k is nonsingular, every Newton step is well defined, and all iterates remain in $\overline{B}(v^*, \varepsilon)$. Moreover,

$$\|v^{k+1} - v^*\| \leq \frac{1}{2} \|v^k - v^*\|,$$

so $v^k \rightarrow v^*$. The sharper estimate

$$\|v^{k+1} - v^*\| = o(\|v^k - v^*\|)$$

shows that the convergence is Q-superlinear.

If $\mathcal{R}$ is semismooth of order γ , then there exists $C > 0$ such that

$$\|\mathcal{R}(v) - Ve\| \leq C \|e\|^{1+\gamma}$$

for all v sufficiently close to v^* and every $V \in \partial_C \mathcal{R}(v)$. Hence

$$\|v^+ - v^*\| \leq \beta C \|v - v^*\|^{1+\gamma}.$$

Applying this estimate to the iterates gives

$$\|v^{k+1} - v^*\| = O(\|v^k - v^*\|^{1+\gamma}).$$

For $\gamma = 1$, this is Q-quadratic convergence. $\square$

6.3 Merit functions and line searches

Definition 6.10 (Merit function). Let $S \subseteq \mathbb{R}^N$ be the solution set of a problem. A function $\psi : \mathbb{R}^N \rightarrow \mathbb{R} \cup \{+\infty\}$ is a *merit function* for it if $\operatorname{argmin} \psi = S$.

Definition 6.11 (Descent direction). d is a *descent direction* for ψ at w if ψ is differentiable at w and $\langle \nabla \psi(w), d \rangle < 0$.

Definition 6.12 (Line search). Given w , a direction d and a merit ψ , a *line search* is a rule returning $\alpha > 0$ from the restriction $\alpha \mapsto \psi(w + \alpha d)$ and its slope $\varsigma(\alpha) = \langle \nabla \psi(w + \alpha d), d \rangle$.

Definition 6.13 (Exact line search). $\alpha \in \operatorname{argmin}_{\alpha > 0} \psi(w + \alpha d)$.

Definition 6.14 (Secant search). Given $0 \leq \alpha_{\text{lo}} < \alpha_{\text{hi}}$ with $\varsigma(\alpha_{\text{lo}}) \leq 0 \leq \varsigma(\alpha_{\text{hi}})$, the *secant search* performs at most N steps of

$$\alpha = \alpha_{\text{lo}} - \varsigma(\alpha_{\text{lo}}) \frac{\alpha_{\text{hi}} - \alpha_{\text{lo}}}{\varsigma(\alpha_{\text{hi}}) - \varsigma(\alpha_{\text{lo}})}, \quad (6.14)$$

replacing α_{lo} or α_{hi} by α according to the sign of $\varsigma(\alpha)$, falling back to the midpoint when α is not interior to the bracket, and stopping when the bracket is narrower than a tolerance. It returns α_{hi} .

Definition 6.15 (Armijo line search). Given $\gamma \in (0, 1)$, $\beta \in (0, 1)$ and a descent direction d , the *backtracking Armijo* rule accepts the first $\alpha \in \{\beta^j\}_{j \geq 0}$ with

$$\psi(w + \alpha d) \leq \psi(w) + \gamma \alpha \varsigma(0). \quad (6.15)$$

Definition 6.16 (Backtracking decrease line search). Given $\beta \in (0, 1)$, the rule accepts the first $\alpha \in \{\beta^j\}_{j \geq 0}$ with $\psi(w + \alpha d) < \psi(w)$.

Algorithm 6.4 Semismooth Newton method with a line search on a merit function

Input: $\mathcal{R}$, a merit ψ for $\{\mathcal{R} = 0\}$, a line search rule, v^0 , tolerance $\varepsilon > 0$

Output: v with $\|\mathcal{R}(v)\| \leq \varepsilon$

```

1: for  $k = 0, 1, 2, \dots$  do
2:   if  $\|\mathcal{R}(v^k)\| \leq \varepsilon$  then
3:     return  $v^k$ 
4:   end if
5:   choose  $V_k \in \partial \mathcal{R}(v^k)$ 
6:   solve  $V_k \Delta v^k = -\mathcal{R}(v^k)$ 
7:   obtain  $\alpha_k$  from the line search rule applied to  $\psi$  at  $v^k$  along  $\Delta v^k$ 
8:    $v^{k+1} \leftarrow v^k + \alpha_k \Delta v^k$ 
9: end for
```

Proposition 6.17 (Semismoothness of the catalogue projections). *The projections onto the following cones, and onto their duals, are semismooth at every point:*

1. the zero cone;
2. the nonnegative orthant;
3. the Lorentz cone;
4. the semidefinite cone;
5. the exponential cone;
6. the power cone.

6.4 Self-concordant functions and damped Newton methods

Definition 6.18 (Self-concordant function). Let $D \subseteq \mathbb{R}^N$ be open and convex, and let $f : D \rightarrow \mathbb{R}$ be convex and three times differentiable. The *local seminorm* induced by f at $x \in D$ is

$$\|h\|_x = (D^2 f(x)[h, h])^{1/2}. \quad (6.16)$$

The function f is *self-concordant* if

$$\left| D^3 f(x)[h, h, h] \right| \leq 2 \|h\|_x^3 \quad \text{for every } x \in D \text{ and } h \in \mathbb{R}^N. \quad (6.17)$$

Proposition 6.19 (Stability under linear maps). *Let f be self-concordant on D , and let $A \in \mathbb{R}^{N \times M}$. Then*

$$g = f \circ A$$

is self-concordant on

$$A^{-1}(D) = \{y \in \mathbb{R}^M : Ay \in D\}.$$

Proof. The chain rule gives

$$D^2 g(y)[h, h] = D^2 f(Ay)[Ah, Ah]$$

and

$$D^3 g(y)[h, h, h] = D^3 f(Ay)[Ah, Ah, Ah].$$

Applying the self-concordance inequality for f at Ay in the direction Ah , we obtain

$$\left| D^3 g(y)[h, h, h] \right| = \left| D^3 f(Ay)[Ah, Ah, Ah] \right| \quad (6.18)$$

$$\leq 2 (D^2 f(Ay)[Ah, Ah])^{3/2} \quad (6.19)$$

$$= 2 (D^2 g(y)[h, h])^{3/2}. \quad (6.20)$$

Thus g is self-concordant. No injectivity or surjectivity assumption on A is required. $\square$

Proposition 6.20 (Stability under sums and positive scaling). *Let $f_1, \dots, f_r$ be self-concordant on the open convex sets $D_1, \dots, D_r$, and suppose that*

$$D = \bigcap_{i=1}^r D_i \neq \emptyset.$$

Then

$$f = \sum_{i=1}^r f_i$$

is self-concordant on D .

Moreover, if f is self-concordant and $t \geq 1$, then tf is self-concordant.

Definition 6.21 (Feasible points and directions). Consider the equality-constrained problem

$$\min_{x \in D} f(x) \quad \text{subject to} \quad Ax = b.$$

A point x is *feasible* if

$$x \in D \quad \text{and} \quad Ax = b.$$

A direction h is a *feasible direction* at x if

$$Ah = 0$$

and $x + th \in D$ for all sufficiently small $t > 0$.

Definition 6.22 (Dikin ellipsoid). Let f be twice differentiable on D , with $\nabla^2 f(x) \succ 0$. For $x \in D$ and $r > 0$, the *Dikin ellipsoid of radius r* centered at x is

$$\mathcal{E}_x(r) = \{x + h : \|h\|_x < r\}. \quad (6.21)$$

Equivalently,

$$\mathcal{E}_x(r) = \left\{x + h : \left\langle h, \nabla^2 f(x) h \right\rangle < r^2\right\}.$$

The set $\mathcal{E}_x(1)$ is called the *unit Dikin ellipsoid*.

Proposition 6.23 (Dikin ellipsoid property). *Let*

$$f : \mathbb{R}^N \rightarrow \mathbb{R} \cup \{+\infty\}$$

be closed, proper, and convex, with open domain

$$D = \text{dom } f.$$

Suppose that f is self-concordant on D and that

$$\nabla^2 f(x) \succ 0 \quad \text{for every } x \in D.$$

Then

$$\mathcal{E}_x(1) = \{x + h : \|h\|_x < 1\} \subseteq D \quad \text{for every } x \in D. \quad (6.22)$$

Definition 6.24 (Newton step and Newton decrement). Let f be a closed self-concordant function with open domain $D = \text{dom } f$, and suppose that

$$\nabla^2 f(x) \succ 0 \quad \text{for every } x \in D.$$

Consider the equality-constrained problem

$$\min_{x \in D} f(x) \quad \text{subject to} \quad Ax = b,$$

where A has full row rank.

At a feasible point x , the *Newton step* Δx and the associated multiplier w are defined by

$$\begin{pmatrix} \nabla^2 f(x) & A^\top \\ A & 0 \end{pmatrix} \begin{pmatrix} \Delta x \\ w \end{pmatrix} = \begin{pmatrix} -\nabla f(x) \\ 0 \end{pmatrix}. \quad (6.23)$$

The *Newton decrement* is

$$\lambda(x) = \|\Delta x\|_x = (D^2 f(x)[\Delta x, \Delta x])^{1/2}. \quad (6.24)$$

Proposition 6.25 (Properties of the Newton decrement). *Let x be feasible. Then*

$$\lambda(x)^2 = -\langle \nabla f(x), \Delta x \rangle. \quad (6.25)$$

If $\lambda(x) > 0$, then Δx is a feasible descent direction. Moreover,

$$\lambda(x) = 0$$

if and only if x minimizes f subject to $Ax = b$.

Algorithm 6.5 Damped Newton method for a self-concordant function

Input: A closed self-concordant function f with open domain $D = \text{dom } f$ and $\nabla^2 f \succ 0$, a full-row-rank matrix A , a feasible point x^0 , a threshold $\eta \in (0, \frac{1}{4}]$, and a tolerance $\varepsilon \in (0, \frac{1}{16}]$

Output: A feasible point x satisfying $f(x) - p^* \leq \varepsilon$

```

1: for  $k = 0, 1, 2, \dots$  do
2:   Compute  $\Delta x^k$  and  $\lambda_k = \lambda(x^k)$ 
3:   if  $\lambda_k^2 \leq \varepsilon$  then
4:     return  $x^k$ 
5:   end if
6:   if  $\lambda_k < \eta$  then
7:      $t_k \leftarrow 1$ 
8:   else
9:      $t_k \leftarrow \frac{1}{1 + \lambda_k}$ 
10:  end if
11:   $x^{k+1} \leftarrow x^k + t_k \Delta x^k$ 
12: end for

```

Theorem 6.26 (Convergence of damped Newton, [13]). *Let f be a closed self-concordant function with open domain $D = \text{dom } f$, and suppose that*

$$\nabla^2 f(x) \succ 0 \quad \text{for every } x \in D.$$

Suppose that f attains its finite minimum p^ on*

$$\{x \in D : Ax = b\}.$$

Define

$$\varrho(t) = t - \log(1 + t).$$

Then the damped Newton method has the following properties:

1. *Every iterate belongs to D and satisfies $Ax^k = b$.*

2. *During the damped phase, when $\lambda_k \geq \eta$,*

$$f(x^k) - f(x^{k+1}) \geq \varrho(\eta).$$

3. *Once $\lambda_k < \eta \leq \frac{1}{4}$, full Newton steps are taken and*

$$2\lambda_{k+1} \leq (2\lambda_k)^2.$$

4. *Whenever $\lambda_k \leq \frac{1}{4}$,*

$$f(x^k) - p^* \leq \lambda_k^2.$$

Consequently, the number of iterations required to obtain

$$f(x^k) - p^* \leq \varepsilon$$

is at most

$$\frac{f(x^0) - p^*}{\varrho(\eta)} + O(\log \log(1/\varepsilon)). \quad (6.26)$$

For fixed η , the dependence on the problem data enters only through the initial objective gap $f(x^0) - p^$.*

6.5 Smoothing families

A smoothing family replaces the cone indicator by a sequence of convex functions that are differentiable on suitable open domains. Each resulting subproblem is an equality-constrained smooth convex problem.

Definition 6.27 (Smoothing of an indicator). Let $K \subseteq \mathbb{R}^m$ be a closed convex cone. A *smoothing of δ_K* is a family

$$\{(\phi_\theta, A_\theta, b_\theta)\}_{\theta \in \Theta}$$

with the following properties for every $\theta \in \Theta$:

1. The function

$$\phi_\theta : \mathbb{R}^m \rightarrow \mathbb{R} \cup \{+\infty\}$$

is closed, proper, and convex.

2. Its interior domain

$$D_\theta = \text{int}(\text{dom } \phi_\theta)$$

is nonempty.

3. The function ϕ_θ is differentiable on D_θ , and $\nabla \phi_\theta$ is semismooth there.

4. For every $s \in D_\theta$ and every

$$H \in \partial(\nabla \phi_\theta)(s),$$

the matrix H is symmetric positive semidefinite.

5. The matrix A_θ and vector b_θ specify the affine constraint carried by the corresponding smoothed problem.

Definition 6.28 (Interior and exterior smoothing). A smoothing family is *interior* if

$$D_\theta \subseteq K \quad \text{for every } \theta \in \Theta.$$

It is *exterior* if

$$K \subseteq D_\theta \quad \text{for every } \theta \in \Theta.$$

Algorithm 6.6 Penalty method based on a smoothing family

Input: A differentiable convex function f , an affine mapping $c(x) = Cx + d$, a cone K , a smoothing family $\{(\phi_\theta, A_\theta, b_\theta)\}_{\theta \in \Theta}$, parameters (θ_k) , and tolerances (ε_k)

Output: An approximate solution of

$$\min_x f(x) \quad \text{subject to} \quad -c(x) \in K$$

1: **for** $k = 0, 1, 2, \dots$ **do**

2: Compute x^{k+1} , to accuracy ε_k , by solving

$$\min_x \{f(x) + \phi_{\theta_k}(-c(x))\} \quad \text{subject to} \quad A_{\theta_k} x = b_{\theta_k}$$

3: Choose the next parameter θ_{k+1}

4: **end for**

Proposition 6.29 (Optimality conditions for a smoothed problem). *Let*

$$c(x) = Cx + d,$$

fix $\theta \in \Theta$, and suppose that

$$-c(x) \in D_\theta, \quad A_\theta x = b_\theta.$$

Then x minimizes

$$f(x) + \phi_\theta(-c(x))$$

subject to $A_\theta x = b_\theta$ if and only if there exist vectors y and z such that

$$\begin{aligned} \nabla f(x) + C^\top z + A_\theta^\top y &= 0, \\ z &= -\nabla \phi_\theta(-c(x)), \\ A_\theta x &= b_\theta. \end{aligned} \tag{6.27}$$

Proof. Define

$$F_\theta(x) = f(x) + \phi_\theta(-c(x)).$$

This function is convex and differentiable at x . Since

$$D(-c)(x) = -C,$$

the chain rule gives

$$\nabla F_\theta(x) = \nabla f(x) - C^\top \nabla \phi_\theta(-c(x)).$$

A feasible point x minimizes F_θ subject to $A_\theta x = b_\theta$ if and only if its gradient is orthogonal to every feasible direction:

$$\langle \nabla F_\theta(x), h \rangle = 0 \quad \text{for every } h \in \ker A_\theta.$$

Since

$$(\ker A_\theta)^\perp = \text{range}(A_\theta^\top),$$

this is equivalent to the existence of y such that

$$\nabla F_\theta(x) + A_\theta^\top y = 0.$$

Setting

$$z = -\nabla \phi_\theta(-c(x))$$

gives

$$\nabla f(x) + C^\top z + A_\theta^\top y = 0.$$

Together with $A_\theta x = b_\theta$, this is precisely (6.27). Since the problem is convex, these first-order conditions are both necessary and sufficient. $\square$

Chapter 7

Interior-point method basics

7.1 Barrier smoothing

Definition 7.1 (Conic quadratic problem). Let

$$f(x) = \frac{1}{2} \langle x, Px \rangle + \langle q, x \rangle, \quad c(x) = Cx + d,$$

where $P \succeq 0$, and let $K \subseteq \mathbb{R}^m$ be a closed convex cone. We consider

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to} \quad -c(x) \in K. \quad (7.1)$$

Definition 7.2 (Slack formulation). Introducing the slack variable

$$s = -c(x),$$

problem (7.1) becomes

$$\min_{x,s} f(x) \quad \text{subject to} \quad s + c(x) = 0, \quad s \in K. \quad (7.2)$$

Definition 7.3 (Self-concordant barrier). Let K be a closed convex cone. A function

$$B : \mathbb{R}^m \rightarrow \mathbb{R} \cup \{+\infty\}$$

is a *self-concordant barrier* for K if:

1. B is closed, proper, and convex, with

$$\text{dom } B = \text{int } K;$$

2. B is three times differentiable and self-concordant on $\text{int } K$;

- 3.

$$B(s_k) \longrightarrow +\infty$$

whenever $s_k \in \text{relint } K$ converges to a point of $\text{rbd } K$.

Definition 7.4 (Barrier parameter). A self-concordant barrier B has *parameter* $\nu > 0$ if

$$\|\nabla B(s)\|_{s,B}^* \leq \sqrt{\nu} \quad \text{for every } s \in \text{int } K, \quad (7.3)$$

where

$$\|h\|_{s,B} = (h^\top \nabla^2 B(s) h)^{1/2}$$

and

$$\|u\|_{s,B}^* = (u^\top [\nabla^2 B(s)]^{-1} u)^{1/2}.$$

Equivalently,

$$|DB(s)[h]| \leq \sqrt{\nu} \|h\|_{s,B} \quad \text{for every } h \in \mathbb{R}^m.$$

Definition 7.5 (Barrier problem). Let B be a self-concordant barrier for K . For $\tau > 0$, the *barrier problem* is

$$\min_{x,s} \{f(x) + \tau B(s)\} \quad \text{subject to} \quad s + c(x) = 0. \quad (7.4)$$

Since $\tau > 0$, it has the same minimizers as

$$\min_{x,s} \left\{ \frac{f(x)}{\tau} + B(s) \right\} \quad \text{subject to} \quad s + c(x) = 0. \quad (7.5)$$

The rescaled formulation (7.5) is important. Although B is self-concordant, the function τB is self-concordant with the standard normalization only when $\tau \geq 1$. By contrast, $f/\tau + B$ is self-concordant for every $\tau > 0$, because f/τ is a convex quadratic and has zero third derivative.

Proposition 7.6 (Optimality conditions for the barrier problem). *Let $s \in \text{int } K$ and $s + c(x) = 0$. Then (x, s) solves (7.4) if and only if there exist $y, z \in \mathbb{R}^m$ satisfying*

$$\nabla f(x) + C^\top y = 0, \quad (7.6)$$

$$y - z = 0, \quad (7.7)$$

$$s + c(x) = 0, \quad (7.8)$$

$$z + \tau \nabla B(s) = 0. \quad (7.9)$$

Proof. Consequence of the general smoothing optimality condition. □

Definition 7.7 (Central path). Suppose that the barrier problem has a unique solution for every $\tau > 0$. The *central path* is

$$\{(x(\tau), s(\tau), y(\tau), z(\tau)) : \tau > 0\}, \quad (7.10)$$

where the point at parameter τ satisfies (7.6) to (7.9).

Algorithm 7.1 Barrier method

Input: A self-concordant barrier B with parameter ν , a strictly feasible point, an initial parameter $\tau_0 > 0$, a reduction factor $\rho \in (0, 1)$, a centering tolerance $\eta \in (0, \frac{1}{4}]$, and a final tolerance $\varepsilon > 0$

Output: An approximate primal-dual solution

```

1: for  $k = 0, 1, 2, \dots$  do
2:   if  $\nu\tau_k \leq \varepsilon$  then
3:     return the current iterate
4:   end if
5:   Approximately minimize

```

$$\frac{f(x)}{\tau_k} + B(s) \quad \text{subject to} \quad s + c(x) = 0$$

by damped Newton steps until the Newton decrement is at most η

```

6:   Set  $z \leftarrow -\tau_k \nabla B(s)$ 
7:   Set  $\tau_{k+1} \leftarrow \rho\tau_k$ 
8: end for

```

7.2 Short-step path following

Definition 7.8 (Local norm and dual local norm). Let g be self-concordant and suppose that $\nabla^2 g(v) \succ 0$. Define

$$\|h\|_{v,g} = (h^\top \nabla^2 g(v) h)^{1/2}, \quad \|u\|_{v,g}^* = (u^\top [\nabla^2 g(v)]^{-1} u)^{1/2}. \quad (7.11)$$

The Newton decrement is

$$\lambda_g(v) = \|\nabla g(v)\|_{v,g}^*.$$

For an equality-constrained problem, the decrement is instead computed from the constrained Newton direction.

For the barrier problem, write

$$g_\tau(x, s) = \frac{f(x)}{\tau} + B(s)$$

on the affine set $s + c(x) = 0$, and denote its constrained Newton decrement by $\lambda_\tau(x, s)$.

Proposition 7.9 (Effect of decreasing the barrier parameter). *Suppose that*

$$\lambda_\tau(v) \leq \eta$$

and set

$$\tau^+ = \left(1 - \frac{\gamma}{\sqrt{\nu}}\right) \tau, \quad 0 < \gamma < \sqrt{\nu}.$$

Then

$$\lambda_{\tau^+}(v) \leq \frac{\eta + \gamma}{1 - \gamma/\sqrt{\nu}}. \quad (7.12)$$

Algorithm 7.2 Short-step path-following method

Input: A barrier B with parameter ν , constants $\eta, \gamma > 0$ satisfying

$$\eta' := \frac{\eta + \gamma}{1 - \gamma/\sqrt{\nu}} \leq \frac{1}{4}, \quad 2(\eta')^2 \leq \eta,$$

 an initial parameter $\tau_0 > 0$, a tolerance $\varepsilon > 0$, and a point v^0 satisfying $\lambda_{\tau_0}(v^0) \leq \eta$
Output: A point with duality gap at most ε

```

1: for  $k = 0, 1, 2, \dots$  do
2:   if  $\nu\tau_k \leq \varepsilon$  then
3:     return  $v^k$ 
4:   end if
5:   Set

```

$$\tau_{k+1} \leftarrow \left(1 - \frac{\gamma}{\sqrt{\nu}}\right) \tau_k$$

```

6:   Take one full Newton step for  $g_{\tau_{k+1}}$  at  $v^k$ 
7:   Denote the resulting point by  $v^{k+1}$ 
8: end for

```

 The Newton system for g_τ is

$$\begin{pmatrix} \nabla^2 f(x)/\tau & 0 & C^\top \\ 0 & \nabla^2 B(s) & I \\ C & I & 0 \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta s \\ \Delta y \end{pmatrix} = - \begin{pmatrix} \nabla f(x)/\tau + C^\top y \\ \nabla B(s) + y \\ s + c(x) \end{pmatrix}. \quad (7.13)$$

Proposition 7.10 (Iteration count of the short-step method, [13]). *Every iterate of the short-step method satisfies*

$$\lambda_{\tau_k}(v^k) \leq \eta.$$

Moreover, the method reaches

$$\nu\tau_k \leq \varepsilon$$

after at most

$$\left\lceil \frac{\log(\nu\tau_0/\varepsilon)}{-\log(1 - \gamma/\sqrt{\nu})} \right\rceil \quad (7.14)$$

 iterations. In particular, for fixed γ ,

$$k = O\left(\sqrt{\nu} \log \frac{\nu\tau_0}{\varepsilon}\right). \quad (7.15)$$

 The interest of the short step method is at each step you don't have an inner damped newton only one step per τ .

7.3 Logarithmically homogeneous barriers

Definition 7.11 (Logarithmic homogeneity). A barrier B for K is *logarithmically homogeneous with parameter ν* if

$$B(ts) = B(s) - \nu \log t \quad \text{for every } s \in \text{int } K \text{ and } t > 0. \quad (7.16)$$

7.4 Barriers for the catalogue

Proposition 7.12 (Barrier of a symmetric cone). *Let K be a symmetric cone associated with a Euclidean Jordan algebra V of rank r . With respect to the canonical trace inner product,*

$$B(s) = -\log \det(s)$$

is a logarithmically homogeneous self-concordant barrier with parameter

$$\nu = r.$$

Its derivatives are in the canonical inner product:

$$\nabla B(s) = -s^{-1}, \quad \nabla^2 B(s) = Q(s)^{-1} = Q(s^{-1}). \quad (7.17)$$

The standard barriers for the classical symmetric cones are:

1. For the nonnegative orthant,

$$K = \mathbb{R}_+^n, \quad B(x) = -\sum_{i=1}^n \log x_i, \quad \nu = n.$$

2. For the Lorentz cone,

$$K = \mathcal{L}^{n+1} = \{(t, u) \in \mathbb{R} \times \mathbb{R}^n : t \geq \|u\|\},$$

the standard Euclidean barrier is

$$B(t, u) = -\log(t^2 - \|u\|^2), \quad \nu = 2.$$

3. For the positive semidefinite cone,

$$K = \mathcal{S}_+^n, \quad B(X) = -\log \det X, \quad \nu = n.$$

Remark 7.13 (Lorentz cone specific case). The Lorentz cone requires particular care because its canonical trace inner product is twice the usual Euclidean inner product:

$$\langle x, y \rangle_{\text{tr}} = 2 \langle x, y \rangle_{\text{Euc}}.$$

Hence in the classic inner product we have :

$$\nabla B(s) = -2s^{-1}$$

Proposition 7.14 (Barriers for the exponential and power cones, [13, 5]). *The function*

$$B_{\text{exp}}(s) = -\log(s_2 \log(s_3/s_2) - s_1) - \log s_2 - \log s_3 \quad (7.18)$$

is a logarithmically homogeneous self-concordant barrier for the exponential cone with parameter 3.

For $\alpha \in (0, 1)$, the function

$$B_\alpha(s) = -\log(s_1^{2\alpha} s_2^{2(1-\alpha)} - s_3^2) - (1-\alpha) \log s_1 - \alpha \log s_2 \quad (7.19)$$

is a logarithmically homogeneous self-concordant barrier for the power cone with parameter 3.

Proposition 7.15 (Euler's identity for a logarithmically homogeneous barrier). *Let $K \subseteq \mathbb{R}^m$ be a cone and let B be differentiable and ν -logarithmically homogeneous, that is*

$$B(\lambda s) = B(s) - \nu \log \lambda \quad \text{for all } s \in \text{int } K, \lambda > 0. \quad (7.20)$$

Then

$$\langle s, -\nabla B(s) \rangle = \nu \quad \text{for all } s \in \text{int } K. \quad (7.21)$$

Proof. Fix $s \in \text{int } K$ and let

$$\varphi(\lambda) = B(\lambda s)$$

for $\lambda > 0$, which is well defined since $\text{int } K$ is a cone.

Differentiating the left side by the chain rule gives

$$\varphi'(\lambda) = \langle \nabla B(\lambda s), s \rangle$$

, and differentiating the right side of (7.20) gives $\varphi'(\lambda) = -\nu/\lambda$. Evaluating at $\lambda = 1$ yields $\langle \nabla B(s), s \rangle = -\nu$, which is (7.21). $\square$

Euler identity is useful cause it permit you to know where you are on the central path.

Corollary 7.16 (The pairing on the central path). *If $z = -\tau \nabla B(s)$ for some $\tau > 0$, then*

$$\langle s, z \rangle = \nu \tau. \quad (7.22)$$

Proof. $\langle s, z \rangle = \langle s, -\tau \nabla B(s) \rangle = \nu \tau$ by Proposition 7.15. $\square$

Proposition 7.17 (Barrier of a product). *Let*

$$K = K_1 \times \cdots \times K_p,$$

and let B_j be a logarithmically homogeneous self-concordant barrier for K_j with parameter ν_j . Then

$$B(s) = \sum_{j=1}^p B_j(s_j) \quad (7.23)$$

is a logarithmically homogeneous self-concordant barrier for K with parameter

$$\nu = \sum_{j=1}^p \nu_j. \quad (7.24)$$

Remark 7.18 (Zero-cone blocks). Let

$$K_j = \{0\}.$$

This cone has empty interior in its ambient space, but its relative interior is

$$\text{relint } K_j = \{0\},$$

and its relative boundary is empty. Under the relative-interior convention adopted here, any constant function on $\{0\}$ is therefore a barrier by vacuity. We choose

$$B_j \equiv 0,$$

with barrier parameter $\nu_j = 0$.

Traditionally, zero-cone blocks are excluded from the definition of a barrier and treated directly as equality constraints. We retain them in the present framework to obtain a uniform treatment of product cones.

Since the Hessian of a constant barrier is zero, it provides no curvature to the Newton system. In the implementation, the zero block is therefore replaced by a fixed positive regularization block of the form

$$H_j = \Lambda_j I, \quad \Lambda_j > 0.$$

This block comes from the augmented-Lagrangian or proximal regularization(that we will view later on); it is not the Hessian of the zero-cone barrier.

7.5 Primal-dual Newton systems

Let

$$v = (x, s, y, z),$$

where y is the multiplier of $s + c(x) = 0$ and z is the cone dual variable. The residuals common to all the systems considered below are

$$r_1 = Px + q + C^\top y, \tag{7.25}$$

$$r_2 = y - z, \tag{7.26}$$

$$r_3 = s + c(x). \tag{7.27}$$

The final residual r_4 describes complementarity.

A general linearized system has the form

$$\begin{pmatrix} P & 0 & C^\top & 0 \\ 0 & 0 & I & -I \\ C & I & 0 & 0 \\ 0 & H & 0 & I \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta s \\ \Delta y \\ \Delta z \end{pmatrix} = - \begin{pmatrix} r_1 \\ r_2 \\ r_3 \\ r_4 \end{pmatrix}. \tag{7.28}$$

Eliminating

$$\Delta z = -H\Delta s - r_4$$

gives

$$\begin{pmatrix} P & 0 & C^\top \\ 0 & H & I \\ C & I & 0 \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta s \\ \Delta y \end{pmatrix} = - \begin{pmatrix} r_1 \\ r_2 + r_4 \\ r_3 \end{pmatrix}. \tag{7.29}$$

This matrix is symmetric whenever H is symmetric.

Further elimination gives

$$(P + C^\top H C) \Delta x = -r_1 + C^\top (r_2 + r_4) - C^\top H r_3. \tag{7.30}$$

The remaining directions are recovered from

$$\Delta s = -r_3 - C \Delta x, \tag{7.31}$$

$$\Delta z = -H \Delta s - r_4, \tag{7.32}$$

$$\Delta y = \Delta z - r_2. \tag{7.33}$$

Definition 7.19 (Admissible Newton system). Let G_τ denote the nonlinear residual whose linearization produces the Newton system. The system is *admissible* if

$$G_\tau(v) = 0$$

holds exactly at the central-path points with parameter τ .

Definition 7.20 (Tangent Newton system). An admissible Newton system is *tangent* if, at every central-path point,

$$H = \tau \nabla^2 B(s). \quad (7.34)$$

Thus its linearization agrees with the exact barrier linearization on the central path. Away from the path, H may be an approximation.

Remark 7.21. Admissibility concerns the zero set of the nonlinear residual. Tangency concerns its first-order behavior at that zero set. These are different properties.

At a central-path point, the residual and the Newton step are both zero, so the value of the step cannot be used to test tangency. Tangency is a statement about the derivative of the residual.

7.5.1 Exact barrier Newton system

For the barrier residual

$$r_4 = z + \tau \nabla B(s),$$

the exact derivative with respect to s is

$$H = \tau \nabla^2 B(s).$$

The resulting system is both admissible and tangent.

After eliminating Δz , it coincides with the Newton system (7.13), up to the rescaling of the equality multiplier caused by dividing the objective by τ .

7.5.2 Averaged Newton system

Definition 7.22 (Averaged Newton system). Let B be logarithmically homogeneous with parameter ν , and define the current average complementarity by

$$\mu = \frac{\langle s, z \rangle}{\nu}. \quad (7.35)$$

The *averaged Newton system* uses

$$r_4 = z + \tau \nabla B(s)$$

and

$$H = \mu \nabla^2 B(s).$$

Thus

$$\begin{pmatrix} P & 0 & C^\top & 0 \\ 0 & 0 & I & -I \\ C & I & 0 & 0 \\ 0 & \mu \nabla^2 B(s) & 0 & I \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta s \\ \Delta y \\ \Delta z \end{pmatrix} = - \begin{pmatrix} Px + q + C^\top y \\ y - z \\ s + c(x) \\ z + \tau \nabla B(s) \end{pmatrix}. \quad (7.36)$$

After eliminating Δz ,

$$\begin{pmatrix} P & 0 & C^\top \\ 0 & \mu \nabla^2 B(s) & I \\ C & I & 0 \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta s \\ \Delta y \end{pmatrix} = - \begin{pmatrix} Px + q + C^\top y \\ y + \tau \nabla B(s) \\ s + c(x) \end{pmatrix}. \quad (7.37)$$

Proposition 7.23 (Admissibility and tangency of the averaged system). *The averaged Newton system is admissible. It is also tangent on the central path.*

Proof. The nonlinear residuals are

$$r_1, \quad r_2, \quad r_3, \quad r_4 = z + \tau \nabla B(s).$$

Their simultaneous vanishing is exactly the optimality system of the barrier problem. The choice of H changes only the linearized direction and does not change the zero set. Hence the system is admissible.

On the central path by Euler identity:

$$\langle s, z \rangle = -\tau \langle s, \nabla B(s) \rangle = \nu \tau.$$

Thus

$$\mu = \frac{\langle s, z \rangle}{\nu} = \tau,$$

and therefore

$$H = \mu \nabla^2 B(s) = \tau \nabla^2 B(s).$$

Hence the system is tangent. □

7.5.3 Jordan Newton system

In this subsection, K is symmetric and solid and the Jordan variables are written using the canonical trace inner product.

Proposition 7.24 (Jordan form of centrality). *Let*

$$B(s) = -\log \det(s).$$

For $s, z \in \int K$,

$$z + \tau \nabla B(s) = 0$$

supposing that $\langle \cdot, \cdot \rangle_{\text{can}} = C \langle \cdot, \cdot \rangle$ to:

$$s \circ z = C\tau e. \tag{7.38}$$

Proof. With respect to the used inner product:

$$\nabla B(s) = -Cs^{-1}.$$

Therefore,

$$z + \tau \nabla B(s) = 0 \iff z = C\tau s^{-1}.$$

Taking the Jordan product with s gives

$$s \circ z = C\tau(s \circ s^{-1}) = C\tau e.$$

□

Definition 7.25 (Jordan Newton system). Linearizing

$$s \circ z - C\tau e = 0$$

gives

$$L(z)\Delta s + L(s)\Delta z = -(s \circ z - C\tau e).$$

Since $s \in \text{int } K$, the operator $L(s)$ is invertible. Multiplying by $L(s)^{-1}$ gives

$$\Delta z + L(s)^{-1}L(z)\Delta s = -L(s)^{-1}(s \circ z - C\tau e).$$

Thus the Jordan system uses

$$H = L(s)^{-1}L(z) \tag{7.39}$$

and

$$r_4 = L(s)^{-1}(s \circ z - C\tau e) = z + \tau \nabla B(s). \tag{7.40}$$

The operator $L(s)^{-1}L(z)$ is generally not symmetric because $L(s)$ and $L(z)$ need not commute. This is the reason for introducing Nesterov-Todd scaling.

Proposition 7.26 (Admissibility of the Jordan system). *The Jordan Newton system is admissible.*

Proof. For $s, z \in \text{int } K$,

$$s \circ z = C\tau e$$

is equivalent to

$$z + \tau \nabla B(s) = 0.$$

Thus the Jordan system has the same zero set as the barrier optimality system and is admissible. $\square$

7.5.4 Nesterov-Todd scaling

Definition 7.27 (Nesterov-Todd scaling). Let K be symmetric and let $s, z \in \text{int } K$. The *Nesterov-Todd scaling* between s and z is the unique self-adjoint positive-definite automorphism $W \in \text{Aut}(K)$ satisfying

$$Ws = z.$$

It is given by

$$W = Q(w), \quad w = Q(s^{-1/2}) \left(Q(s^{1/2})z \right)^{1/2}. \tag{7.41}$$

Definition 7.28 (Nesterov-Todd Newton system). Define the scaled variables

$$\tilde{s} = W^{1/2}s, \quad \tilde{z} = W^{-1/2}z.$$

Since $Ws = z$,

$$\tilde{s} = \tilde{z}.$$

Linearizing the Jordan complementarity equation in these scaled variables, while holding W fixed, produces the symmetric block

$$H = W.$$

The resulting Newton system is

$$\begin{pmatrix} P & 0 & C^\top & 0 \\ 0 & 0 & I & -I \\ C & I & 0 & 0 \\ 0 & W & 0 & I \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta s \\ \Delta y \\ \Delta z \end{pmatrix} = - \begin{pmatrix} Px + q + C^\top y \\ y - z \\ s + c(x) \\ z + \tau \nabla B(s) \end{pmatrix}. \quad (7.42)$$

After eliminating Δz ,

$$\begin{pmatrix} P & 0 & C^\top \\ 0 & W & I \\ C & I & 0 \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta s \\ \Delta y \end{pmatrix} = - \begin{pmatrix} Px + q + C^\top y \\ y + \tau \nabla B(s) \\ s + c(x) \end{pmatrix}. \quad (7.43)$$

Proposition 7.29 (Properties of the Nesterov-Todd system). *The Nesterov-Todd system is admissible, its block W is symmetric positive definite, and it is tangent on the central path.*

Proof. The nonlinear residuals are the barrier optimality residuals, so the system is admissible. By construction, W is self-adjoint and positive definite.

On the central path,

$$z = C\tau s^{-1}.$$

Logarithmic homogeneity differentiated twice gives at $\lambda = 1$:

$$\nabla^2 B(s)s = -\nabla B(s) = Cs^{-1},$$

and hence

$$\tau \nabla^2 B(s)s = z.$$

The operator $\tau \nabla^2 B(s)$ is a self-adjoint positive-definite automorphism of K mapping s to z . By uniqueness of the Nesterov-Todd scaling,

$$W = \tau \nabla^2 B(s).$$

Therefore, the system is tangent. $\square$

Proposition 7.30 (Short-step complexity with Nesterov-Todd scaling, [14]). *For a symmetric cone, the short-step method using the Nesterov-Todd system reaches a duality gap of ε in*

$$O\left(\sqrt{\nu} \log \frac{1}{\varepsilon}\right)$$

iterations, with one Newton step per parameter update.

7.6 Predictor-corrector methods

The short-step schedule has a polynomial complexity bound, but its reduction factor approaches 1 as ν grows. Practical solvers therefore use more aggressive parameter updates.

For example, if $K = \mathbb{R}_+^{1000}$, then $\nu = 1000$. With $\gamma = 1/16$, the short-step reduction factor is

$$1 - \frac{\gamma}{\sqrt{\nu}} \approx 0.998.$$

Reducing the parameter by eight orders of magnitude then requires about 9300 iterations. Practical primal-dual solvers usually require only a few tens of iterations, although this faster behavior is not described by the short-step worst-case bound.

Definition 7.31 (Fast Newton system). A tangent Newton system is called *fast* if its matrix block H does not depend explicitly on the target parameter τ . Changing τ then changes only the right-hand side, so several directions can be computed using the same matrix factorization.

The averaged, Jordan, and Nesterov-Todd systems are fast. The exact barrier system, whose block is

$$H = \tau \nabla^2 B(s),$$

is not fast.

Definition 7.32 (Affine predictor step). The *affine predictor step* is obtained by setting the target complementarity parameter to

$$\tau = 0$$

in a fast Newton system.

Definition 7.33 (Maximum affine step lengths). Let

$$(\Delta x^a, \Delta s^a, \Delta y^a, \Delta z^a)$$

be the affine predictor direction. Define

$$\alpha_p^a = \sup \{ \alpha \in [0, 1] : s + \alpha \Delta s^a \in K \}, \quad (7.44)$$

$$\alpha_d^a = \sup \{ \alpha \in [0, 1] : z + \alpha \Delta z^a \in K^* \}. \quad (7.45)$$

Definition 7.34 (Affine complementarity ratio). Let

$$\mu = \frac{\langle s, z \rangle}{\nu}$$

and define

$$\mu^a = \frac{1}{\nu} \left\langle s + \alpha_p^a \Delta s^a, z + \alpha_d^a \Delta z^a \right\rangle.$$

The *affine complementarity ratio* is

$$r = \frac{\mu^a}{\mu}. \quad (7.46)$$

If r is small intuitively the affine step worked and we should trust it otherwise when it is near 1 we shouldn't trust it as it didn't made the situation better. A way to transform this intuition into calculation is Mehrotra's heuristic:

Definition 7.35 (Mehrotra centering rule, [11]). Let

$$0 < \sigma_{\min} < \sigma_{\max} < 1.$$

The Mehrotra centering parameter is

$$\sigma = \min \left\{ \sigma_{\max}, \max \{ \sigma_{\min}, r^3 \} \right\}, \quad (7.47)$$

and the target complementarity is

$$\tau = \sigma \mu.$$

Definition 7.36 (Corrector term). For a symmetric cone the exact expansion is

$$(s + \Delta s) \circ (z + \Delta z) = s \circ z + L(z)\Delta s + L(s)\Delta z + \Delta s \circ \Delta z,$$

in which no inner product convention appears. The Newton linearisation omits the final quadratic term, and the *corrector* estimates it by the affine predictor $\Delta s^a \circ \Delta z^a$. Since the centrality target is $s \circ z = C\tau e$, the corrected residual is

$$r_4 = s \circ z + \Delta s^a \circ \Delta z^a - C\sigma\tau e, \quad (7.48)$$

the constant C entering through the target alone.

For a nonsymmetric cone there is no Jordan product, so neither the corrector nor C is defined; those blocks carry the centring target in the barrier form,

$$r_4 = z + \sigma\tau \nabla B(s), \quad (7.49)$$

and receive pure centring.

The barrier is finite only on the interior of the cone, so a step must stop short of the boundary. The rule that does so is the following, and it is used by every interior point method below.

Definition 7.37 (Largest feasible step and fraction to the boundary). Let K be a closed convex cone, $v \in \text{int } K$ and Δv a direction. The *largest feasible step* along Δv is

$$\bar{\alpha}(v, \Delta v) = \sup\{\alpha \geq 0 : v + \alpha\Delta v \in K\} \in (0, +\infty], \quad (7.50)$$

infinite exactly when the ray never leaves the cone. For a parameter $\theta \in (0, 1)$, the *fraction to the boundary* rule takes the step

$$\alpha = \min(1, \theta \bar{\alpha}(v, \Delta v)). \quad (7.51)$$

On a product cone the step is the smallest over the blocks, and the primal and the dual variables each take their own, giving a pair (α_p, α_d) .

Proposition 7.38 (The rule keeps the iterate interior). *With $v \in \text{int } K$, one has $\bar{\alpha} > 0$, and*

$$v + \alpha\Delta v \in \text{int } K \quad \text{for every } 0 \leq \alpha < \bar{\alpha}. \quad (7.52)$$

In particular the step of Definition 7.37 is strictly positive and lands strictly inside the cone, since $\theta < 1$.

Taking θ strictly below one is therefore not a safeguard against a numerical accident but the condition under which the next iteration exists at all: at $\alpha = \bar{\alpha}$ the barrier is infinite and its Hessian singular, and there would be no scaling to build.

Algorithm 7.3 Predictor-corrector method

Input: A fast admissible Newton system, an interior initial point $s^0 \in \text{relint } K$, $z^0 \in \text{relint } K^*$, initial points x^0, y^0 , constants $0 < \sigma_{\min} < \sigma_{\max} < 1$, a fraction-to-the-boundary parameter $\theta \in (0, 1)$, and a tolerance $\varepsilon > 0$

Output: An approximate primal-dual solution

- 1: **for** $k = 0, 1, 2, \dots$ **do**
- 2: Compute

$$\mu_k = \frac{\langle s^k, z^k \rangle}{\nu}$$

- 3: **if** $\nu\mu_k \leq \varepsilon$ **then**
- 4: **return** (x^k, s^k, y^k, z^k)
- 5: **end if**
- 6: Assemble and factorize the Newton matrix
- 7: Solve the system with target $\tau = 0$ to obtain the affine predictor direction
- 8: Compute $\alpha_p^a, \alpha_d^a, \mu^a$, and $r = \mu^a / \mu_k$
- 9: Set

$$\sigma = \min \left\{ \sigma_{\max}, \max \{ \sigma_{\min}, r^3 \} \right\}$$

- 10: Reuse the factorization to solve the corrected system with target $\tau = \sigma\mu_k$
- 11: Compute the maximum primal and dual step lengths α_p and α_d
- 12: Update

$$x^{k+1} = x^k + \theta\alpha_p\Delta x, \quad s^{k+1} = s^k + \theta\alpha_p\Delta s$$

- 13: Update

$$y^{k+1} = y^k + \theta\alpha_d\Delta y, \quad z^{k+1} = z^k + \theta\alpha_d\Delta z$$

- 14: **end for**
-

Remark 7.39 (Theory and practice). The short-step method has a polynomial complexity bound but uses conservative parameter reductions. The Mehrotra predictor-corrector method uses much more aggressive reductions and is usually far faster in practice.

The centering rule and corrector are heuristic. Consequently, the short-step complexity proof does not apply directly to the practical predictor-corrector algorithm. Its effectiveness is empirical and is supported by extensive solver experience rather than by the short-step iteration bound.

Chapter 8

Augmented Lagrangian method basics

8.1 Augmented Lagrangian smoothing family

Throughout this chapter the problem is

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to} \quad c(x) \preceq_K 0, \quad (8.1)$$

with $f(x) = \frac{1}{2} \langle x, Px \rangle + \langle q, x \rangle$ and $c(x) = Cx + d$, and $s = -c(x)$ denotes the slack, so that the constraint is $s \in K$.

Definition 8.1 (Augmented Lagrangian smoothing). Let $\theta = (M, z_c)$ with $M \in \mathcal{A}(K)$ and $z_c \in \mathbb{R}^m$. The *augmented Lagrangian smoothing* of δ_K is

$$\phi_\theta^{\text{AL}}(s) = \sup_{z \in K^*} \left\{ -\langle z, s \rangle - \frac{1}{2} \|z - z_c\|_M^2 \right\} = \frac{1}{2} \|\Pi_{K^*}(Mz_c - s)\|_{M^{-1}}^2 - \frac{1}{2} \|z_c\|_M^2. \quad (8.2)$$

Definition 8.2 (Moreau smoothing). With $\theta = (M, z_c)$ as above, the *Moreau smoothing* of δ_K is the Moreau envelope of δ_K in the metric M^{-1} evaluated at the shifted point,

$$\phi_\theta^{\text{Mor}}(s) = e_{\delta_K}^{M^{-1}}(s - Mz_c) = \frac{1}{2} \text{dist}_{M^{-1}}(s - Mz_c, K)^2. \quad (8.3)$$

Proposition 8.3 (The two smoothings differ by a constant). *Let $\theta = (M, z_c)$. Then, for every $s \in \mathbb{R}^m$,*

$$\phi_\theta^{\text{Mor}}(s) = \phi_\theta^{\text{AL}}(s) + \frac{1}{2} \|z_c\|_M^2. \quad (8.4)$$

Proof. Since M is admissible M^{-1} too hence in this metric:

$$\text{dist}(s - Mz_c, K) = \|\Pi_{K^*}(Mz_c - s)\|_{M^{-1}}$$

□

Proposition 8.4 (Optimality conditions for the smoothed subproblem). *Let*

$$\theta = (M, z_c),$$

and consider

$$\min_x \{f(x) + \phi_\theta(-c(x))\}, \quad (8.5)$$

where ϕ_θ is either ϕ_θ^{Mor} or ϕ_θ^{AL} .

Then x minimizes (8.5) if and only if there exists z such that

$$\nabla f(x) + C^\top z = 0, \quad z = M^{-1} \Pi_{K^*}(Mz_c + c(x)). \quad (8.6)$$

Proof. Consequence of the general smoothing optimality condition, combined with the fact that :

$$\nabla \phi_\theta(u) = \nabla \left(\frac{1}{2} \|\Pi_{K^*}\|_{M^{-1}}^2 (Mz_c - \cdot) + \text{cst} \right)(u) = -M^{-1} \Pi_{K^*}(Mz_c - u)$$

□

Algorithm 8.1 Augmented Lagrangian method

Input: a sequence (M_k) in $\mathcal{A}(K)$, an initial $z_c^0 \in K^*$, inner tolerances (ε_k)

Output: an approximate solution of the problem

1: **for** $k = 0, 1, 2, \dots$ **do**

2: solve, to accuracy ε_k and by the semismooth Newton method,

$$x^{k+1} \in \underset{x}{\operatorname{argmin}} f(x) + \phi_{(M_k, z_c^k)}(-c(x))$$

3: $z^{k+1} \leftarrow M_k^{-1} \Pi_{K^*}(M_k z_c^k + c(x^{k+1}))$

4: $z_c^{k+1} \leftarrow z^{k+1}$

5: **end for**

8.2 The augmented Lagrangian method as a proximal point algorithm

Definition 8.5 (Dual optimality operator). Let g be the dual function, and assume that the dual problem admits at least one point with finite dual value. Define

$$h(z) = -g(z) + \delta_{K^*}(z). \quad (8.7)$$

Then h is closed, proper, and convex, and its minimizers are precisely the solutions of the dual problem.

The *dual optimality operator* is

$$T_D = \partial h = \partial(-g + \delta_{K^*}). \quad (8.8)$$

Proposition 8.6 (Maximal monotonicity of the dual optimality operator). *The operator T_D is maximal monotone, and*

$$T_D^{-1}(0) = \underset{z \in K^*}{\operatorname{argmax}} g(z). \quad (8.9)$$

Proof. The dual function g is concave and upper semicontinuous. Therefore, $-g$ is closed and convex. Since K^* is closed and convex, δ_{K^*} is also closed and convex. By the assumed existence of a dual feasible point with finite dual value,

$$h = -g + \delta_{K^*}$$

is proper.

The subdifferential of a closed proper convex function is maximal monotone. Hence

$$T_D = \partial h$$

is maximal monotone.

Finally, Fermat's rule gives

$$0 \in T_D(z) \iff z \in \operatorname{argmin} h.$$

Since

$$h(z) = \begin{cases} -g(z), & z \in K^*, \\ +\infty, & z \notin K^*, \end{cases}$$

minimizing h is equivalent to maximizing g over K^* . $\square$

Proposition 8.7 (The method of multipliers as a proximal point method, [19]). *Let $M_k \in \mathcal{A}(K)$. Suppose that the augmented Lagrangian subproblem is solved exactly at iteration k , producing (x^{k+1}, z^{k+1}) . Then the multiplier update satisfies*

$$z^{k+1} = J_{T_D}^{M_k}(z^k). \quad (8.10)$$

Thus the multiplier sequence is the variable-metric proximal point iteration applied to the dual optimality operator T_D .

Proof. Fix an iteration and write

$$M = M_k, \quad z_c = z^k, \quad x^+ = x^{k+1}, \quad z^+ = z^{k+1}.$$

The optimality conditions of the augmented Lagrangian subproblem give

$$\nabla f(x^+) + C^\top z^+ = 0 \quad (8.11)$$

and

$$z^+ = M^{-1} \Pi_{K^*} (M z_c + c(x^+)). \quad (8.12)$$

Equation (8.11) says that x^+ minimizes $L(\cdot, z^+)$. Consequently,

$$g(z^+) = L(x^+, z^+).$$

For every z , the definition of the dual function gives

$$g(z) \leq L(x^+, z) \quad (8.13)$$

$$= L(x^+, z^+) + \langle z - z^+, c(x^+) \rangle \quad (8.14)$$

$$= g(z^+) + \langle z - z^+, c(x^+) \rangle. \quad (8.15)$$

Thus $c(x^+)$ is a supergradient of g at z^+ , or equivalently,

$$-c(x^+) \in \partial(-g)(z^+). \quad (8.16)$$

By admissibility, (8.12) is equivalent to

$$z^+ = \Pi_{K^*} (z_c + M^{-1} c(x^+))^M.$$

The metric projection characterization therefore gives

$$M(z_c + M^{-1} c(x^+) - z^+) \in N_{K^*}(z^+).$$

Equivalently,

$$M(z_c - z^+) + c(x^+) \in \partial \delta_{K^*}(z^+). \quad (8.17)$$

Adding (8.16) and (8.17), we obtain

$$M(z_c - z^+) \in \partial(-g)(z^+) + \partial\delta_{K^*}(z^+) \subseteq \partial h(z^+).$$

Therefore,

$$M(z_c - z^+) \in T_D(z^+),$$

which is equivalent to

$$z^+ = J_{T_D}^M(z_c).$$

□

8.3 The proximal point method on the KKT operator

Definition 8.8 (KKT operator). For $v = (x, z) \in \mathbb{R}^{n+m}$, define

$$T(x, z) = \begin{pmatrix} Px + q + C^\top z \\ -c(x) + N_{K^*}(z) \end{pmatrix}. \quad (8.18)$$

Equivalently,

$$T(x, z) = \begin{pmatrix} Px + q + C^\top z \\ -c(x) + \partial\delta_{K^*}(z) \end{pmatrix}.$$

Proposition 8.9 (Maximal monotonicity and zeros). *The KKT operator T is maximal monotone. Moreover,*

$$0 \in T(x, z)$$

if and only if (x, z) satisfies the KKT conditions

$$Px + q + C^\top z = 0, \quad (8.19)$$

$$-c(x) \in K, \quad (8.20)$$

$$z \in K^*, \quad (8.21)$$

$$\langle z, c(x) \rangle = 0. \quad (8.22)$$

Proof. Write

$$T = T_1 + T_2,$$

where

$$T_1(x, z) = \begin{pmatrix} P & C^\top \\ -C & 0 \end{pmatrix} \begin{pmatrix} x \\ z \end{pmatrix} + \begin{pmatrix} q \\ -d \end{pmatrix}$$

and

$$T_2(x, z) = \begin{pmatrix} 0 \\ N_{K^*}(z) \end{pmatrix}.$$

The symmetric part of the linear map defining T_1 is

$$\begin{pmatrix} P & 0 \\ 0 & 0 \end{pmatrix} \succeq 0.$$

Indeed,

$$\langle T_1(v) - T_1(v'), v - v' \rangle = \langle x - x', P(x - x') \rangle \geq 0.$$

Thus T_1 is monotone. Since it is affine and defined on the whole space, it is maximal monotone.

The function

$$(x, z) \mapsto \delta_{K^*}(z)$$

is closed, proper, and convex, and its subdifferential is T_2 . Hence T_2 is maximal monotone. Since T_1 has full domain, the maximal monotone sum theorem implies that $T = T_1 + T_2$ is maximal monotone.

Now $0 \in T(x, z)$ means

$$Px + q + C^\top z = 0$$

and

$$c(x) \in N_{K^*}(z).$$

For $z \in K^*$,

$$N_{K^*}(z) = (K^*)^\circ \cap z^\perp = -K \cap z^\perp.$$

Therefore,

$$c(x) \in N_{K^*}(z)$$

is equivalent to

$$-c(x) \in K, \quad z \in K^*, \quad \langle z, c(x) \rangle = 0.$$

These are precisely the KKT conditions. $\square$

Algorithm 8.2 Proximal method of multipliers

Input: Sequences (ρ_k) with $\rho_k > 0$, admissible metrics $M_k \in \mathcal{A}(K)$, an initial center (x_c^0, z_c^0) , and summable errors (ε_k)

Output: An approximate point of $T^{-1}(0)$

1: **for** $k = 0, 1, 2, \dots$ **do**

2: Set

$$\mathcal{M}_k = \text{diag}(\rho_k I_n, M_k)$$

3: Compute (x^{k+1}, z^{k+1}) satisfying

$$\left\| (x^{k+1}, z^{k+1}) - J_T^{\mathcal{M}_k}(x_c^k, z_c^k) \right\|_{\mathcal{M}_k} \leq \varepsilon_k$$

4: Update

$$(x_c^{k+1}, z_c^{k+1}) \leftarrow (x^{k+1}, z^{k+1})$$

5: **end for**

8.4 Primal formulation of a proximal step

Definition 8.10 (Augmented Lagrangian penalty). Let $M \in \mathcal{A}(K)$ and $z_c \in \mathbb{R}^m$. Define

$$\phi_{(M, z_c)}^{\text{AL}}(s) = \frac{1}{2} \|\Pi_{K^*}(Mz_c - s)\|_{M^{-1}}^2 - \frac{1}{2} \|z_c\|_M^2. \quad (8.23)$$

Then

$$\nabla \phi_{(M, z_c)}^{\text{AL}}(s) = -M^{-1} \Pi_{K^*}(Mz_c - s). \quad (8.24)$$

Proposition 8.11 (Primal formulation of one proximal step). *Let*

$$\rho > 0, \quad M \in \mathcal{A}(K), \quad \mathcal{M} = \text{diag}(\rho I_n, M),$$

and let

$$v_c = (x_c, z_c).$$

Define

$$\varphi_c(x) = f(x) + \frac{\rho}{2} \|x - x_c\|^2 + \phi_{(M, z_c)}^{\text{AL}}(-c(x)). \quad (8.25)$$

Then

$$v^+ = (x^+, z^+) = J_T^{\mathcal{M}}(v_c)$$

if and only if x^+ minimizes φ_c and

$$z^+ = M^{-1} \Pi_{K^*} \left(M z_c + c(x^+) \right). \quad (8.26)$$

Proof. The resolvent inclusion

$$\mathcal{M}(v_c - v^+) \in T(v^+)$$

is equivalent to

$$\rho(x_c - x^+) = P x^+ + q + C^\top z^+, \quad (8.27)$$

$$M(z_c - z^+) \in -c(x^+) + N_{K^*}(z^+). \quad (8.28)$$

Set

$$w = z_c + M^{-1} c(x^+).$$

The second inclusion becomes

$$M(w - z^+) \in N_{K^*}(z^+).$$

By the characterization of the metric projection,

$$z^+ = \Pi_{K^*}(w)^M.$$

Admissibility gives

$$\Pi_{K^*}(w)^M = \Pi_{K^*}(w) = M^{-1} \Pi_{K^*}(Mw).$$

Since

$$Mw = M z_c + c(x^+),$$

we obtain (8.26).

Now set

$$\sigma(x) = M z_c + c(x).$$

Differentiating (8.25) gives

$$\nabla \varphi_c(x) = P x + q + \rho(x - x_c) + C^\top M^{-1} \Pi_{K^*}(\sigma(x)).$$

Using (8.26), the condition

$$\nabla \varphi_c(x^+) = 0$$

is precisely (8.27). Since φ_c is strongly convex, this stationarity condition is equivalent to x^+ being its unique minimizer. $\square$

Proposition 8.12 (Properties of the primal objective). *The function φ_c is finite and continuously differentiable on $\mathbb{R}^n$. It is strongly convex with modulus ρ , and therefore has a unique minimizer.*

Its gradient is

$$\nabla\varphi_c(x) = Px + q + \rho(x - x_c) + C^\top M^{-1} \Pi_{K^*}(\sigma(x)), \quad \sigma(x) = Mz_c + c(x). \quad (8.29)$$

For every

$$F \in \partial \Pi_{K^*}(\cdot)(\sigma(x)),$$

the matrix

$$G_F(x) = P + \rho I + C^\top M^{-1} FC \quad (8.30)$$

belongs to

$$\partial(\nabla\varphi_c)(x)$$

and satisfies

$$G_F(x) \succeq \rho I. \quad (8.31)$$

At a point where the projection is differentiable,

$$\nabla^2\varphi_c(x) = P + \rho I + C^\top M^{-1} FC,$$

with $F = D\Pi_{K^*}(\cdot)(\sigma(x))$.

Proof. The squared projection potential is finite and continuously differentiable. Consequently, φ_c is finite and continuously differentiable, with gradient (8.29).

The function f is convex, the augmented Lagrangian penalty is convex, and

$$x \mapsto \frac{\rho}{2} \|x - x_c\|^2$$

is strongly convex with modulus ρ . Thus φ_c is strongly convex with modulus ρ .

The Clarke chain rule gives

$$G_F(x) \in \partial(\nabla\varphi_c)(x).$$

Admissibility implies that M^{-1} commutes with F , and

$$M^{-1}F$$

is symmetric positive semidefinite. Hence

$$C^\top M^{-1} FC \succeq 0.$$

Since $P \succeq 0$,

$$G_F(x) = P + \rho I + C^\top M^{-1} FC \succeq \rho I.$$

□

The semismooth Newton direction is therefore defined by

$$(P + \rho I + C^\top M^{-1} FC)\Delta x = -\nabla\varphi_c(x), \quad F \in \partial \Pi_{K^*}(\cdot)(\sigma(x)). \quad (8.32)$$

Algorithm 8.3 Semismooth Newton method for the primal proximal problem

Input: Parameters ρ, M , centers (x_c, z_c) , an initial point x^0 , a merit function, a line-search rule, and a tolerance $\varepsilon > 0$

Output: A point x satisfying $\|\nabla\varphi_c(x)\| \leq \varepsilon$

```

1: for  $j = 0, 1, 2, \dots$  do
2:   if  $\|\nabla\varphi_c(x^j)\| \leq \varepsilon$  then
3:     return  $x^j$ 
4:   end if
5:   Set

```

$$\sigma^j = Mz_c + c(x^j)$$

```

6:   Choose

```

$$F_j \in \partial\Pi_{K^*}(\cdot)(\sigma^j)$$

```

7:   Solve

```

$$(P + \rho I + C^\top M^{-1} F_j C) \Delta x^j = -\nabla\varphi_c(x^j)$$

```

8:   Compute a step length  $\alpha_j$  using the line-search rule

```

```

9:   Set

```

$$x^{j+1} = x^j + \alpha_j \Delta x^j$$

```

10: end for

```

8.5 Infeasibility certificates

Proposition 8.13 (Primal alternative). *Exactly one of the following statements holds:*

1.

$$-d \in \text{cl}(K + \text{range } C).$$

2. *There exists $z \in \mathbb{R}^m$ such that*

$$z \in K^*, \quad C^\top z = 0, \quad \langle z, d \rangle > 0.$$

If the primal problem is feasible, then the first statement holds. Consequently, the second statement certifies primal infeasibility.

Proof. Set

$$S = \text{cl}(K + \text{range } C).$$

Then S is a closed convex cone. We first observe that

$$S^* = \{z \in K^* : C^\top z = 0\}. \quad (8.33)$$

Indeed, if $z \in S^*$, then $K \subseteq S$ implies $z \in K^*$. Moreover, $\text{range } C \subseteq S$ is a subspace. Hence both Cx and $-Cx$ belong to S , so

$$\langle z, Cx \rangle = 0 \quad \text{for every } x.$$

Thus $C^\top z = 0$. The converse follows immediately from

$$\langle z, k + Cx \rangle = \langle z, k \rangle \geq 0$$

for $k \in K$.

The two alternatives cannot hold simultaneously. Indeed, if $-d \in S$ and $z \in S^*$, then

$$\langle z, -d \rangle \geq 0,$$

so

$$\langle z, d \rangle \leq 0.$$

Suppose now that $-d \notin S$. Suppose that for any $z \in S^*$ we have that $\langle z, -d \rangle \geq 0$ hence that $S^* \subset \{-d\}^*$ thus taking the dual given that S is closed and convex and dual inverse monotonicity:

$$\{-d\} \in S^{**} = S$$

Which is contradictory. Thus there exist z such that:

$$z \in S^*$$

and

$$\langle z, d \rangle > 0.$$

Using (8.33), we obtain

$$z \in K^*, \quad C^\top z = 0.$$

Finally, if x is primal feasible, then

$$-(Cx + d) \in K,$$

and therefore

$$-d = -(Cx + d) + Cx \in K + \text{range } C \subseteq S.$$

□

Proposition 8.14 (Improving ray). *Suppose that there exists $\delta \in \mathbb{R}^n$ satisfying*

$$P\delta = 0, \quad -C\delta \in K, \quad \langle q, \delta \rangle < 0. \quad (8.34)$$

Then the dual problem is infeasible. If the primal problem is feasible, then

$$p^* = -\infty.$$

Proof. Suppose, for contradiction, that $z \in K^*$ is dual feasible. Then there exists x_z satisfying

$$Px_z + q + C^\top z = 0.$$

Taking the inner product with δ , and using $P\delta = 0$, gives

$$0 = \langle Px_z + q + C^\top z, \delta \rangle \quad (8.35)$$

$$= \langle q, \delta \rangle + \langle z, C\delta \rangle \quad (8.36)$$

$$= \langle q, \delta \rangle - \langle z, -C\delta \rangle. \quad (8.37)$$

Since

$$z \in K^* \quad \text{and} \quad -C\delta \in K,$$

we have

$$\langle z, -C\delta \rangle \geq 0.$$

It follows that

$$0 \leq \langle q, \delta \rangle,$$

contradicting $\langle q, \delta \rangle < 0$. Thus the dual problem is infeasible.

Now suppose that x is primal feasible. For every $t \geq 0$,

$$-(C(x + t\delta) + d) = -(Cx + d) + t(-C\delta) \quad (8.38)$$

$$\in K, \quad (8.39)$$

because K is a convex cone. Hence $x + t\delta$ remains feasible.

Since $P\delta = 0$ and P is symmetric,

$$f(x + t\delta) = f(x) + t \langle q, \delta \rangle.$$

Because $\langle q, \delta \rangle < 0$,

$$f(x + t\delta) \longrightarrow -\infty \quad \text{as } t \rightarrow +\infty.$$

Therefore,

$$p^* = -\infty.$$

□

Definition 8.15 (Infeasibility certificates). A *primal infeasibility certificate* is a vector $z \in \mathbb{R}^m$ satisfying

$$z \in K^*, \quad C^\top z = 0, \quad \langle z, d \rangle > 0. \quad (8.40)$$

A *dual infeasibility certificate* is a vector $\delta \in \mathbb{R}^n$ satisfying

$$P\delta = 0, \quad -C\delta \in K, \quad \langle q, \delta \rangle < 0. \quad (8.41)$$

They are the certificates used in the algorithm for testing infeasibility.

8.6 Asymptotic displacement and certificates

Proposition 8.16 (Asymptotic displacement, [15]). Let T be maximal monotone, let $\mathcal{M} \succ 0$, and define

$$v^{k+1} = J_T^{\mathcal{M}}(v^k).$$

Then the displacement sequence

$$v^{k+1} - v^k$$

converges to a limit δ that is independent of the initial point v^0 . If $T^{-1}(0) \neq \emptyset$, then

$$\delta = 0.$$

Proposition 8.17 (Decomposition of the asymptotic displacement). Let

$$\mathcal{M} = \text{diag}(\rho I_n, M), \quad \rho > 0, \quad M \succ 0,$$

and let T be the KKT operator. Write

$$\delta = (\delta_x, \delta_z)$$

for the asymptotic displacement of the proximal iteration. Then

$$P\delta_x = 0, \quad -C\delta_x \in K, \quad (8.42)$$

$$C^\top \delta_z = 0, \quad \delta_z \in K^*, \quad (8.43)$$

$$\langle C\delta_x, \delta_z \rangle = 0, \quad (8.44)$$

$$\langle q, \delta_x \rangle = -\rho \|\delta_x\|^2, \quad (8.45)$$

$$\langle d, \delta_z \rangle = \|\delta_z\|_M^2. \quad (8.46)$$

Consequently:

1. If $\delta_x \neq 0$, then δ_x is a dual infeasibility certificate.
2. If $\delta_z \neq 0$, then δ_z is a primal infeasibility certificate.
3. If $\delta \neq 0$, then at least one infeasibility certificate is obtained.

Remark 8.18. The theorem is only expressed here for fixed metric but we use it for variable metric.

Remark 8.19 (Vanishing displacement does not imply solvability). The preceding proposition gives certificates only when the asymptotic displacement is nonzero. Although

$$T^{-1}(0) \neq \emptyset \implies \delta = 0,$$

the converse is false in general. It is possible to have

$$\delta = 0 \quad \text{and} \quad T^{-1}(0) = \emptyset.$$

This situation may occur for weakly infeasible problems or for problems that are unbounded along a curved feasible path but admit no improving recession direction. In the algorithm we use the implication:

$$\delta \neq 0 \implies T^{-1}(0) = \emptyset$$

To detect infeasibility.

Part II

Method construction

Chapter 9

Getting a Newton

9.1 First systems

This section derives the Newton system of the solver's semismooth method. We first recall the optimality conditions of the problem.

Proposition 9.1 (Optimality conditions of one proximal step). *Let $\rho > 0$, $M \in \mathcal{A}(K)$, $\mathcal{M} = \text{diag}(\rho I_n, M)$ and $v_c = (x_c, z_c)$. Then $v = (x, z) = J_T^{\mathcal{M}}(v_c)$ if and only if*

$$r_{\text{stat}} = Px + q + \rho(x - x_c) + C^\top z = 0, \quad (9.1)$$

$$r_{\text{cone}} = Mz - \Pi_{K^*}(\sigma) = 0, \quad \sigma = Mz_c + c(x). \quad (9.2)$$

$$Px + q + \rho(x - x_c) + C^\top M^{-1} \Pi_{K^*}(Mz_c + c(x)) = 0. \quad (9.3)$$

Proposition 9.2 (The system obtained directly from the two residuals). *Let $F \in \partial \Pi_{K^*}(\cdot)(\sigma)$. The semismooth Newton step for $(r_{\text{stat}}, r_{\text{cone}})$ solves*

$$\begin{pmatrix} P + \rho I & C^\top \\ -FC & M \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta z \end{pmatrix} = - \begin{pmatrix} r_{\text{stat}} \\ r_{\text{cone}} \end{pmatrix}, \quad (9.4)$$

Proof. Differentiating gives

$$\partial r_{\text{stat}} / \partial x = P + \rho I$$

and

$$\partial r_{\text{stat}} / \partial z = C^\top$$

; for the second residual

$$\partial \sigma / \partial x = C$$

, so

$$\partial r_{\text{cone}} / \partial x = -FC$$

and

$$\partial r_{\text{cone}} / \partial z = M$$

. The diagonal blocks are symmetric. □

The trouble with the preceding matrix is that it is not symmetric, and that its sparsity pattern changes at every step because of the product FC . The first of these is repaired by a residual carrying F in its own definition.

Definition 9.3 (Symmetrised stationarity residual). With $\sigma = Mz_c + c(x)$ and $F \in \partial\Pi_{K^*}(\cdot)(\sigma)$,

$$r_{\text{stat}}^* = r_{\text{stat}} + C^\top(F - I)z = Px + q + \rho(x - x_c) + C^\top Fz. \quad (9.5)$$

Let recall the following lemma that we proved before:

Lemma 9.4 (F fixes the projection). *Let $\sigma \in \mathbb{R}^m$, $p = \Pi_{K^*}(\sigma)$ and $F \in \partial\Pi_{K^*}(\cdot)(\sigma)$. Then $Fp = p$.*

Proposition 9.5 (The two stationarity residuals agree at a cone zero). *If $r_{\text{cone}} = 0$ then $Fz = z$, and consequently $r_{\text{stat}}^* = r_{\text{stat}}$.*

Proof. $r_{\text{cone}} = 0$ reads $Mz = p$. Since $M \in \mathcal{A}(K)$ commutes with F , so does M^{-1} , and the lemma gives $Fz = FM^{-1}p = M^{-1}Fp = M^{-1}p = z$. Then $r_{\text{stat}}^* - r_{\text{stat}} = C^\top(F - I)z = 0$. $\square$

Proposition 9.6 (The symmetrised system). *Let $V = C^\top D^2\Pi_{K^*}(\cdot)(\sigma)[\cdot, \cdot, z]C$. The semismooth Newton step for $(r_{\text{stat}}^*, r_{\text{cone}})$, with the second block row negated, solves*

$$\begin{pmatrix} P + \rho I + V & C^\top F \\ FC & -M \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta z \end{pmatrix} = \begin{pmatrix} -r_{\text{stat}}^* \\ r_{\text{cone}} \end{pmatrix}, \quad (9.6)$$

and this matrix is symmetric.

Proof. The z derivative of r_{stat}^* is $C^\top F$, since σ does not depend on z . Its x derivative is

$$P + \rho I + C^\top(DF(\sigma)[C \cdot])z$$

, and $DF(\sigma) = D^2\Pi_{K^*}(\cdot)(\sigma)$ contracted against z in its last argument gives V . The second row is unchanged from the previous proposition up to the negation. Symmetry: $C^\top F$ and FC are transposes since $F = F^\top$, M is symmetric, and V is symmetric because a second derivative is symmetric in its first two arguments. $\square$

Remark 9.7. Because r_{stat}^* contains a term involving a generalized Jacobian F of the projection, it is not necessarily semismooth for an arbitrary closed convex cone K . To justify a classical Newton linearisation at this stage, we therefore assume that Π_K is twice differentiable at the chosen point σ . This assumption is restrictive, but it will be removed later. Indeed, the resulting linear system will be shown to be equivalent to a semismooth Newton step applied to $\nabla\phi_c$, which provides a rigorous interpretation even when Π_K is not twice differentiable.

This system still has a sparsity pattern that changes; it carries a product $C^\top C$, which squares the conditioning; V is hard to compute for most curved cones, though it vanishes for the orthant; and no merit function is attached to it.

9.2 A family of merit functions

Definition 9.8 (Merit family). Write $w = (x, z)$, $R(w) = (r_{\text{stat}}^*, r_{\text{cone}})$ and $J = \partial R$. A function $\mathcal{N}$ belongs to the *family* if

$$\nabla\mathcal{N}(w) = U(w)R(w) \quad (9.7)$$

for some $U(w)$ invertible at every w .

Proposition 9.9 (Properties of the family). *Let $\mathcal{N}$ belong to the family. Then $\nabla \mathcal{N}(w) = 0$ if and only if $R(w) = 0$ and $\nabla^2 \mathcal{N} = UJ + \mathcal{E}$ with $\mathcal{E} = (\partial U)[\cdot, R]$, so that Newton's method on $\mathcal{N}$ solves*

$$(UJ + \mathcal{E})\Delta w = -UR, \quad \text{equivalently} \quad (J + U^{-1}\mathcal{E})\Delta w = -R. \quad (9.8)$$

Proof. The first is immediate from invertibility of U . For the second, differentiating $\nabla \mathcal{N} = UR$ by the product rule gives

$$\nabla^2 \mathcal{N} = U \partial R + (\partial U)[\cdot, R]$$

, and Newton's method on $\mathcal{N}$ is $\nabla^2 \mathcal{N} \Delta w = -\nabla \mathcal{N}$, which is the displayed system; multiplying by U^{-1} gives the second form. $\square$

We use the term merit family here not in the strict sense given in the Newton method chapter, even though we just saw that element of the family have gradient that are zero at optimality point only. The one giving the symmetrised is such an example:

Definition 9.10 (Proximal merit). With $\sigma = Mz_c + c(x)$,

$$\mathcal{N}_L(x, z) = f(x) + \frac{\rho}{2} \|x - x_c\|^2 + \langle z, \Pi_{K^*}(\sigma) \rangle - \frac{1}{2} \|z\|_M^2. \quad (9.9)$$

Proposition 9.11 (The proximal merit is a saddle function of the family). *Let $E = \text{diag}(I_n, -I_m)$. Then*

$$\nabla \mathcal{N}_L = E R$$

and $\nabla^2 \mathcal{N}_L$ is the symmetrised matrix of the preceding section. In particular $\mathcal{N}_L$ belongs to the family with $U = E$, and its second block diagonal is $-M \prec 0$, so it is not bounded below.

Proof. Differentiating in z gives $\Pi_{K^*}(\sigma) - Mz = -r_{\text{cone}}$. Differentiating in x gives $Px + q + \rho(x - x_c) + C^\top F^\top z$, and $F = F^\top$, so this is r_{stat}^* . Hence $\nabla \mathcal{N}_L = (r_{\text{stat}}^*, -r_{\text{cone}}) = ER$. Differentiating once more reproduces the blocks of the symmetrised system, the $(1, 1)$ entry acquiring V from the derivative of F contracted against z . $\square$

Definition 9.12 (The proximal second order block).

$$V_{\text{prox}} = C^\top D^2 \Pi_{K^*}(\cdot)(\sigma)[\cdot, \cdot, z] C. \quad (9.10)$$

An actual merit can be found in the family i.e for which the optimum is a minimum.

Definition 9.13 (Gill and Robinson merit). For $Q \in \mathcal{A}(K)$,

$$\mathcal{N}_c(x, z) = \varphi_c(x) + \frac{1}{2} \|r_{\text{cone}}\|_{Q^{-1}}^2. \quad (9.11)$$

Proposition 9.14 (The weight is forced). $\nabla_z \mathcal{N}_c = MQ^{-1}r_{\text{cone}}$, so $\nabla_z \mathcal{N}_c = r_{\text{cone}}$ for every z if and only if $Q = M$.

Proof. φ_c does not depend on z , and

$$r_{\text{cone}} = Mz - \Pi_{K^*}(\sigma)$$

has z derivative M , so

$$\nabla_z \frac{1}{2} \|r_{\text{cone}}\|_{Q^{-1}}^2 = M^\top Q^{-1} r_{\text{cone}} = MQ^{-1} r_{\text{cone}}$$

. This equals r_{cone} for all z exactly when $MQ^{-1} = I$. $\square$

Proposition 9.15 (The Gill and Robinson merit belongs to the family). *With $Q = M$, :*

$$\nabla \mathcal{N}_c = (r_{\text{stat}}^* - 2C^\top M^{-1} F r_{\text{cone}}, r_{\text{cone}}) = U R, \quad U = \begin{pmatrix} I & -2C^\top M^{-1} F \\ 0 & I \end{pmatrix}, \quad (9.12)$$

which is block upper triangular and invertible.

Proposition 9.16 (The Gill and Robinson system). *Using (9.8) and scaling the second block row gives the symmetrised matrix with V replaced by V_{GR} and the same right hand side structure,*

$$\begin{pmatrix} P + \rho I + V_{\text{GR}} & C^\top F \\ FC & -M \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta z \end{pmatrix} = \begin{pmatrix} -r_{\text{stat}}^* \\ r_{\text{cone}} \end{pmatrix}. \quad (9.13)$$

The right hand side is the one of the symmetrised system, unchanged.

Definition 9.17 (The Gill and Robinson second order block).

$$V_{\text{GR}} = C^\top D^2 \Pi_{K^*}(\cdot)(\sigma)[\cdot, \cdot, M^{-1}(p - r_{\text{cone}})] C, \quad p = \Pi_{K^*}(\sigma). \quad (9.14)$$

The Gill and Robinson system still suffers from the changing sparsity pattern and from the cost of computing V . Let recall the following proposition points, that will point us in the right direction.

Proposition 9.18 (The second derivative contracted at $M^{-1}p$). *At a σ where $\Pi_{K^*}(\cdot)$ is twice differentiable, and with $M \in \mathcal{A}(K)$,*

$$D^2 \Pi_{K^*}(\cdot)(\sigma)[\cdot, \cdot, M^{-1}p] = M^{-1}F(I - F) \succeq 0. \quad (9.15)$$

This naturally suggests the following definition.

Definition 9.19 (The block at the projection).

$$V_p = C^\top M^{-1}F(I - F)C \succeq 0. \quad (9.16)$$

$$\begin{pmatrix} P + \rho I + C^\top M^{-1}F(I - F)C & C^\top F \\ FC & -M \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta z \end{pmatrix} = \begin{pmatrix} -r_{\text{stat}}^* \\ r_{\text{cone}} \end{pmatrix}. \quad (9.17)$$

This system is not in the family, so the demand on U has to be relaxed. Nothing is lost by doing so, because the system is in fact the Newton system of φ_c taken as merit.

Proposition 9.20 (The system with V_p is the Newton system of the primal function). *Eliminating Δz from the system above leaves*

$$\nabla^2 \varphi_c(x) \Delta x = -\nabla \varphi_c(x), \quad (9.18)$$

so that system is a representation of the primal Newton step in which Δz is obtained from the same factorisation, and its Δx inherits $\langle \nabla \varphi_c, \Delta x \rangle \leq -\rho \|\Delta x\|^2$.

Proof. The second row gives $\Delta z = M^{-1}(FC\Delta x - r_{\text{cone}})$. Substituting into the first leaves

$$(P + \rho I + C^\top M^{-1}F(I - F)C + C^\top FM^{-1}FC)\Delta x = -r_{\text{stat}}^* + C^\top FM^{-1}r_{\text{cone}}.$$

On the left by admissibility of M :

$$C^\top FM^{-1}FC = C^\top M^{-1}F^2C$$

, which added to

$$C^\top M^{-1}F(I - F)C$$

gives

$$C^\top M^{-1}FC$$

, so the matrix is $\nabla^2\varphi_c(x)$. On the right $FM^{-1} = M^{-1}F$ and $r_{\text{stat}}^* = r_{\text{stat}} + C^\top(F - I)z$ combine with $r_{\text{cone}} = Mz - p$ and $Fp = p$ to give

$$-r_{\text{stat}} - C^\top(F - I)z + C^\top M^{-1}F(Mz - p) = -r_{\text{stat}} + C^\top(z - M^{-1}p) - C^\top z = -\nabla\varphi_c(x)$$

. The descent bound follows from $\langle \nabla\varphi_c, \Delta x \rangle = -\langle \nabla^2\varphi_c \Delta x, \Delta x \rangle$ and $\nabla^2\varphi_c \succeq \rho I$. $\square$

Two problems remain: the sparsity pattern and the product.

9.3 Changing the representation

Most general conic methods use the following form.

Definition 9.21 (Split form). The *split form* of the problem is

$$\min_{x \in \mathbb{R}^n, s \in \mathbb{R}^m} f(x) \quad \text{subject to} \quad Ex + s = b, \quad s \in K. \quad (9.19)$$

It is immediate that :

Proposition 9.22 (The two forms are the same problem). *With $E = C$ and $b = -d$, the map $x \mapsto (x, -c(x))$ is a bijection from the feasible set of the original problem onto the feasible set of the split form which preserves the objective. Consequently the two problems have the same optimal value, and x is optimal for one if and only if $(x, -c(x))$ is optimal for the other.*

Let introduce some notation:

Definition 9.23 (Residual splitting). Let $\rho > 0$, $\rho_p > 0$, $M_s \succ 0$ and $M \in \mathcal{A}(K)$, with centres (x_c, s_c, y_c, z_c) . With $\sigma = Mz_c - s$, $p = \Pi_{K^*}(\sigma)$ and $F \in \partial\Pi_{K^*}(\cdot)(\sigma)$, set

$$r_{\text{stat}}^x := Px + q + \rho(x - x_c) + E^\top y, \quad (9.20)$$

$$r_{\text{stat}}^{*s} := \rho_p(s - s_c) + y - Fz, \quad (9.21)$$

$$r_{\text{eq}} := M_s(y - y_c) - (Ex + s - b), \quad (9.22)$$

$$r_{\text{cone}} := Mz - p. \quad (9.23)$$

$$\varrho_s := r_{\text{stat}}^{*s} + M^{-1}Fr_{\text{cone}}. \quad (9.24)$$

What this form buys is the system below, which settles both the sparsity pattern and the product.

$$\begin{pmatrix} P + \rho I & 0 & E^\top & 0 \\ 0 & \rho_p I + M^{-1}F(I - F) & I & -F \\ E & I & -M_s & 0 \\ 0 & -F & 0 & -M \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta s \\ \Delta y \\ \Delta z \end{pmatrix} = \begin{pmatrix} -r_{\text{stat}}^x \\ -r_{\text{stat}}^{*s} \\ r_{\text{eq}} \\ r_{\text{cone}} \end{pmatrix}. \quad (9.25)$$

Definition 9.24 (Cone operator).

$$G = \rho_p I + M^{-1}F \succ 0. \quad (9.26)$$

In general the three by three version, of size $n + 2m$, is the one solved.

Proposition 9.25 (Reduction to the three by three system). *Eliminating Δz from (10.6) through $\Delta z = -M^{-1}(F\Delta s + r_{\text{cone}})$ leaves*

$$\begin{pmatrix} P + \rho I & 0 & E^\top \\ 0 & G & I \\ E & I & -M_s \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta s \\ \Delta y \end{pmatrix} = \begin{pmatrix} -r_{\text{stat}}^x \\ -\varrho_s \\ r_{\text{eq}} \end{pmatrix}, \quad (9.27)$$

which is symmetric quasi-definite.

Proposition 9.26 (Reduction to the two by two system). *Eliminating Δs from (9.27) through $\Delta s = -G^{-1}(\varrho_s + \Delta \nu)$, leaves*

$$\begin{pmatrix} P + \rho I & E^\top \\ E & -(M_s + G^{-1}) \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} = \begin{pmatrix} -r_{\text{stat}}^x \\ r_{\text{eq}} + G^{-1}\varrho_s \end{pmatrix}, \quad (9.28)$$

again symmetric quasi-definite.

The two by two version is not implemented, since it requires inverting G and which is ill conditioned.

The Schur complement, of size n , is used as well. Written as $\Sigma = G(I + M_s G)^{-1}$ it needs no inverse of G anywhere, which is what makes it usable: $G = \rho_p I + M^{-1}F$ has an eigenvalue exactly ρ_p wherever the Clarke Jacobian F has a zero one, so $\kappa(G)$ is of order $1/(\rho_p \mu)$, and both parameters are driven to their floors. Any route through G^{-1} therefore carries a condition number the working precision cannot hold, while $I + M_s G$ has condition number of order $1 + \|G\| \|M_s\|$. The reduction involves a product, which increases the fill in the sparse case; with G^{-1} we are able to make it reach the same accuracy as the 3x3 using Ruiz (we will talk about it later) to minimize the conditioning of product appearing in it.

Proposition 9.27 (Schur complement). *Let $\Sigma = (M_s + G^{-1})^{-1} = (I + GM_s)^{-1}G = G(I + M_s G)^{-1}$ and*

$$S = P + \rho I + E^\top \Sigma E \succ 0. \quad (9.29)$$

Then Δx solves $S\Delta x = -r_{\text{stat}}^x + E^\top \Sigma(r_{\text{eq}} + G^{-1}\varrho_s)$, after which $\Delta y = \Sigma(E\Delta x - r_{\text{eq}} - G^{-1}\varrho_s)$, $\Delta s = -G^{-1}(\varrho_s + \Delta y)$ and $\Delta z = -M^{-1}(F\Delta s + r_{\text{cone}})$.

We never invert G . Writing $t = r_{\text{eq}} - M_s \varrho_s$ and using

$$(I + M_s G)^{-1} = I - M_s \Sigma, \quad (I + GM_s)^{-1} = I - \Sigma M_s = \Sigma G^{-1}, \quad (9.30)$$

the three steps become

$$S\Delta x = -r_{\text{stat}}^x + E^\top (\Sigma t + \varrho_s), \quad \Delta y = \Sigma e - \varrho_s, \quad \Delta s = M_s \Sigma e - e, \quad (9.31)$$

with $e = E\Delta x - t$, in which G appears only through Σ . Applying KKT conditions we get:

Proposition 9.28 (Optimality conditions in the split variables). *(x, s, z) is optimal for the split form together with a multiplier if and only if*

$$Px + q + E^\top z = 0, \quad Ex + s = b, \quad s \in K, \quad z \in K^*, \quad \langle s, z \rangle = 0. \quad (9.32)$$

Moreover the multiplier y of the equality constraint equals z at such a point.

Proof that $y = z$. The Lagrangian of the split form is

$$L(x, s, y, z) = f(x) + \langle y, Ex + s - b \rangle - \langle z, s \rangle, \quad z \in K^*, \quad (9.33)$$

the term $-\langle z, s \rangle$ carrying the constraint $s \in K$. Stationarity in s reads $\nabla_s L = y - z = 0$, since s enters only through $\langle y, s \rangle$ and $-\langle z, s \rangle$. Hence $y = z$, and substituting into $\nabla_x L = Px + q + E^\top y = 0$ gives the first displayed condition. $\square$

The following proposition is used as the duality gap convergence criteria:

Proposition 9.29 (The gap is what the constraints leave). *Let (x, s) satisfy $Ex + s = b$ with $s \in K$, and let $z \in K^*$ satisfy $Px + q + E^\top z = 0$. Then*

$$\langle s, z \rangle = \langle x, Px \rangle + \langle q, x \rangle + \langle b, z \rangle \geq 0, \quad (9.34)$$

and it vanishes if and only if (x, s, z) is optimal.

Proof. $\langle s, z \rangle = \langle b - Ex, z \rangle = \langle b, z \rangle - \langle x, E^\top z \rangle = \langle b, z \rangle + \langle x, Px + q \rangle$, which is the displayed identity. It is nonnegative because $s \in K$ and $z \in K^*$. If it vanishes, the five conditions of the preceding proposition all hold, so the point is optimal; if it is optimal, complementarity gives $\langle s, z \rangle = 0$. $\square$

Definition 9.30 (Termination test). Given (x, y, z) , let $\hat{s} = b - Ex$ be the slack recomputed from the data, and let

$$\begin{aligned} \pi &= \|\Pi_K(-\hat{s})\|_\infty, & \pi_{\text{scale}} &= \max\{\|Ex\|_\infty, \|\hat{s}\|_\infty, \|b\|_\infty\}, \\ \delta &= \|Px + q + E^\top z\|_\infty, & \delta_{\text{scale}} &= \max\{\|Px\|_\infty, \|q\|_\infty, \|E^\top z\|_\infty\}, \\ \gamma &= \|z - \Pi_{K^*}(z)\|_\infty, & \gamma_{\text{scale}} &= \|z\|_\infty, \\ \eta &= |\langle \hat{s}, z \rangle|, & \eta_{\text{scale}} &= \max\{|p^*|, |d^*|\}, \end{aligned} \quad (9.35)$$

with $p^* = \frac{1}{2} \langle x, Px \rangle + \langle q, x \rangle$ and $d^* = -\frac{1}{2} \langle x, Px \rangle - \langle b, z \rangle$. The test passes when

$$\pi \leq \varepsilon_{\text{abs}} + \varepsilon_{\text{rel}} \pi_{\text{scale}}, \quad \delta \leq \varepsilon_{\text{abs}} + \varepsilon_{\text{rel}} \delta_{\text{scale}}, \quad \gamma \leq \varepsilon_{\text{abs}} + \varepsilon_{\text{rel}} \gamma_{\text{scale}}, \quad \eta \leq \varepsilon_{\text{abs}} + \varepsilon_{\text{rel}} \eta_{\text{scale}} \quad (9.36)$$

all hold. The slack is recomputed so that $Ex + \hat{s} = b$ holds exactly and only the four displayed quantities can fail. The fourth is the duality gap, since $p^* - d^* = \langle \hat{s}, z \rangle$ by Proposition 9.29.

Algorithm 9.1 Proximal method of multipliers in the split variables

Input: data (P, q, E, b, K) , weights $\rho, \rho_p > 0$, $M_s \succ 0$, $M \in \mathcal{A}(K)$, a penalty update rule, tolerances $\varepsilon_{\text{abs}}, \varepsilon_{\text{rel}}, \varepsilon_{\text{inf}}$ and inner tolerances (ε_k)

Output: (x, s, y, z) passing the test of Definition 9.30, or an infeasibility certificate

- 1: $(x_c, s_c, y_c, z_c) \leftarrow$ a starting point, and $(x, s, y, z) \leftarrow (x_c, s_c, y_c, z_c)$
- 2: **for** $k = 0, 1, 2, \dots$ **do**
- 3: $(x^-, z^-) \leftarrow (x, z)$ $\triangleright$ kept for the displacement
- 4: **while** $\|(r_{\text{stat}}^x, r_{\text{stat}}^s, r_{\text{eq}}, r_{\text{cone}})\|_\infty > \min_i (\varepsilon_k)_i$ **do** $\triangleright \varepsilon_k$ is per row; the loop stops against its smallest entry
- 5: $\sigma \leftarrow Mz_c - s$, $p \leftarrow \Pi_{K^*}(\sigma)$, choose $F \in \partial \Pi_{K^*}(\cdot)(\sigma)$ and form $G = \rho_p I + M^{-1}F$
- 6: solve (9.27), or its reduction (9.28) or (9.29), for $(\Delta x, \Delta s, \Delta y)$
- 7: $\Delta z \leftarrow -M^{-1}(F\Delta s + r_{\text{cone}})$
- 8: obtain α from the line search rule applied to φ_c $\triangleright$ the subproblem's objective, not $\mathcal{N}_c$
- 9: $(x, s, y, z) \leftarrow (x, s, y, z) + \alpha(\Delta x, \Delta s, \Delta y, \Delta z)$
- 10: **end while**
- 11: **if** the test of Definition 9.30 passes **then**
- 12: **return** $(x, b - Ex, y, z)$
- 13: **end if**
- 14: $\delta x \leftarrow x - x^-$, $\delta z \leftarrow z - z^-$ $\triangleright$ the displacement over the outer iteration
- 15: **if** δz certifies primal infeasibility to ε_{inf} **then**
- 16: **return** PRIMALINFEASIBLE
- 17: **end if**
- 18: **if** δx certifies dual infeasibility to ε_{inf} **then**
- 19: **return** DUALINFEASIBLE
- 20: **end if**
- 21: apply the penalty update rule, which moves the centres (x_c, s_c, y_c, z_c) , the weights ρ_p, M_s, M and ε_{k+1}
- 22: **end for**

Chapter 10

Smoothing point of view and interior point version

10.1 The proximal step as a smoothing of the slack

$$\min_{x \in \mathbb{R}^n, s \in \mathbb{R}^m} \varphi_c(x, s). \quad (10.1)$$

Proposition 10.1 (The hard constraint on s problem). *Let*

$$h(x, s) = f(x) + \frac{\rho}{2} \|x - x_c\|^2 + \frac{\rho_p}{2} \|s - s_c\|^2 + \langle y_c, \ell \rangle + \frac{1}{2} \|\ell\|_{M_s^{-1}}^2, \quad \ell = Ex + s - b. \quad (10.2)$$

The problem with hard constraint on s is:

$$\min_{x, s \in K} h(x, s)$$

10.2 General theory for a general smoothing

Definition 10.2 (Smoothed proximal problem). Let (ϕ_θ) be a smoothing of δ_K , for a fixed θ use $\phi = \phi_\theta$ then the *smoothed proximal problem* is

$$\min_{x \in \mathbb{R}^n, s \in D^\theta} h(x, s) + \phi(s). \quad (10.3)$$

Definition 10.3 (Smoothing stationarity residual). Introducing the multipliers y and z , define

$$\begin{aligned} r_{\text{stat}}^x &:= Px + q + \rho(x - x_c) + E^\top y, \\ r_\phi^s &:= \rho_p(s - s_c) + y - z, \\ r_\phi &:= z + \nabla \phi(s), \\ r_{\text{eq}} &:= M_s(y - y_c) - (Ex + s - b). \end{aligned}$$

Proposition 10.4 (Stationary condition of the smoothed problem). *A pair (x, s) solves the smoothed problem if and only if there exist multipliers y and z such that*

$$r_{\text{stat}}^x = 0, \quad r_\phi^s = 0, \quad r_\phi = 0, \quad r_{\text{eq}} = 0.$$

Proof. Consequence of the general smoothing stationary conditions. □

Proposition 10.5 (Newton on the proximal smoothed problem). *Let $H \in \partial(\nabla\phi)(s)$. Then the Newton system is*

$$\begin{pmatrix} P + \rho I & 0 & E^\top & 0 \\ 0 & \rho_p I & I & -I \\ -E & -I & M_s & 0 \\ 0 & H & 0 & I \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta s \\ \Delta y \\ \Delta z \end{pmatrix} = - \begin{pmatrix} r_{\text{stat}}^x \\ r_\phi^s \\ r_{\text{eq}} \\ r_\phi \end{pmatrix}, \quad (10.4)$$

equivalently, negating the third block row,

$$\begin{pmatrix} P + \rho I & 0 & E^\top & 0 \\ 0 & \rho_p I & I & -I \\ E & I & -M_s & 0 \\ 0 & H & 0 & I \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta s \\ \Delta y \\ \Delta z \end{pmatrix} = - \begin{pmatrix} r_{\text{stat}}^x \\ r_\phi^s \\ -r_{\text{eq}} \\ r_\phi \end{pmatrix}. \quad (10.5)$$

Definition 10.6 (Perturbed residual). Let Υ satisfy $\Upsilon(s, z) = 0$ at the optimum of the smoothed problem. The perturbed residual is

$$r_\phi^{\Upsilon, s} = r_\phi^s + \Upsilon(s, z).$$

Definition 10.7 (Smoothed system with perturbed right hand side).

$$\begin{pmatrix} P + \rho I & 0 & E^\top & 0 \\ 0 & \rho_p I & I & -I \\ E & I & -M_s & 0 \\ 0 & H & 0 & I \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta s \\ \Delta y \\ \Delta z \end{pmatrix} = - \begin{pmatrix} r_{\text{stat}}^x \\ r_\phi^s \\ -r_{\text{eq}} \\ r_\phi^\Upsilon \end{pmatrix}. \quad (10.6)$$

Definition 10.8 (Cone operator and reduced residual). For

$$H \in \partial(\nabla\phi)(s),$$

define

$$G_\phi := \rho_p I + H, \quad (10.7)$$

and

$$\varrho_s^\Upsilon := r_\phi^{\Upsilon, s} + r_\phi. \quad (10.8)$$

Proposition 10.9 (3x3 system of proximal smoothing subproblem). *Eliminate Δz in the preceding system you get:*

$$\begin{pmatrix} P + \rho I & 0 & E^\top \\ 0 & G_\phi & I \\ E & I & -M_s \end{pmatrix} \begin{pmatrix} \Delta x \\ \Delta s \\ \Delta y \end{pmatrix} = - \begin{pmatrix} r_{\text{stat}}^x \\ \varrho_s^\Upsilon \\ -r_{\text{eq}} \end{pmatrix}. \quad (10.9)$$

For the augmented Lagrangian smoothing we have:

$$G_{\phi^{\text{AL}}} = \rho_p I + M^{-1}F. \quad (10.10)$$

Proposition 10.10 (Proposition (Our problem is this problem)). $\varphi_c = h + \phi_{(M, z_c)}^{\text{AL}}$, that is the proximal subproblem is exactly the augmented Lagrangian smoothing, with parameter $\theta = (M, z_c)$, of the constrained problem

$$\min_{x, s} h(x, s) \quad \text{subject to} \quad s \in K. \quad (10.11)$$

And using $\Upsilon(s, z) = z - Fz$.

A natural question is what if we use an interior smoothing? This lead to the following:

Definition 10.11 (Proximal interior smoothing problem).

$$\nabla\phi_\tau(s) = \tau\nabla B(s), \quad \partial(\nabla\phi_\tau)(s) = \{\tau\nabla^2 B(s)\}, \quad (10.12)$$

$$G_\phi = \rho_p I + \tau\nabla^2 B(s), \quad \Upsilon = 0, \quad (10.13)$$

$$r_\phi = z + \tau\nabla B(s), \quad r_\phi^{\Upsilon,s} = r_\phi^s, \quad (10.14)$$

$$\varrho_s^\Upsilon = \rho_p(s - s_c) + y + \tau\nabla B(s). \quad (10.15)$$

Which give the following algorithm:

Algorithm 10.1 Proximal interior point method in the split variables

Input: data (P, q, E, b, K) , weights $\rho, \rho_p > 0$, $M_s \succ 0$, a barrier B of degree ν , centring bounds $0 < \sigma_{\min} \leq \sigma_{\max} < 1$, a penalty update rule, tolerances $\varepsilon_{\text{abs}}, \varepsilon_{\text{rel}}, \varepsilon_{\text{inf}}$

Output: (x, s, y, z) passing the test of Definition 9.30, or an infeasibility certificate

```

1:  $(x_c, s_c, y_c, z_c) \leftarrow$  a starting point with  $s \in \text{int } K$  and  $z \in \text{int } K^*$ , and  $(x, s, y, z) \leftarrow$ 
    $(x_c, s_c, y_c, z_c)$   $\triangleright$  the smoothing is interior, so the start must be
2: for  $k = 0, 1, 2, \dots$  do
3:    $(x^-, z^-) \leftarrow (x, z)$   $\triangleright$  kept for the displacement
4:    $\tau \leftarrow \langle s, z \rangle / \nu$   $\triangleright$  Euler's identity read backwards; one  $\tau$  for every block
5:   form  $H \leftarrow \tau\nabla^2 B(s)$  and  $G_\phi \leftarrow \rho_p I + H$ 
6:   solve (10.6) with target 0 for  $(\Delta x^a, \Delta s^a, \Delta y^a)$ , then  $\Delta z^a \leftarrow -(H\Delta s^a + r_\phi) \triangleright$  the
   affine predictor: no centring
7:    $(\alpha_p, \alpha_d) \leftarrow$  fraction to the boundary along  $(\Delta s^a, \Delta z^a)$ 
8:    $\tau_a \leftarrow \langle s + \alpha_p \Delta s^a, z + \alpha_d \Delta z^a \rangle / \nu$ 
9:    $\sigma \leftarrow \min\{\sigma_{\max}, \max\{\sigma_{\min}, (\tau_a/\tau)^3\}\}$   $\triangleright$  the affine step reports how much centring
   is needed
10:  solve (10.6) again with target  $\sigma\tau$  and the residual corrected by  $\Delta s^a \circ \Delta z^a$ , for
    $(\Delta x, \Delta s, \Delta y)$ , then  $\Delta z \leftarrow -(H\Delta s + r_\phi^\Upsilon) \triangleright$  same matrix, so one factorisation serves
   both
11:   $(\alpha_p, \alpha_d) \leftarrow$  fraction to the boundary along  $(\Delta s, \Delta z)$ 
12:   $(x, s) \leftarrow (x, s) + \alpha_p(\Delta x, \Delta s)$ ,  $(y, z) \leftarrow (y, z) + \alpha_d(\Delta y, \Delta z)$   $\triangleright$  one Newton step
   per outer iteration
13:  if the test of Definition 9.30 passes then
14:    return  $(x, b - Ex, y, z)$ 
15:  end if
16:   $\delta x \leftarrow x - x^-$ ,  $\delta z \leftarrow z - z^-$   $\triangleright$  the displacement over the outer iteration
17:  if  $\delta z$  certifies primal infeasibility to  $\varepsilon_{\text{inf}}$  then
18:    return PRIMALINFEASIBLE
19:  end if
20:  if  $\delta x$  certifies dual infeasibility to  $\varepsilon_{\text{inf}}$  then
21:    return DUALINFEASIBLE
22:  end if
23:  apply the penalty update rule, which moves the centres  $(x_c, s_c, y_c, z_c)$  and the
   weights  $\rho, \rho_p, M_s$ 
24: end for

```

Part III

Implementation detail

Chapter 11

The map

The theory part ends on a claim: the smoothing enters the Newton system only through the cone operator G_ϕ and the residuals, and everything downstream is common to both methods. The code is arranged so that this claim is not a remark but a compilable statement.

The solver is small enough to read in full, but raw file order hides its architecture. It is laid out in five layers. The cone kernel is a shared service used by smoothing, linear algebra, initialisation, globalisation and termination; it is not itself another stage of the iteration.

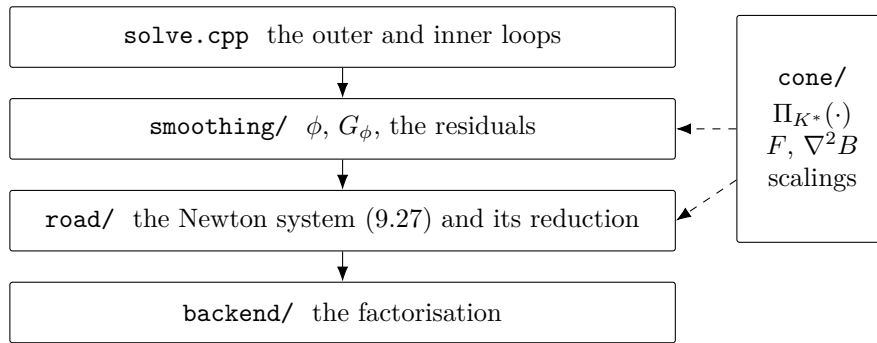


Figure 11.1: The five layers. Each solid arrow is a narrow interface: the layer above holds a pointer to an abstract type and never learns which implementation it has. The dashed arrows are the cone kernel, which every layer calls and which owns all six cones.

11.1 What is where

Every directory implements one object of the theory part, and the table names which.

directory	what it implements
<code>solve.cpp</code> and the root headers	the two algorithms and the iterate
<code>ruiz/</code>	equilibration and scaling maps
<code>cone/</code>	projections, linearisations, barriers and cone geometry
<code>smoothing/</code>	G_ϕ , residuals and recovery of Δz
<code>road/</code>	assembly and solution of (9.27) or (9.29)
<code>backend/</code>	symbolic and numeric factorisation
<code>globalisation/</code>	step selection and the merit φ_c
<code>schedule/</code>	centres, penalties, tolerances and regularisation
<code>initialisation/</code>	cold and warm starting points
<code>termination/</code>	Definition 9.30 and certificates
<code>bindings/</code>	the Python surface

Table 11.1: The implementation components. Counts are omitted deliberately: the contracts are stable while file lengths are not.

Only two of the three systems of the theory part are built. The two by two form (9.28) is never assembled, because eliminating Δs requires G^{-1} , and $G = \rho_p I + M^{-1}F$ has an eigenvalue exactly ρ_p wherever F has a zero one, so its conditioning grows as $1/(\rho_p \mu)$. The Schur road reaches (9.29) from (9.27) directly, through Σ , without ever passing through (9.28).

11.2 A reading order

The shortest path through the method is the following sequence.

1. `solve.cpp`, the outer and inner loops. Read it first and once, without following any call; it names every phase in order and nothing else in the repository has that view.
2. `smoothing/smoothing.hpp`, the interface. This is the claim of the theory part written as a type: eleven pure virtual functions, of which `blocks()` and `road_residuals()` are the only two the linear algebra ever calls.
3. `smoothing/projection.cpp`, the augmented Lagrangian smoothing.
4. `road/three_by_three.cpp`, the assembly and solve of (9.27), which is the default road. The pattern is built once and the values refilled every iteration; that split is the reason the road is fast and the reason a cone may not change the sparsity of its block between iterations.
5. `cone/nonneg.cpp`, the smallest complete cone. Every function a cone must provide appears here in its simplest form, so it is the template against which the other five are read.

After those, `smoothing/barrier.cpp` gives the interior method in the same shape as the third file.

Chapter 12

One solve, end to end

This chapter follows `solve.cpp` from the first line to the last. It is the only file with a view of the whole method, and the order of its phases is the order of this chapter. At each phase the object of the theory part being computed is named, so that the reader who has the theory can check the correspondence rather than take it on trust.

The point to carry through the chapter is that both algorithms of the theory part are this one loop.

12.1 Setup

The settings are split by *schedule*, not merely by road: `bcl`, `gbcl`, `semismooth` and `interior` each own a block, so that tuning one cannot move another. The four blocks that carry a road also inherit a shared part holding ρ , ρ_p , the equality penalty, the refinement budget and the refactorisation limits, which every road needs and none of them needs differently. The split is what makes `bcl` usable as a fixed reference while `gbcl` is tuned against it. The problem is then equilibrated.

12.2 The starting point

The iterate carries x, s, y, z , their four centres, the two penalty vectors and the scalars ρ and ρ_p . The penalties are set per row rather than globally, and a block with empty interior, that is a zero cone block, receives the equality penalty instead of the cone penalty, since there is no complementarity to relax there.

A warm start scales the given point into the working problem and sets $y \leftarrow z$. The two multipliers are distinct variables and are stepped separately, but they agree at the solution, by the stationarity in s , so a point inherited from a neighbouring problem has nothing better to offer for y . A cold start is delegated to the initialisation, which is where the two methods first diverge: the barrier requires $s \in \text{int } K$ and $z \in \text{int } K^*$ and reports this through `requires_interior()`, so it is given an interior point, while the projection smoothing is exterior and accepts any point at all.

12.3 The outer loop

The outer loop is the proximal iteration. Each pass performs the following, in order.

The numeric values contributed by P , E and ρ are refilled, but only if ρ has moved since they were last written. The pattern was built once at setup and never changes,

and since ρ is usually held fixed for many iterations the guard saves a full refill. The guard covers only these three: the cone operator and the equality penalty are written by `assemble`, which runs on every inner iteration without a guard, so a change in M , M_s or ρ_p is picked up immediately. The split is by rate of change, not by importance.

The inner loop then runs, and is described below. Afterwards the iterate is unscaled and graded by `termination.evaluate` against the original data, giving the four quantities of Definition 9.30.

The best iterate seen is tracked, ranked by the largest of those four. This is not cosmetic. On exit, if the current iterate is worse than the best recorded one, the best is restored and graded again, and a point that fails the test as the last iterate may pass it as the best.

The displacement over the outer iteration is then formed, $\delta x = x - x^-$ and $\delta z = z - z^-$, and each is offered to the certificate it can prove. The pairing is crossed: the dual displacement certifies primal infeasibility, and the primal displacement certifies dual infeasibility. Finally the schedule is given the violation per row, the reference value the smoothing supplies, the two residuals and whether the inner loop converged, and it moves the centres, the penalties and the tolerances.

12.4 How the metrics are represented

A natural question at this point is what M and M_s are in the code, since the theory allows any $M \in \mathcal{A}(K)$ and any $M_s \succ 0$. Both are penalties, and both are stored as vectors: `iterate.cone_penalty` is M and `iterate.equality_penalty` is M_s , one entry per row. They are therefore diagonal, and the products $M^{-1}v$ and $M_s v$ that appear throughout are elementwise.

Diagonal is not a restriction on M_s , which only has to be positive definite, and a diagonal choice is what makes (9.27) cheap. On M it is a restriction, and admissibility says exactly which one. A diagonal metric commutes with the projection onto the orthant for any positive entries, so there the per row freedom is real and the schedule uses it. On a curved block it does not: admissibility forces M to be a positive multiple of the identity on that block. The code enforces this rather than assuming it, in `flatten_on_curved`, which sets every entry of a curved block to the smallest entry in that block whenever the schedule has moved the penalties. The equilibration does the same to its row scaling.

So M is diagonal as stored, uniform per block where the cone is curved, and free per row where it is not. That is $\mathcal{A}(K)$, written as a vector and one flattening pass.

12.5 The inner loop

The inner budget :

$$\text{budget} = \begin{cases} 1 & \text{if } \text{one_step_per_outer}(), \\ \text{max_iter_inner} & \text{otherwise,} \end{cases} \quad (12.1)$$

and the inner loop stops in any case once the running total of Newton steps reaches `max_newton`. The three limits answer different questions: `max_iter_outer` bounds the proximal iteration, `max_iter_inner` bounds one subproblem, and `max_newton` bounds the whole solve in the only unit that costs anything, factorisations. The barrier returns true and therefore takes exactly one Newton step per outer iteration, which is the interior

point method; the projection smoothing returns false and iterates to a tolerance, which is the semismooth method.

The residuals are computed by the smoothing, and the inner loop stops when their largest entry falls below the smallest entry of the per row tolerance. The tolerance is a vector because the schedule sets it per row; the loop needs a scalar, and the smallest entry is the conservative choice.

The matrix is then assembled from `blocks()` and `scalings()` and factored. A failed factorisation is not fatal: ρ_p and ρ are multiplied by a fixed factor, the values are rewritten, the residuals are recomputed at the new weights and the factorisation is retried, up to a budget. Only if the budget is exhausted does the solve stop, with the best point recovered as above.

A factorisation that *succeeds* can be just as bad, and is treated the same way. A backend may report success and still return a direction that is not finite, or one so large that the fraction to the boundary admits almost no step; the outer iteration is then spent taking no step at all, and the next repeats it at the same regularisation. A non finite direction triggers the same response as a failed factorisation, with one addition: it also ratchets a floor under the regularisation that the schedule is not allowed to undo. A short step is recognised the same way, against `degenerate_step`, but that test is off by default: it is left at zero, since recovering from a short step cost more solves than it saved on every set. Without the ratchet a bump lasts exactly one attempt, because the schedule drives the regularisation straight back down on the next outer iteration.

The system is solved for $(\Delta x, \Delta s, \Delta y)$, and `recover_dual_step` reconstructs Δz from Δs . The cone multiplier is eliminated before the factorisation and rebuilt afterwards, so it costs nothing in the linear algebra while remaining a genuine increment rather than a substitution.

The step lengths come from the globalisation, one for the primal pair and one for the dual pair. The smoothing is then offered the chance to correct: if `corrector` returns true it has rewritten its own residual, using the step just computed as an affine predictor, and the system is solved a second time and the step recomputed. The matrix is untouched, so the second solve reuses the factorisation. The projection smoothing declines and this costs nothing; the barrier accepts, which is the Mehrotra predictor and corrector.

Finally the four variables are advanced, the primal pair by α_p and the dual pair by α_d . Every step is guarded: a non finite direction, a non finite step length or a non finite iterate ends the inner loop, and in the last case the best recorded point is restored first.

12.6 What leaves the solver

The reported primal and dual variables are the unscaled x and z . The slack is not the stored s but is recomputed as $b - Ex$ from the original data, so that $Ex + s = b$ holds exactly in the answer rather than to the accuracy the iteration reached. The multiplier y never leaves; it exists for the linear algebra and has no meaning to the caller.

Chapter 13

Equilibration

13.1 Where it happens

Equilibration is `ruiz/ruiz.cpp`, 172 lines, and it is called exactly once, from `solve.cpp` before anything else touches the data:

```
equilibrate(P, q, E, b, cones, ruiz_settings, scaled)
```

It is guarded by `settings.equilibrate`; when that is false the scaled problem is a copy of the original with both scalings set to one and the cost scale set to one, so every downstream formula is unchanged and the branch costs nothing but a copy. The number of passes comes from the shared tuning as `ruiz_iter`, and whether the objective is rescaled from `settings.scale_cost`.

Nothing else in the solver calls it, and nothing else writes to the scaled problem. From that call until the final unscaling every quantity in the solver lives in the scaled problem, which is the contract stated in Chapter 12.

13.2 What is produced

The routine returns a `ScaledProblem`: the four scaled data items and the three scalings that generated them. Writing

$$D = \text{diag}(\text{column_scale}) \in \mathbb{R}^{n \times n}, \quad R = \text{diag}(\text{row_scale}) \in \mathbb{R}^{m \times m}, \quad c = \text{cost_scale} > 0,$$

the scaled data are

$$\tilde{P} = cDPD, \quad \tilde{q} = cDq, \quad \tilde{E} = RED, \quad \tilde{b} = Rb.$$

The variables travel with them. The solver works with $\tilde{x}$, $\tilde{s}$, $\tilde{z}$ related to the original by

$$x = D\tilde{x}, \quad s = R^{-1}\tilde{s}, \quad z = \frac{1}{c}R\tilde{z},$$

Two of the three are `unscale_primal` and `unscale_dual`; the third has no implementation, because the slack is never unscaled. What leaves the solver is $b - Ex$ recomputed from the original data, as the termination chapter describes, so the map for s is stated here for completeness and used nowhere. The three inverse maps `scale_primal`, `scale_slack` and `scale_dual` do exist, and are used only by the warm start, which receives a point in the original problem and must push it in.

Proposition 13.1 (The scaled problem is the same problem). *Suppose $RK = K$. Then $\tilde{x}$ solves*

$$\min_{\tilde{x}, \tilde{s}} \frac{1}{2} \langle \tilde{x}, \tilde{P}\tilde{x} \rangle + \langle \tilde{q}, \tilde{x} \rangle \quad \text{subject to} \quad \tilde{E}\tilde{x} + \tilde{s} = \tilde{b}, \quad \tilde{s} \in K \quad (13.1)$$

if and only if $x = D\tilde{x}$ solves the original problem, and the two objectives differ by the factor c .

Proof. For the objective,

$$\frac{1}{2} \langle \tilde{x}, \tilde{P}\tilde{x} \rangle = \frac{c}{2} \langle \tilde{x}, DP D\tilde{x} \rangle = \frac{c}{2} \langle D\tilde{x}, P D\tilde{x} \rangle = \frac{c}{2} \langle x, Px \rangle, \quad \langle \tilde{q}, \tilde{x} \rangle = c \langle Dq, \tilde{x} \rangle = c \langle q, x \rangle,$$

using that D is symmetric. So the scaled objective is c times the original, and $c > 0$ does not move the minimiser.

For the constraint, $\tilde{E}\tilde{x} = RED\tilde{x} = REx$, so

$$\tilde{E}\tilde{x} + \tilde{s} = \tilde{b} \iff REx + \tilde{s} = Rb \iff Ex + R^{-1}\tilde{s} = b,$$

which is $Ex + s = b$ for $s = R^{-1}\tilde{s}$. Finally $\tilde{s} \in K$ is equivalent to $s = R^{-1}\tilde{s} \in K$ precisely because $RK = K$. $\square$

13.3 The hypothesis on the row scaling

The solver hands the *same* cone objects to the scaled problem that it received for the original one. It never builds a scaled cone. That is only sound if the row scaling leaves each cone where it was.

The row scaling must satisfy $RK = K$, that is $R \in \text{Aut}(K)$.

This is a real restriction, and it is exactly the restriction admissibility imposes on the metric, for a different reason. The two are connected.

It is clear that M admissible implies $MK = K$ but the reverse isn't true. In the solver we take as hypothesis that R is admissible.

This is what `flatten_curved` enforces, and it imposes the same value on the same block in R . After each pass computes a candidate row scaling entry by entry, every block whose cone reports `is_curved` has its entries replaced by a single shared value, the geometric mean of what the pass proposed:

$$r_i \longleftarrow \exp\left(\frac{1}{|J|} \sum_{j \in J} \log r_j\right) \quad \text{for every } i \in J.$$

The geometric mean is the natural choice here: it is the arithmetic mean in the logarithm, which is the space the iteration actually works in, and it leaves a block that is already uniform untouched.

13.4 Ruiz's iteration

The scaling itself is the algorithm of [20], applied to the pair (P, E) rather than to a single matrix. The two share their column index, since both act on x , and only E has rows of its own; P is symmetric and so contributes to a column and to the matching row of D at once.

Algorithm 13.1 Ruiz equilibration of the pair (P, E) **Input:** P, q, E, b , the cones, a pass budget and a tolerance**Output:** $\tilde{P}, \tilde{q}, \tilde{E}, \tilde{b}$ and the scalings D, R

```

1:  $D \leftarrow I, \quad R \leftarrow I$ 
2: for pass = 1, ..., budget do
3:    $\gamma_j \leftarrow \max_i |\tilde{P}_{ij}|$  over the column, then  $\gamma_j \leftarrow \max(\gamma_j, \max_i |\tilde{E}_{ij}|)$ 
4:    $\eta_i \leftarrow \max_j |\tilde{E}_{ij}|$  ▷  $\infty$  norms of the columns and of the rows
5:    $\omega \leftarrow \max\{|\gamma_j - 1|, |\eta_i - 1|\}$  over the nonempty rows and columns
6:    $\gamma_j \leftarrow 1/\sqrt{\gamma_j}, \quad \eta_i \leftarrow 1/\sqrt{\eta_i}$  ▷ clamped, see below
7:   flatten  $\eta$  on every curved block ▷ so that  $R$  stays admissible
8:    $\tilde{P}_{ij} \leftarrow \gamma_i \gamma_j \tilde{P}_{ij}, \quad \tilde{E}_{ij} \leftarrow \eta_i \eta_j \tilde{E}_{ij}$ 
9:    $\tilde{q} \leftarrow \gamma \odot \tilde{q}, \quad \tilde{b} \leftarrow \eta \odot \tilde{b}$ 
10:   $D \leftarrow \gamma D, \quad R \leftarrow \eta R$ 
11:  if  $\omega \leq \text{tolerance}$  then
12:    break ▷ every nonempty row and column is within tolerance of unit norm
13:  end if
14: end for
15: rescale the objective, if enabled

```

The convergence is fast enough that the pass budget is small: the default is ten, and the loop usually leaves early on the tolerance 10^{-3} .

Three guards. The square root is taken through

$$\text{guarded_root}(v) = \begin{cases} 1 & v \leq 10^{-12}, \\ 1/\sqrt{\min(10^8, \max(10^{-8}, v))} & \text{otherwise,} \end{cases}$$

so an empty row or column is left at scale one rather than producing an infinite scaling, and a row whose entries span more than sixteen orders of magnitude is clamped instead of being trusted. The convergence test ignores rows and columns whose norm is below 10^{-12} , so an empty one cannot keep the loop running to its budget.

13.5 The cost scaling

After the passes, and only if `scale_cost` is set, the objective is brought to unit size:

$$c = \frac{1}{\max(\|\tilde{P}\|_{\max}, \|\tilde{q}\|_{\infty})}, \quad \tilde{P} \leftarrow c\tilde{P}, \quad \tilde{q} \leftarrow c\tilde{q},$$

skipped entirely when that maximum falls outside $[10^{-8}, 10^8]$. This touches the objective alone, so the primal solution is unchanged and only the dual is scaled, which is why c appears in `unscale_dual` and in no other unscaling map.

Remark 13.2. The solver is tuned on the scaled problem, and the tolerances, the initial penalties and the restart thresholds are all calibrated against data whose rows and columns have unit norm. Turning `equilibrate` off is supported, and useful for isolating a numerical question, but it is not a configuration the schedules were designed for.

Chapter 14

Initialisation

14.1 Where it happens

Initialisation is `initialisation/`, 295 lines over four files: an interface and a dispatch in `initialisation.cpp`, and one implementation for each road.

file	class	used by
<code>feasible_start.cpp</code>	<code>FeasibleStart</code>	the semismooth road
<code>interior_start.cpp</code>	<code>InteriorStart</code>	the barrier road
<code>initialisation.cpp</code>	the shared helpers	both

The choice is not a setting. It is made in `solve.cpp` by asking the smoothing which it needs,

```
make_initialisation(smoothing->requires_interior(), settings)
```

and the factory returns the interior start when that answer is true and the feasible start when it is false. A road therefore cannot be given the wrong initialisation, because the question is answered by the object that will consume the point.

Both classes provide `cold` and `warm`, and `solve.cpp` calls exactly one of the two, immediately after the equilibration and before the outer loop begins. Everything below happens in the scaled problem.

14.2 The semismooth start

The projection smoothing is exterior: it accepts any point at all, and nothing about s has to lie in the cone for the residuals to be defined. That freedom makes the question interesting rather than trivial, since some choice still has to be made and there is no feasibility requirement to force it.

The choice made is the following. Set

$$x = 0, \quad y = 0, \quad z = 0, \quad s = b + \Pi_{K^*}(-b),$$

then anchor the four centres on that point. In the code the slack is built in two lines, the projection being taken onto the dual cone because that is the only projection the kernel exposes.

Proposition 14.1 (The semismooth start is the nearest feasible slack). *The slack above satisfies*

$$s = \Pi_K(b), \quad (14.1)$$

and consequently, among all slacks lying in K , it is the one that minimises the initial equality residual: $s \in K$ and

$$\|r_{\text{eq}}\| = \|b - s\| = \min_{\sigma \in K} \|b - \sigma\|. \quad (14.2)$$

Proof. Moreau's decomposition applied at b gives

$$b = \Pi_K(b) + \Pi_{K^\circ}(b), \quad K^\circ = -K^*.$$

Projection commutes with negating both the point and the cone, so

$$\Pi_{K^\circ}(b) = \Pi_{-K^*}(b) = -\Pi_{K^*}(-b),$$

and substituting gives $b = \Pi_K(b) - \Pi_{K^*}(-b)$, that is $\Pi_K(b) = b + \Pi_{K^*}(-b)$, which is the displayed s .

For the second claim, the equality residual is $r_{\text{eq}} = M_s(y - y_c) - (Ex + s - b)$, and at this point $x = 0$ and $y = y_c = 0$, so $r_{\text{eq}} = b - s$. The projection onto a closed convex set is by definition the unique minimiser of the distance to it, so $\Pi_K(b)$ minimises $\|b - \sigma\|$ over $\sigma \in K$. $\square$

So the start is feasible for the cone and as close as it can be to feasible for the equality, which is what the class name records. Note what it does *not* do: it makes no attempt to satisfy the stationarity, and with $z = 0$ the initial dual residual is simply q .

Algorithm 14.1 The semismooth cold start

Input: the scaled data, the cones

Output: an iterate and its centres

- 1: $x \leftarrow 0, \quad y \leftarrow 0, \quad z \leftarrow 0$
 - 2: $w \leftarrow -b$
 - 3: $w \leftarrow \Pi_{K^*}(w)$ $\triangleright$ blockwise, the only projection the kernel exposes
 - 4: $s \leftarrow b + w$ $\triangleright = \Pi_K(b)$ by Proposition 14.1
 - 5: $(x_c, s_c, y_c, z_c) \leftarrow (x, s, y, z)$
-

The warm path does none of this. It takes the point it was handed, already scaled into the working problem by the caller, and only anchors the centres on it. There is nothing to repair: an exterior smoothing has no feasibility for a warm point to violate.

14.3 The interior start

The barrier road cannot begin anywhere. It requires

$$s \in \text{int } K, \quad z \in \text{int } K^*,$$

since the barrier and its derivatives are undefined outside, and it requires the two to be far enough inside that the first step of Definition 7.37 is not immediately tiny. The construction has three stages, and the second may be skipped.

14.3.1 A reference point

Every cone supplies a fixed strictly feasible direction through `interior_direction`, and the first stage simply takes it:

$$x \leftarrow 0, \quad y \leftarrow 0, \quad s_j \leftarrow d_j^{\text{primal}}, \quad z_j \leftarrow d_j^{\text{dual}},$$

on every block with an interior, and $s_j = z_j = 0$ on the blocks without one.

cone	primal direction	dual direction
Nonneg	1	1
SecondOrder	e_0	e_0
PSDTriangle	$\text{svec}(I)$	$\text{svec}(I)$
Exponential	$(-1, 1, 2)$	$(-1, 0, 1)$
Power	α dependent	α dependent

Table 14.1: The interior directions. Each has margin exactly one, by Proposition 14.4, which is what the shift relies on.

This point is strictly feasible and nothing else. It ignores the problem entirely, and if the seeded stage is disabled it is what the solver starts from.

14.3.2 A seeded point

When `seeded_start` is set, which it is by default, the reference point is used only as a place to build a scaling, and the actual start comes from one linear solve.

The barrier scaling is built at the reference point, the cone operator blocks are materialised from it, the road is assembled and factored, and the system is solved once with the problem data itself on the right hand side:

$$r_{\text{stat}}^x = q, \quad \varrho_s = 0, \quad r_{\text{eq}} = b.$$

The three components of the solution become x , s and y directly, and the dual is recovered from the slack by

$$z = -(G - \rho_p I)s = -Hs.$$

There is no theorem here; the construction is the heuristic of [11] carried over to the conic case, and what it computes is worth stating plainly.

Proposition 14.2 (What the seed solves). *Let $G = \rho_p I + H$ be the cone operator built at the reference point. The seeded start is the exact solution of the problem obtained from the original one by replacing the cone constraint with the quadratic form of G ,*

$$\min_{x,s} \frac{1}{2} \langle x, (P + \rho I)x \rangle + \langle q, x \rangle + \frac{1}{2} \langle s, Gs \rangle \quad \text{subject to} \quad Ex + s - M_s y = b, \quad (14.3)$$

and $z = -Hs$ is the dual it induces.

Proof. The Newton system of the road, with the right hand side above, reads

$$(P + \rho I)\Delta x + E^\top \Delta y = -q, \quad G\Delta s + \Delta y = 0, \quad E\Delta x + \Delta s - M_s \Delta y = b.$$

These are exactly the stationarity conditions of the displayed problem in (x, s) with multiplier y : the first is stationarity in x , the second is stationarity in s , and the third

is the constraint. Since $P + \rho I \succ 0$ and $G \succ 0$, the problem is strictly convex and its solution is unique, so the Newton step from the origin is that solution rather than an approximation to it. The second equation gives $\Delta y = -G\Delta s$, and the dual assigned to the cone is the part of that coming from H alone, $z = -Hs$, which is the gradient at s of the quadratic model $\frac{1}{2} \langle s, Hs \rangle$ that replaced the barrier. $\square$

In words: the seed is the point the solver would reach if the barrier were exactly its own second order model at the reference point. That model is a poor approximation far from the reference, which is why the result is not trusted and is repaired in the next stage. If the factorisation fails, the whole stage is abandoned and the reference point stands.

14.3.3 Pushing the point inside

Nothing so far guarantees that the seeded point is interior at all: it solves a quadratic problem that has forgotten the cone. The last stage restores the requirement, on the primal and on the dual separately.

The quantity both stages measure is the margin. It is a property of a cone together with a chosen direction inside it, and every cone in the kernel supplies both.

Definition 14.3 (Margin). Let $K \subseteq \mathbb{R}^d$ be a pointed closed convex cone and let $d \in \text{int } K$, $d \neq 0$, be a distinguished interior direction. The *margin* of a point $v \in \mathbb{R}^d$ is

$$m_K(v) = \sup\{t \in \mathbb{R} : v - td \in K\}. \quad (14.4)$$

The supremum is over all of $\mathbb{R}$, not only over $t \geq 0$, so the margin is signed: it is the amount of d that can be removed from v before leaving the cone when v is inside, and minus the amount that must be added to enter it when v is outside. That the definition is signed is what allows one formula to repair a point that is merely too close to the boundary and a point that is not in the cone at all, which is the situation the seeded start leaves behind.

Proposition 14.4 (Properties of the margin). *With K and d as above, the supremum is finite and attained, and*

- (i) $m_K(v) \geq 0$ if and only if $v \in K$, and $m_K(v) > 0$ if and only if $v \in \text{int } K$;
- (ii) m_K is positively homogeneous of degree one;
- (iii) m_K is concave, and therefore superadditive;
- (iv) $m_K(d) = 1$.

Proof. Write $T(v) = \{t : v - td \in K\}$. It is closed, because K is closed and the map $t \mapsto v - td$ is continuous. It is bounded above: were it not, then $v - td \in K$ for arbitrarily large t , hence $v/t - d \in K$ by the cone property, and letting $t \rightarrow \infty$ and using closedness gives $-d \in K$, so $d \in K \cap (-K) = \{0\}$ by pointedness, contradicting $d \neq 0$. A closed set in $\mathbb{R}$ bounded above contains its supremum.

(i) If $v \in K$ then $0 \in T(v)$, so $m_K(v) \geq 0$; conversely $m_K(v) \geq 0$ with the supremum attained gives $v - m_K(v)d \in K$ and hence $v \in K$, being the sum of that element with $m_K(v)d \in K$. If $v \in \text{int } K$ then $v - td \in K$ for all small $t > 0$, so $m_K(v) > 0$; conversely if $m_K(v) = t > 0$ then

$$v = (v - td) + td \in K + \text{int } K \subseteq \text{int } K.$$

Where the last inequality is due to the fact that if O is convex then $\alpha x + (1 - \alpha)y \in \text{int } O$ for any $x \in O, y \in \text{int } O, \alpha \in [0, 1[$. Used at $\alpha = \frac{1}{2}$ combined with the fact that K is a cone.

(ii) For $\lambda > 0$,

$$T(\lambda v) = \{t : \lambda v - td \in K\} = \{t : v - (t/\lambda)d \in K\} = \lambda T(v),$$

dividing by λ inside K , so the suprema scale by λ .

(iii) Let $t_1 = m_K(v_1)$, $t_2 = m_K(v_2)$ and $\theta \in [0, 1]$. Both $v_1 - t_1 d$ and $v_2 - t_2 d$ lie in K , so their convex combination does too:

$$\theta v_1 + (1 - \theta)v_2 - (\theta t_1 + (1 - \theta)t_2)d \in K,$$

whence $m_K(\theta v_1 + (1 - \theta)v_2) \geq \theta t_1 + (1 - \theta)t_2$. Superadditivity follows from concavity and homogeneity:

$$m_K(a + b) = 2 m_K\left(\frac{a + b}{2}\right) \geq 2\left(\frac{1}{2}m_K(a) + \frac{1}{2}m_K(b)\right) = m_K(a) + m_K(b).$$

(iv) $T(d) = \{t : (1 - t)d \in K\}$. For $t \leq 1$ the factor $1 - t$ is nonnegative and $(1 - t)d \in K$ since K is a cone containing d . For $t > 1$ membership would give $-d \in K$, excluded as above. Hence $m_K(d) = 1$. $\square$

Part (iv) holds for every pointed cone and every nonzero interior direction, so it is not a property the five directions of the table were chosen to have. On the three symmetric cones the margin has a closed form, and each is the definition evaluated:

$$m_{\mathbb{R}_+^d}(v) = \min_i v_i, \quad m_{\mathcal{Q}^d}(v) = v_0 - \|v_{1:}\|, \quad m_{\mathcal{S}_+^k}(X) = \lambda_{\min}(X). \quad (14.5)$$

The exponential and power cones have no closed form and bisect on the membership predicate instead, bracketing by doubling and then halving a fixed number of times, so what they return is Definition 14.3 to the bisection tolerance rather than exactly.

Algorithm 14.2 `shift_inside`, on one side

Input: a point v , the cones, the constants π_{abs} and π_{rel}

Output: a point whose every block has margin at least the target

```

1:  $m \leftarrow \min_j \text{margin}(K_j, v_j)$  over the blocks with an interior
2:  $\varsigma \leftarrow \max_j \|v_j\|_\infty$  over the same blocks
3:  $t^* \leftarrow \pi_{\text{abs}} + \pi_{\text{rel}} \varsigma$   $\triangleright$  the target margin, scaled to the point
4:  $t \leftarrow \max(0, t^* - m)$ 
5: if  $t = 0$  then
6:   return  $v$   $\triangleright$  already inside by enough
7: end if
8: for every block  $j$  with an interior do
9:    $v_j \leftarrow v_j + t d_j$   $\triangleright d_j$  the interior direction of that cone
10: end for
11: return  $v$ 
```

The defaults are $\pi_{\text{abs}} = 1$ and $\pi_{\text{rel}} = 0.1$, so the target is one plus a tenth of the largest entry of the point. Making the target grow with the point matters: a fixed absolute margin would be generous on a problem whose data is of order one and negligible on one whose data is of order 10^6 .

That the shift achieves what it claims is elementary, and rests on one property of the margin.

Proposition 14.5 (The shift restores the margin). *Let t^* be the target, let $m^{\min}$ be the smallest block margin before the shift, and let $t = \max(0, t^* - m^{\min})$. Then after the shift every block has margin at least t^* .*

Proof. Let j be any block. By superadditivity and $m(d_j) = 1$, both from Proposition 14.4,

$$m(v_j + t d_j) \geq m(v_j) + t m(d_j) = m(v_j) + t.$$

If $t > 0$ then $m(v_j) \geq m^{\min}$ gives $m(v_j) + t \geq m^{\min} + t^* - m^{\min} = t^*$. If $t = 0$ then $t^* \leq m^{\min} \leq m(v_j)$ already. $\square$

Note that the proposition needs no hypothesis beyond Proposition 14.4, so it covers the exponential and power blocks as well, up to the accuracy with which their margins are bisected.

Algorithm 14.3 The interior cold start

Input: the scaled data, the cones, a factored road

Output: a strictly feasible iterate and its centres

```

1:  $x \leftarrow 0, \quad y \leftarrow 0$ 
2: for every block  $j$  do
3:    $s_j \leftarrow d_j^{\text{primal}}, \quad z_j \leftarrow d_j^{\text{dual}}$  if the cone has an interior, else 0
4: end for
5: if seeded_start then
6:   build the scalings and the blocks of  $G$  at this point
7:   assemble and factor the road
8:   if the factorisation succeeded then
9:     solve once with right hand side  $(q, 0, b)$ 
10:     $(x, s, y) \leftarrow$  the solution,  $z \leftarrow -(G - \rho_p I)s$ 
11:    shift_inside the primal, then the dual
12:   end if
13: end if
14:  $(x_c, s_c, y_c, z_c) \leftarrow (x, s, y, z)$ 
```

Note where the shift sits: inside the seeded branch. If the seed is skipped, or if the factorisation fails, no shift is applied, because the reference point is already interior by construction and there is nothing to repair.

The warm path applies the two shifts and nothing else. A point inherited from a neighbouring problem is usually interior already, but it may sit very close to the boundary of the new one, and the shift is what prevents the first step of Definition 7.37 from being vanishingly small.

14.4 The centres

Both roads finish by calling `anchor_centres`, which is four assignments:

$$x_c \leftarrow x, \quad s_c \leftarrow s, \quad y_c \leftarrow y, \quad z_c \leftarrow z.$$

This is the only place the centres are set outside the schedule.

Remark 14.6 (The warm path trusts its input.). Neither class validates the point it is handed. The semismooth road has nothing to validate, but the interior road assumes the caller has already scaled the point into the working problem, and the shift will happily push an unscaled point further away from where it should be rather than towards it.

Chapter 15

Termination

15.1 Where it happens

Termination is `termination/`, 183 lines over two files, and it is the smallest directory in the solver. It is reached from three places in `solve.cpp`, all of them in the outer loop.

call	when	decides
<code>evaluate</code>	after the inner loop	SOLVED
<code>primal_infeasible</code>	after the displacement	PRIMALINFEASIBLE
<code>dual_infeasible</code>	after the displacement	DUALINFEASIBLE
<code>evaluate</code>	once more, on exit	the rescue below

The remaining outcomes need no call. `NUMERICALFAILURE` is set where a factorisation exhausts its retries, `MAXNEWTON` where the global count of Newton steps is reached, and `MAXITEROUTER` is the initial value of `results.status`, so a loop that simply runs out of outer iterations reports it without anyone assigning it. The three limits are distinguished on purpose: a solve stopped by the Newton budget and one stopped by the outer budget have failed for different reasons and are fixed by different settings.

Two things about the call are worth stating before the contents. It is given `original` rather than `working`, and explicitly unscaled copies of x and z : the verdict is pronounced on the problem the caller posed, never on the equilibrated one. And it is a free function of the data and the point, holding no state between calls, so the same point always receives the same verdict.

15.2 What is measured

The four quantities of Definition 9.30 are formed as follows.

The slack is not read from the iterate but rebuilt from the data,

$$\hat{s} = b - Ex,$$

so that $Ex + \hat{s} = b$ holds exactly and the equality constraint cannot fail. Only four things remain that can, and they are the four tested.

The primal residual is the distance from that slack to the cone. It is computed through the dual projection, which is the only projection the kernel exposes, and they are the same number by the theory part.

With that, the four quantities are

$$\begin{aligned} \text{primal residual} &= \|\Pi_{K^*}(-\hat{s})\|_\infty = \text{dist}(\hat{s}, K), & \text{dual residual} &= \|Px + q + E^\top z\|_\infty, \\ \text{cone violation} &= \|z - \Pi_{K^*}(z)\|_\infty = \text{dist}(z, K^*), & \text{complementarity} &= |\langle \hat{s}, z \rangle|, \end{aligned}$$

The first three are compared against $\varepsilon_{\text{abs}} + \varepsilon_{\text{rel}}$ times their own scale, and the gap against its own pair $\varepsilon_{\text{abs}}^{\text{gap}} + \varepsilon_{\text{rel}}^{\text{gap}}$ times its own scale,

$$\begin{aligned} \pi_{\text{scale}} &= \max\{\|Ex\|_\infty, \|\hat{s}\|_\infty, \|b\|_\infty\}, & \delta_{\text{scale}} &= \max\{\|Px\|_\infty, \|q\|_\infty, \|E^\top z\|_\infty\}, \\ \gamma_{\text{scale}} &= \|z\|_\infty, & \eta_{\text{scale}} &= \max\{|p^*|, |d^*|\}. \end{aligned}$$

The gap carries its own pair, as it does in PIQP and in Clarabel.

The scale of the fourth is the pair of objectives, and that is only meaningful because the complementarity *is* the duality gap: by the proposition on the gap, $p^* - d^* = \langle \hat{s}, z \rangle$, so measuring $|\langle \hat{s}, z \rangle|$ against $\max(|p^*|, |d^*|)$ is a relative gap test and not an arbitrary rescaling.

15.3 Termination of the inner loop

The test above ends the solve. A second, quite different test ends each inner loop, and the two should not be confused: the first is about the original problem, the second about the current subproblem.

The inner loop stops when the residuals of the *smoothing* fall below the tolerance the schedule currently holds,

$$\max\{\|r_{\text{stat}}^x\|_\infty, \|r_{\text{stat}}^s\|_\infty, \|r_{\text{eq}}\|_\infty, \|r_{\text{cone}}\|_\infty\} \leq \min_i (\varepsilon_k)_i.$$

The tolerance is a vector because the schedule sets it per row; the loop needs a scalar, and the smallest entry is the conservative reading. Nothing here is compared against ε_{abs} : the subproblem is allowed to be solved loosely, and how loosely is the schedule's decision.

The two roads differ only in the budget. The test is the same for both, but

$$\text{budget} = \begin{cases} 1 & \text{barrier, since } \text{one_step_per_outer} \text{ is true,} \\ \text{max_inner} & \text{projection.} \end{cases}$$

The barrier road therefore evaluates the test once, and if it fails takes exactly one step and leaves; it never iterates to a tolerance, because an interior point method has no subproblem to solve to one. The projection road iterates until the test passes or the budget is exhausted, and reports which through `inner_converged`, a flag the schedule reads when it decides how to move the penalties.

15.4 The certificates

Neither certificate is derived here; both are Definition 8.15 of the theory part, and what follows is how the code tests for them.

Both are tested on the displacement over one outer iteration, δx and δz , which is the object the asymptotic result of the theory part identifies as converging to a certificate when one exists. The pairing is crossed, the dual displacement certifying primal infeasibility and the primal displacement certifying dual infeasibility.

Normalisation comes first. The three conditions of each certificate are positively homogeneous: if w satisfies them so does λw for every $\lambda > 0$. A certificate is therefore only ever determined up to positive scale, and a tolerance applied to an unnormalised vector would be meaningless, passing for a large displacement and failing for a small one that points the same way. Both tests begin by dividing through by $\|\cdot\|_\infty$, and return false at once if that norm vanishes.

Algorithm 15.1 The primal infeasibility test

Input: the dual displacement δz , a tolerance ε_{inf}

Output: whether δz certifies that the primal is infeasible

```

1: if  $\|\delta z\|_\infty = 0$  then
2:   return false
3: end if
4:  $w \leftarrow \delta z / \|\delta z\|_\infty$ 
5: if  $\|w - \Pi_{K^*}(w)\|_\infty > \varepsilon_{\text{inf}}$  then
6:   return false  $\triangleright w \notin K^*$ 
7: end if
8: if  $\|E^\top w\|_\infty > \varepsilon_{\text{inf}}$  then
9:   return false  $\triangleright E^\top w \neq 0$ 
10: end if
11: return  $\langle b, w \rangle < -\varepsilon_{\text{inf}}$ 

```

The three checks are the three conditions of the definition, in the split notation where $b = -d$, so $\langle z, d \rangle > 0$ reads $\langle b, w \rangle < 0$. That they suffice is one line, worth repeating because it is the whole content of the test.

Proposition 15.1 (The primal certificate is sound). *If $w \in K^*$, $E^\top w = 0$ and $\langle b, w \rangle < 0$, then no (x, s) satisfies $Ex + s = b$ with $s \in K$.*

Algorithm 15.2 The dual infeasibility test

Input: the primal displacement δx , a tolerance ε_{inf}

Output: whether δx certifies that the dual is infeasible

```

1: if  $\|\delta x\|_\infty = 0$  then
2:   return false
3: end if
4:  $u \leftarrow \delta x / \|\delta x\|_\infty$ 
5: if  $\|Pu\|_\infty > \varepsilon_{\text{inf}}$  then
6:   return false  $\triangleright Pu \neq 0$ 
7: end if
8: if  $\langle q, u \rangle > -\varepsilon_{\text{inf}}$  then
9:   return false  $\triangleright$  the objective does not decrease along  $u$ 
10: end if
11: return  $\|\Pi_{K^*}(Eu)\|_\infty \leq \varepsilon_{\text{inf}}$   $\triangleright \Pi_{K^*}(v) = 0$  exactly when  $-v \in K$ 

```

Proposition 15.2 (The dual certificate is sound). *If $Pu = 0$, $-Eu \in K$ and $\langle q, u \rangle < 0$, then the primal objective is unbounded below on the feasible set whenever that set is nonempty, so the dual is infeasible.*

15.5 The verdict, and the rescue

After the loop the status is whatever the loop left, and one further step is taken before returning. If the status is not SOLVED, the largest of the four quantities at the final iterate is compared with the best value seen during the solve; if the final iterate is the worse of the two, the best point is restored, graded again, and may be declared SOLVED even though the last iterate was not.

Remark 15.3 (The slack is rebuilt, so one residual can never be seen.). Because $\hat{s} = b - Ex$ is recomputed, the equality residual is identically zero at the point being graded, whatever the iterate's own s was doing. That is the correct choice, since the answer returned to the caller carries that same recomputed slack. But it means a bug that lets s drift away from $b - Ex$ inside the solver will never show up in the verdict, and must be caught by the inner residuals instead.

Remark 15.4 (A certificate is only ever approximate.). Every test is up to ε_{inf} , so what is reported is a vector that nearly satisfies the three conditions, not one that satisfies them. On a problem that is feasible but very badly conditioned, a displacement can pass the three checks without a certificate existing, and the solver will report infeasibility for a problem that has an answer. The tolerance is set to 10^{-9} for that reason, and lowering it trades one kind of wrong answer for another.

Chapter 16

The Newton systems

16.1 Where it happens

The two roads are `road/`, 847 lines over four files: an interface and the selection rule in `road.cpp`, and one implementation each in `three_by_three.cpp` and `schur.cpp`. They are the only place in the solver that knows what a sparse matrix is.

A road is used in four steps, and the separation between them is the reason the solver is fast.

call	how often	what it does
<code>setup</code>	once per solve	plans the sparsity pattern and factorises symbolically
<code>values</code>	when ρ moves	writes the entries coming from P , E and ρ
<code>assemble</code>	every inner iteration	writes the entries coming from G and M_s
<code>factor</code>	every inner iteration	the numeric factorisation
<code>solve</code>	once or twice per iteration	the triangular solves and refinement

Table 16.1: The road interface. `setup` is the only call that decides where a nonzero may appear; everything after it writes into positions that already exist.

That split is what `PatternBuilder` and `position_of` exist for. The pattern is built once from triplets, compressed, and then every later write goes through an index recorded at setup. If a write ever asks for a position the pattern does not contain, `position_of` raises rather than corrupting a neighbouring entry, with a message naming the disagreement between setup and assembly.

16.2 Choosing a road

When the caller asks for `Auto`, the choice is made once, before anything is built, from three numbers: the column count n , the row count m , and the number of nonzeros in E .

Algorithm 16.1 `resolve_road`**Input:** the requested kind, n , m , and $\text{nnz}(E)$ **Output:** a concrete road

```

1: if the request is not Auto then
2:   return it                                ▷ the caller's choice is never overridden
3: end if
4: if  $m \leq n$  then
5:   return THREEBYTHREE                      ▷ too few rows for the reduction to pay
6: end if
7:  $d \leftarrow \text{nnz}(E)/(nm)$                     ▷ the density of the constraint
8:  $\hat{d} \leftarrow \min(1, d^2 m)$                 ▷ the expected density of  $E^\top \Sigma E$ 
9: if  $\hat{d} < \hat{d}_{\min}$  then
10:  return THREEBYTHREE
11: end if
12: return SCHUR

```

Remark 16.1 (The first guard is not about size.). The reduced system is $n \times n$ and the three by three system is $(n + 2m) \times (n + 2m)$, so the reduction is always the smaller of the two and the guard $m \leq n$ is not a statement about dimension. It is a statement about flops, and the two counts can be written down. Assuming that G is diagonal, P is sparse and we are using a minimum degree ordering, one can give the following estimate of the number flops of the 3x3 road:

$$\underbrace{F_P}_{\text{the } n \text{ sparse columns}} + \underbrace{\frac{nm^2}{2}}_{d_k=m \text{ for each of them}} + \underbrace{\frac{m^3}{6}}_{\text{the dense } m \times m \text{ corner}},$$

where F_P is the cost of factoring P alone.

The Schur road pays instead assuming that E is dense:

$$\underbrace{\frac{mn^2}{2}}_{\text{forming } E^\top \Sigma E} + \underbrace{\frac{n^3}{6}}_{\text{factoring the dense } S},$$

the second term because adding $E^\top \Sigma E$ to P makes every entry of S nonzero when E is dense, so $d_k = n - k$ and no ordering can help.

The two expressions are exchanged by swapping m and n . With F_P negligible they therefore cross exactly at $m = n$, which is where the guard sits.

m	three by three	Schur	measured ratio	from the counts
n	$6.64 \cdot 10^5$	$6.64 \cdot 10^5$	1.00	1.00
$n/2$	$1.55 \cdot 10^5$	$4.14 \cdot 10^5$	2.67	2.86
$n/4$	$3.88 \cdot 10^4$	$2.89 \cdot 10^5$	7.45	8.62
$n/10$	$7.31 \cdot 10^3$	$2.14 \cdot 10^5$	29.3	41.9

Table 16.2: Flops by symbolic elimination on the two patterns, $n = 100$, P banded and E dense, under a minimum degree ordering computed for each. The crossover at $m = n$ is exact. The counts overstate the advantage at small m , where F_P is no longer negligible against a border term that has become small.

The argument does not need P sparse: a dense P adds $n^3/6$ to the three by three road, but the Schur road pays that same $n^3/6$ and the $mn^2/2$ to form the product, so it still does not gain. Sparsity in P widens the gap rather than creating it. What creates it is that the reduction always pays $O(mn^2)$ to build S and recoups it only by shrinking the factorisation from $n + 2m$ to n , which is no great shrink when $m \leq n$.

Remark 16.2 (The guard is coarser than the count.). The counts assume E dense, and the guard does not: it returns on $m \leq n$ alone, before the density of E is ever looked at. The argument above therefore justifies it on only half of its domain, and that is deliberate, because the two halves are not symmetric.

When E is dense and $m \leq n$, taking the Schur road is expensive, and the table gives the price: a factor of thirty at $m = n/10$. When E is sparse and $m \leq n$, the product $E^\top \Sigma E$ may itself stay sparse, the $n^3/6$ never appears, and the Schur road might well have been the cheaper of the two; the guard is then conservative and gives something up. But both systems are sparse in that case and the larger has size at most $3n$, so what is given up is bounded and small in absolute terms. The guard accepts a bounded loss on sparse problems in order to avoid an unbounded one on dense problems.

Remark 16.3 (On a curved block none of this holds.). A semidefinite block of side k contributes a dense G of dimension $k(k+1)/2$, and is treated differently than a pure diagonal block. The guard $m \leq n$ is applied to curved problems all the same, on the grounds that argument is the right intuition even where the count is not available, and it should be read there as a heuristic rather than as the conclusion of the paragraphs above.

Remark 16.4 (The second guard is about the density of the reduction.). By the argument above $m \geq n$ and E is dense we should take the Schur road. The quantity $\hat{d}$ is the estimate that decides it. If E has density d , then a given entry of $E^\top \Sigma E$ is a sum over m terms, each present when two independent entries of E are, so under an independence assumption it is nonzero with probability about $1 - (1 - d^2)^m \approx d^2 m$. The rule sends the problem to the Schur road when that estimate exceeds $\hat{d}_{\min}$, a fixed threshold of the selection rule rather than a setting. Another reason why the Schur road is preferred when the reduced system is that its matrix is positive definite, so the backend may use a supernodal factorisation, which spends its time in dense level three kernels; on a dense reduced system that is far faster than the simplicial factorisation the indefinite three by three system is restricted to. A sparse reduced system gives the supernodal algorithm nothing to work with, and then the larger but sparser three by three matrix wins.

16.3 The three by three road

The system is (9.27), of size $n + 2m$, and it is assembled directly. Its virtue is stated in the theory part: it is symmetric quasi-definite, so an $LDL^\top$ factorisation exists for any symmetric permutation, and the ordering may therefore be chosen for sparsity alone without any pivoting for stability.

16.3.1 Three ways to hold a block

The cone operator occupies the middle diagonal block, and how it is stored depends on what shape the cone will produce. The decision is taken at setup, from `widest_operator_kind`, and never revisited.

allocation	when	entries for a block of size ℓ
DIAGONAL	the widest kind is diagonal	ℓ
EXPANDED	diagonal plus low rank, and $\ell \geq 8$	$3\ell + 2$
DENSE	otherwise	$\ell(\ell + 1)/2$

The middle row is the interesting one. A diagonal plus rank two block could always be stored densely, but it need not be: introducing two auxiliary variables carries the low rank part exactly while keeping the matrix sparse.

Proposition 16.5 (The expansion is exact). *Let $G = \delta I + UTU^\top$ with $T \in \mathcal{S}^2$ invertible, and let $T = Q\Lambda Q^\top$ with $V = UQ$, so that $G = \delta I + V\Lambda V^\top$. Then the augmented matrix*

$$\begin{pmatrix} \delta I & V \\ V^\top & -\Lambda^{-1} \end{pmatrix} \quad (16.1)$$

has Schur complement G onto its leading block, and its inertia is that of δI together with one negative eigenvalue for each positive λ_k and one positive for each negative λ_k .

Proof. Eliminating the trailing block gives

$$\delta I - V(-\Lambda^{-1})^{-1}V^\top = \delta I + V\Lambda V^\top = G.$$

The inertia statement is the inertia additivity of a symmetric block elimination: the trailing block $-\Lambda^{-1}$ contributes the signs of $-1/\lambda_k$, which are opposite to those of λ_k . $\square$

The sign bookkeeping is not decoration. The three by three matrix is quasi-definite because its leading block is positive definite and its trailing block negative definite, and an auxiliary variable must be placed on the side matching the sign of $-1/\lambda_k$ or the property is lost. The code counts the two cases separately as it writes them, which is what `auxiliary_positive` and `auxiliary_negative` record.

Algorithm 16.2 `write_expanded`, one block

Input: a block $G = \delta I + UTU^\top$ of size ℓ

Output: its entries, written into the pattern reserved at setup

- 1: $(Q, \Lambda) \leftarrow$ eigendecomposition of the 2×2 middle T
 - 2: $V \leftarrow UQ$
 - 3: write δ on the ℓ diagonal positions of the block
 - 4: **for** $k = 1, 2$ **do**
 - 5: **if** $|\lambda_k| \leq \varepsilon_{\text{pin}}$ **then**
 - 6: write a zero column and -1 on the auxiliary diagonal $\triangleright$ the correction has rank below two here
 - 7: **else**
 - 8: write the column V_k and $-1/\lambda_k$ on the auxiliary diagonal
 - 9: **end if**
 - 10: record the sign of that diagonal
 - 11: **end for**
-

The pinning branch is what makes the expansion safe when the rank two correction degenerates to rank one or zero, which happens on a second order block whose linearisation point sits on the axis. Writing $-1/\lambda_k$ there would overflow; instead the column is zeroed and the diagonal set to -1 , so that the auxiliary variable decouples and contributes nothing.

Remark 16.6 (Why eight.). Dense storage of a block of size ℓ costs $\ell(\ell + 1)/2$ entries in the lower triangle, and the expansion costs ℓ on the diagonal, 2ℓ in the two columns and 2 on the auxiliary diagonal. The expansion is cheaper when

$$\frac{\ell(\ell + 1)}{2} > 3\ell + 2 \quad \Longleftrightarrow \quad \ell^2 - 5\ell - 4 > 0 \quad \Longleftrightarrow \quad \ell \geq 6,$$

so the default threshold of eight is that crossover with a small margin, and a second order block of dimension three or four is stored densely because it is genuinely cheaper that way.

16.3.2 Solving

The right hand side is assembled from the three residuals, with zeros on the auxiliary rows, and the sign of the third block matches the theory part:

$$\text{rhs} = (-r_{\text{stat}}^x, -\varrho_s, +r_{\text{eq}}, 0).$$

After the backend's solve, the direction is read off the first three segments and the auxiliary entries are discarded; they exist only to carry the low rank part through the factorisation.

Iterative refinement. Every solve is refined, at most `refinement_budget` times, which is the shared setting `refine` and defaults to eight. Each pass forms the true residual against the assembled matrix, solves with it, and adds the correction. The loop stops early on either of two conditions, and the second matters as much as the first:

$$\|r\|_{\infty} \geq \text{the previous pass's value}, \quad \text{or} \quad \|r\|_{\infty} \leq \varepsilon_{\text{mach}} \max(\|\text{rhs}\|_{\infty}, 1).$$

Refinement that has stopped improving is refinement that has started chasing rounding, and continuing costs a solve per pass for nothing.

16.4 The Schur road

The reduced system is (9.29), of size n and positive definite. Two things have to be arranged: the weight Σ , and the reduction itself without ever forming G^{-1} .

16.4.1 The weight

The weight is $\Sigma = G(I + M_s G)^{-1}$, computed once per block per iteration by `schur_weight`, and its form follows the block's.

block	Σ
diagonal	$g/(1 + g\mu_s)$, elementwise
diagonal plus low rank	diagonal plus low rank, one $r \times r$ solve
dense	one dense solve of $(I + M_s G)X = G$

Table 16.3: The Schur weight preserves the shape of the block. That it does so for the middle row is Proposition 16.7.

Proposition 16.7 (The low rank form is closed under the weight). *Let $G = \delta I + UTU^\top$ and let $M_s = \mu_s I$ be a multiple of the identity on the block. Then $\Sigma = G(I + M_s G)^{-1}$ is again of the form $\delta' I + UT'U^\top$ with the same columns U , and*

$$\delta' = \frac{\delta}{1 + \mu_s \delta}. \quad (16.2)$$

Proof. Write $\sigma = 1 + \mu_s \delta$. Then $I + M_s G = \sigma I + \mu_s UTU^\top$, and by the Sherman and Morrison and Woodbury identity its inverse is

$$\frac{1}{\sigma} I - \frac{1}{\sigma^2} U \left(I + \frac{\mu_s}{\sigma} T U^\top U \right)^{-1} \mu_s T U^\top,$$

which is a multiple of the identity plus a correction whose columns are again U . Multiplying on the left by $G = \delta I + UTU^\top$, every term is either a multiple of the identity or has U on the left and $U^\top$ on the right, so the product has the stated form, and the coefficient of the identity is δ/σ . $\square$

This is why the metric must be uniform on a curved block, and the code says so where the requirement bites: `add_penalty` raises if a diagonal plus low rank block is handed a penalty that varies across its rows, because the scalar δ cannot absorb it. Admissibility guarantees the uniformity, and the proposition is what turns that guarantee into a closed representation.

16.4.2 The reduction

The elimination itself is where the road was once wrong, and where the identity that fixed it lives.

Algorithm 16.3 `reduce`, one solve of the Schur road

Input: the three residuals, the stored weight Σ and penalty M_s

Output: $(\Delta x, \Delta s, \Delta y)$

- 1: $t \leftarrow r_{\text{eq}} - M_s \varrho_s$
 - 2: $\text{rhs} \leftarrow -r_{\text{stat}}^x + E^\top (\Sigma t + \varrho_s)$
 - 3: solve $S \Delta x = \text{rhs}$ $\triangleright$ the only factorised solve
 - 4: $e \leftarrow E \Delta x - t$
 - 5: $\Delta y \leftarrow \Sigma e - \varrho_s$
 - 6: $\Delta s \leftarrow M_s \Sigma e - e$ $\triangleright = -(I - M_s \Sigma) e$
-

Every appearance of G in that algorithm is inside Σ . The last line is the one that matters, and it is the push through identity of the theory part written out: $(I + M_s G)^{-1} = I - M_s \Sigma$. Eliminating Δs the obvious way would ask for G^{-1} instead, and

$$G = \rho_p I + M^{-1} F$$

has an eigenvalue exactly ρ_p wherever F has a zero one, so its condition number grows like $1/(\rho_p \mu)$ and reaches 10^{25} late in a solve. The identity is not an optimisation; it is the difference between a road that converges on curved cones and one that does not.

Refinement, on the full system. The Schur road refines like the other, but against the *unreduced* 3×3 residual rather than against $S\Delta x - \text{rhs}$. The correction is obtained by calling `reduce` again on that residual, so each pass costs one triangular solve and no refactorisation. Refining against the reduced residual instead would measure only the error the reduction did not introduce.

Remark 16.8 (The pattern is a contract, and it is one sided.). `setup` promises that every position a later write can ask for exists, and it keeps that promise by asking each cone for the widest block it will ever emit. A cone that under-reported would produce a `position_of` failure, which is loud. A cone that over-reported would produce a matrix with structural zeros, which is silent and merely slower. Only one of the two mistakes is caught.

Remark 16.9 (Quasi-definiteness is assumed, not verified.). The three by three road never checks the inertia of what it assembled. The property is guaranteed by construction, from $P + \rho I \succ 0$, $G \succ 0$ and $M_s \succ 0$, and by the sign discipline of Proposition 16.5 on the auxiliary variables. If a change broke that discipline the factorisation would still run, because a backend asked for an $LDL^\top$ without pivoting will produce one; it would simply be the wrong one.

Remark 16.10 (The two roads can disagree.). They solve the same system and return directions that agree to the accuracy of the refinement, which is not the same as agreeing exactly. On a problem near the edge of convergence the two can take different numbers of iterations, and one can converge where the other does not.

Chapter 17

Backends

17.1 Where it happens

A backend is a factorisation, and nothing else. The directory is **backend/**, 769 lines over nine files: an interface and a dispatch, five implementations, and the thread control they share. A road holds one backend and calls it; no other part of the solver knows a backend exists.

The interface is deliberately narrow.

call	how often	what it does
capabilities	never, at present	declares what the backend can do
symbolic	once per solve	takes the pattern, orders it, plans the factor
value_count	once	how many values the pattern holds
numeric	every inner iteration	factors the values into that plan
update	never, at present	applies a low rank change to the factor
solve	once or twice per iteration	the triangular solves
set_threads	once	how many threads to ask for

Table 17.1: The backend interface. The split between **symbolic** and **numeric** is Definitions 2.33 and 2.34, and it is the reason a road may factor a hundred times after ordering once.

numeric returns a boolean rather than raising. A factorisation that fails is an ordinary event, handled by the retry loop of Chapter 12, not an error.

Two members of the contract are declared and not yet called. **update** applies a low rank change to an existing factorisation rather than recomputing it, and takes a **Delta** naming the columns and the sign. Only **eigen_dense** implements it, through a rank update of its factorisation; the other four accept the call and return false. It is kept because the place it would pay is identifiable rather than hypothetical: on the Schur road applied to a quadratic program, a row changing activity moves one entry of Σ and so changes S by a rank one term, which an update would absorb in $O(\text{nnz } L)$ against a refactorisation's $n^3/6$.

capabilities is how a caller would find out whether that is available, reporting **supports_update** and the thread count the backend can use. Nothing reads it yet either, for the same reason: with no caller of **update** there is nothing to condition on. The pair is the extension point for a future active set update, left in place and honestly inert.

17.2 The pattern contract

Everything a backend receives is a pattern in compressed sparse column form, and `validate_pattern` checks five conditions before any of them sees it:

- the column starts begin at zero and do not decrease;
- no column is empty, so every diagonal entry is present;
- each column begins with its own diagonal;
- no entry lies above the diagonal, the matrix being stored lower triangular;
- within a column the row indices increase.

The conditions are not arbitrary. Storing only the lower triangle halves the memory and is what every sparse factorisation expects; requiring the diagonal present means no backend has to consider a structurally singular pivot; requiring it first, and the rest sorted, means a backend can locate an entry by a binary search rather than a scan, which is what `position_of` in the road does on every write.

Validation happens once, at `symbolic`. It is a real check rather than an assertion, and it raises with the condition that failed, because a pattern that violates one of them produces not a wrong answer but a segmentation fault inside a library.

17.3 The five backends

name	factorisation	sparse	threads
<code>cholmod</code>	supernodal $LL^\top$ or simplicial $LDL^\top$	yes	from the BLAS
<code>qdlldl</code>	simplicial $LDL^\top$	yes	one
<code>eigen_sparse</code>	SimplicialLDLT	yes	one
<code>lapack_dense</code>	<code>dpotrf</code> or <code>dsytrf</code>	no	from the BLAS
<code>eigen_dense</code>	LDLT	no	from the BLAS

Each is constructed with a flag saying whether the matrix it will receive is positive definite, and two of the five use it.

`cholmod` uses it twice. It sets `supernodal` to `AUTO` when the matrix is definite and to `SIMPLICIAL` when it is not, and it asks for a final $LL^\top$ in the first case and an $LDL^\top$ in the second. This is the mechanism behind the road selection of the previous chapter: the Schur road passes true and may therefore reach the supernodal algorithm of Definition 2.35, while the three by three road passes false and is confined to the simplicial one, its matrix being quasi-definite rather than definite.

`lapack_dense` uses it to choose between `dpotrf`, the dense Cholesky, and `dsytrf`, the dense symmetric indefinite factorisation with Bunch and Kaufman pivoting.

The other three ignore it and always compute an $LDL^\top$, which is correct in both cases and merely leaves something on the table when the matrix is definite.

Why five. They are not alternatives of equal standing. `cholmod` is the default and the only one used in the benchmark. `qdlldl` is vendored, has no dependencies, and exists so that the solver can be built and tested without `SUITESPARSE` present. The two dense backends exist for small problems and for isolating a question: if a result differs between

a sparse and a dense backend, the difference is in the sparse machinery and not in the method. `eigen_sparse` is the reference implementation against which the others were checked.

17.4 Choosing one

`resolve_backend` takes the requested kind and the cones, and when the request is `AUTO` it returns `cholmod`. It does not look at the cones; the parameter is accepted and discarded, which the code marks explicitly.

That is a deliberate simplification rather than an oversight. The choice that matters was already made, one level up, when the road was selected: the road decides whether the matrix handed down is definite, and that decides whether `cholmod` runs supernodal or simplicial. Choosing a different backend on top of that would be choosing between implementations of the same algorithm, and on the benchmark `cholmod` was the fastest of them in every case.

17.5 Threads

Only the dense kernels are threaded, and the solver does not thread anything itself. `threads.cpp` declares four BLAS entry points as weak symbols,

```
openblas_set_num_threads, MKL_Set_Num_Threads, omp_set_num_threads,
```

together with their query counterparts, and calls whichever the linker actually resolved. If none did, the request is refused and the count reported is one.

Two consequences. A backend that reports one thread is not necessarily single threaded by design; it may be that no threaded BLAS was linked. And the count a backend reports is what it asked for and what some library confirmed, not a measurement, so a build that links a threaded BLAS and then runs under an environment variable capping it will report the request rather than the cap.

Remark 17.1 (A failed factorisation is not an exception.). `numeric` returns false, and the caller raises the regularisation and tries again. The consequence is that a backend must leave itself in a usable state after failing, since it will be called again immediately with different values and the same pattern. All five do, but nothing in the interface says so, and a backend that freed its symbolic analysis on failure would work until the first ill conditioned problem.

Chapter 18

Globalisation

18.1 Where it happens

A globalisation rule answers one question: given a direction, how far along it to move. The directory is `globalisation/`, 370 lines over four files, holding five rules and the merit function three of them share.

rule	used by	decides from
<code>unit_step</code>	neither, by default	nothing
<code>fraction_to_boundary</code>	the barrier road	the cone boundary
<code>exact_search</code>	the projection road, by default	φ'_c
<code>armijo_search</code>	the projection road, optional	φ_c and $\varphi'_c(0)$
<code>decrease_search</code>	the projection road, optional	φ_c alone

Every rule returns a `StepPair`, one length for the primal pair (x, s) and one for the dual pair (y, z) . Only the barrier rule returns two different numbers; the three searches return the same value twice, because the merit they work on is a function of the primal variables and has nothing to say about the dual. The choice of rule is made in `solve.cpp` from the road and, on the projection road, from a setting.

18.2 The merit along the ray

The three searches all work on the same function: the proximal subproblem's own objective φ_c , restricted to the ray. Write

$$\psi(t) = \varphi_c(x + t\Delta x, s + t\Delta s), \quad t \geq 0. \quad (18.1)$$

That choice is not free. The residual norm $\mathcal{N}_c$ is also a merit for the same system, and searching on it instead was measured to be worse.

18.2.1 What can be asked of it, and at what price

`Merit` exposes exactly two queries, and the split between them is what separates the three searches.

call	returns	cost
<code>slope(t)</code>	$\psi'(t)$	one projection of the slack
<code>decrease(t)</code>	$\psi(t) - \psi(0)$	one projection of the slack

Both are cheap for the same reason. Everything in φ_c except the cone term is quadratic along the ray, so its contribution is a constant and a curvature, both computed once by `bind`. Only the cone term has to be recomputed at each trial:

$$\psi'(t) = c + \kappa t - \langle \Pi_{K^*}(\sigma - t\Delta s), \Delta s/\mu \rangle, \quad \sigma = Mz_c - s. \quad (18.2)$$

A trial is therefore one projection and two inner products, against a Newton step that is a factorisation. This is why a few dozen trials are affordable.

The two coefficients are

$$c = \langle Px + q, \Delta x \rangle + \rho \langle x - x_c, \Delta x \rangle + \rho_p \langle s - s_c, \Delta s \rangle + \langle \nu, E\Delta x + \Delta s \rangle, \quad (18.3)$$

$$\kappa = \langle \Delta x, (P + \rho I)\Delta x \rangle + \rho_p \|\Delta s\|^2 + \|E\Delta x + \Delta s\|_{M_s^{-1}}^2. \quad (18.4)$$

18.2.2 Why a bracket exists

The exact search looks for a zero of ψ' rather than a decrease in ψ , and that is only sensible because ψ' is increasing. It is, and the same fact supplies the bracket the search needs.

Proposition 18.1 (The restriction is strongly convex). *Let the direction $(\Delta x, \Delta s)$ be nonzero and let κ be given by (18.4). Then $\kappa > 0$ and*

$$\psi''(t) \geq \kappa \quad \text{for every } t \geq 0, \quad (18.5)$$

so ψ is strongly convex along the ray and ψ' is strictly increasing.

Proof. Every term of (18.4) is nonnegative, and $\rho \|\Delta x\|^2 + \rho_p \|\Delta s\|^2 > 0$ for a nonzero direction since $\rho, \rho_p > 0$, so $\kappa > 0$.

Differentiating (18.2) once more, the quadratic part contributes κ and the cone term contributes

$$\frac{d}{dt} \left(-\langle \Pi_{K^*}(\sigma - t\Delta s), \Delta s/\mu \rangle \right) = \langle F\Delta s, \Delta s/\mu \rangle = \langle \Delta s, M^{-1}F\Delta s \rangle \geq 0,$$

using $\frac{d}{dt} \Pi_{K^*}(\sigma - t\Delta s) = -F\Delta s$ and the fact that $M^{-1}F$ is positive semidefinite, F being so and commuting with M^{-1} by admissibility. Hence $\psi'' \geq \kappa$. $\square$

Proposition 18.2 (A bracket for the minimiser). *Suppose $\psi'(0) < 0$. Then ψ has a unique minimiser $t^* > 0$ on the ray, and*

$$0 < t^* \leq \frac{|\psi'(0)|}{\kappa}. \quad (18.6)$$

In particular $[0, |\psi'(0)|/\kappa]$ brackets the zero of ψ' .

Proof. By Proposition 18.1, integrating $\psi'' \geq \kappa$ from 0 to t gives

$$\psi'(t) \geq \psi'(0) + \kappa t.$$

At $t = |\psi'(0)|/\kappa = -\psi'(0)/\kappa$ the right side is zero, so ψ' is nonnegative there. Since ψ' is strictly increasing and negative at 0, it has exactly one zero, and that zero lies in $(0, |\psi'(0)|/\kappa]$. Strong convexity makes it the unique minimiser. $\square$

The bound is exactly what the implementation uses. It is not a guess at a large enough step but the point at which a quadratic with the observed slope and the guaranteed curvature would turn, and the true ψ turns no later because its curvature is at least κ everywhere.

18.3 Fraction to the boundary

The rule for the barrier road, which is Definition 7.37. The implementation adds one thing the definition does not: the fraction is not a single constant.

$$\theta = \begin{cases} \text{tau} & \text{on a polyhedral block,} \\ \text{tau_curved} & \text{on a curved block,} \end{cases}$$

with the second the smaller of the two by a wide margin, and the step is the smallest over the blocks, capped at one, with the primal and dual computed separately from $(s, \Delta s)$ and $(z, \Delta z)$.

The two values differ by a great deal, and the reason is not accuracy: the largest feasible step is exact on the second order and semidefinite cones, by a quadratic root and by an eigenvalue. It is that the conditioning degrades much faster near a curved boundary. On the orthant, approaching it makes one ratio z_i/s_i large; on a curved block the whole Nesterov and Todd scaling degenerates at once, and the next factorisation pays for it. Twenty percent of margin is the price of keeping the block conditioned.

Blocks with no interior are skipped, having no boundary to respect.

18.4 The exact search

The default on the projection road. It finds the zero of ψ' inside the bracket of Proposition 18.2.

Algorithm 18.1 exact_search

Input: the direction, bound into the merit

```

1:  $g_0 \leftarrow \text{slope}(0)$ 
2: if  $g_0 \geq 0$  then
3:   return 1 ▷ not a descent direction; there is nothing to search
4: end if
5:  $a \leftarrow 0, g_a \leftarrow g_0, b \leftarrow 1, g_b \leftarrow \text{slope}(1)$ 
6: if  $g_b < 0$  then ▷ the unit step does not yet reach the minimiser
7:    $a \leftarrow b, g_a \leftarrow g_b$ 
8:    $b \leftarrow |g_0|/\kappa$  ▷ Proposition 18.2; the minimiser is at or before this
9:   if  $b \leq a$  then
10:    return  $a$ 
11:   end if
12:    $g_b \leftarrow \text{slope}(b)$ 
13: end if
14: if  $g_a \leq 0 \leq g_b$  then
15:   for at most ls_max_secant trials do
16:     stop once  $b - a$  is below  $\sqrt{\varepsilon_{\text{mach}}}$  relative to  $b$ 
17:      $t \leftarrow$  the secant point of  $(a, g_a)$  and  $(b, g_b)$ , or the midpoint if that falls outside
18:     replace whichever end shares the sign of  $\text{slope}(t)$ 
19:     halve the slope kept at the other end if that end has now been kept twice
    running ▷ the Illinois correction
20:   end for
21: end if
22: return  $\min(b, t_{\text{max}})$  ▷  $t_{\text{max}}$  a fixed cap on the returned step

```

Three points about it.

The bracket is established rather than assumed. If the unit step still has a negative slope the search does not simply try larger numbers; it jumps to the bound of Proposition 18.2, which is guaranteed to lie at or beyond the minimiser, so one extra slope evaluation closes the bracket for good.

The iteration is the secant method with the Illinois correction. Plain regula falsi keeps one endpoint fixed whenever the function is convex, which it is here, and then converges linearly; halving the retained slope after two consecutive retentions of the same end restores the superlinear rate.

And what is returned is b , the end on which the slope is nonnegative, not the midpoint or the last trial. The step therefore never overshoots the minimiser, which matters because ψ is only a model of the merit the outer loop cares about.

18.5 The Armijo search

Backtracking from the unit step, accepting the first t with

$$\psi(t) - \psi(0) \leq \sigma_{\text{ls}} t \psi'(0), \quad \sigma_{\text{ls}} = \text{ls_sigma}. \quad (18.7)$$

This is the standard sufficient decrease: it asks for a decrease proportional to what the slope at zero promised, rather than merely for a decrease, and it is the condition under which a convergence proof for a backtracking method is available. It needs $\psi'(0)$, one extra query before the loop, and returns the unit step immediately if that slope is not negative.

Algorithm 18.2 `armijo_search`

```

1:  $g_0 \leftarrow \text{slope}(0)$ ; if  $g_0 \geq 0$  then return 1
2:  $t \leftarrow 1$ 
3: for at most ls_max_back halvings do
4:   if  $\text{decrease}(t) \leq \sigma_{\text{ls}} t g_0$  then
5:     return  $t$ 
6:   end if
7:    $t \leftarrow t/2$ 
8: end for
9: return  $t$ 

```

18.6 The decrease search

The same loop with the condition weakened to

$$\psi(t) - \psi(0) < 0, \quad (18.8)$$

which proves nothing but costs one query fewer per call, needing no slope at all. On a strongly convex ψ along a descent direction, every sufficiently small step satisfies it, so the loop terminates; what it does not guarantee is that the accepted step makes progress proportional to the slope, which is exactly what Armijo buys.

Both backtracking rules give up after `ls_max_back` halvings and return the last step tried rather than failing. A step that small is not useful, but returning it lets the outer loop observe a stalled iteration and react through the schedule, which is a better outcome than an exception.

18.7 The unit step

Returns one for both pairs whatever the direction. It is the default branch of the dispatch and no road selects it. It exists as the null rule against which the others are compared: running the projection road with it is how the value of a line search was measured in the first place.

Remark 18.3 (The searches never look at the dual direction.). All three take Δx and Δs , ignore Δz , and return the same length for both pairs. That is consistent, φ_c being a function of the primal variables alone, but it means the dual step is chosen by a rule with no information about the dual residual. On the barrier road the two lengths are genuinely separate; on the projection road they are equal by omission rather than by design.

Remark 18.4 (The bracket bound depends on the penalties.). The bound of Proposition 18.2 is $|\psi'(0)|/\kappa$, and κ contains $\rho \|\Delta x\|^2 + \rho_p \|\Delta s\|^2$. As the schedule drives those weights down the bound grows, so late in a solve the search may have to look much further out than the unit step before it brackets. That is not a defect of the bound, which stays valid, but it does mean the number of slope evaluations rises as the penalties fall.

Chapter 19

The parameter schedule

19.1 Where it happens

The schedule owns everything the outer loop changes between subproblems: the four centres, the proximal weights ρ and ρ_p , the two penalty vectors, and the tolerance the inner loop stops against. It is `schedule/`, 407 lines over two files, and it holds three implementations behind one call.

```
update(iterate, violation, reference, primal_residual, dual_residual,
       inner_converged, inner_taken)
```

It is called once per outer iteration, at the very end, after the termination test and the infeasibility tests have both declined to stop. It returns nothing; everything it does is written into the iterate and into its own tolerance vector.

schedule	road	penalties are
<code>bcl</code>	projection	one value for all rows
<code>gbcl</code>	projection, by default	one value per row, per block where curved
<code>interior</code>	barrier	not used; ρ and ρ_p only

No convergence result covers any of the three as implemented. The theory part gives conditions on the metrics under which the proximal iteration converges, summable tolerances among them, and the schedules below satisfy none of them exactly. What follows is what they do, and why, not a proof that it works.

Remark 19.1 (Penalty). The element of M are called "penalties", the term is misleading here given that when penalties get smaller constraint violation are penalize more i.e they contribute more to the objective of the smoothed problem.

19.2 What the schedule is trying to balance

Every subproblem is easier when the penalties are large, because the cone term is then weakly weighted and the Newton system is well conditioned; and every subproblem is a worse approximation of the original problem for the same reason. The schedule moves the penalties down when the iterate is behaving and holds them when it is not.

The quantity it watches is the *violation*, one number per row, measuring how far that row is from satisfying its own cone constraint, and the *tolerance* it is compared against,

also one number per row. Those two vectors are the schedule's state; everything else is derived from them.

19.3 The parameters, once

No number appears in the algorithms below that is not a setting. The two projection schedules do not share a parameter block: each names its own, and they are tabulated where each schedule is described. The pair μ_{restart} and μ_{th} belongs to **bcl** alone. The defaults are the subject of a later chapter; what matters here is which quantity each one controls.

symbol	setting	controls, bcl
α	alpha	how fast the tolerance loosens on a reject
β	beta	how fast it tightens on an accept
κ	mu_update_factor	the factor a rejected penalty is cut by
$\mu_{\min}$	mu_min_in	the floor under every penalty
N_{guard}	safe_guard	after how many outers an accept is forced
$\mu_{\text{restart}}, \mu_{\text{th}}$	cold_reset*	the restart and its trigger

Two derived quantities are fixed at setup and used throughout by **bcl**:

$$\bar{w}_0 = (0.1)^\alpha, \quad \varepsilon_{\text{in}}^{\min} = \min(\varepsilon_{\text{abs}}, 10^{-9}), \quad (19.1)$$

the first the tolerance a rejected outer iteration is reset from, the second the floor under the inner tolerance, so that a schedule cannot ask the inner loop for an accuracy the outer test would not itself demand.

19.4 **bcl**, one penalty for the whole problem

The classical rule, and here deliberately a faithful one: it is PROXQP's **bcl_update** written in this solver's variables, kept as a reference against which the generalisation is measured rather than tuned on its own account. A single scalar decides for every row at once, and the decision is taken on the *primal residual* of the original problem, not on the violation vector.

Algorithm 19.1 **bcl**, one outer update

Input: the primal and dual residuals

```

1:  $\mu \leftarrow \max_i(\text{penalty})_i$ 
2: if primal residual  $\leq \bar{w}$  or taken  $> N_{\text{guard}}$  then ▷ accept
3:   move all four centres onto the current iterate
4:    $\bar{w} \leftarrow \bar{w} \mu^\beta, \quad \varepsilon_{\text{in}} \leftarrow \max(\varepsilon_{\text{in}} \mu, \varepsilon_{\text{in}}^{\min})$ 
5: else ▷ reject
6:   restore  $y$  and  $z$  to their values before the subproblem
7:    $\mu \leftarrow \max(\mu_{\min}, \kappa \mu)$  on every row
8:    $\bar{w} \leftarrow \bar{w}_0 \mu^\alpha, \quad \varepsilon_{\text{in}} \leftarrow \max(\mu, \varepsilon_{\text{in}}^{\min})$ 
9: end if
10: if neither residual improved and  $\mu \leq \mu_{\text{th}}$  then
11:   reset every penalty to  $\mu_{\text{restart}}$ 
12: end if
```

19.5 gbcl, one penalty per row

The default, and the generalisation: BCL's decision taken for each constraint separately rather than for the problem as a whole. It carries three tolerances rather than one. A bar per row, $\bar{w}_i$, decides which rows may move their centres and how hard each penalty is pushed. A scalar bar, b , decides when the proximal centre follows the iterate. A Newton tolerance, ε_N , stops the inner loop. The three are scheduled separately, and only the first is what BCL calls the bar.

symbol	setting	controls
e	<code>mu_exp</code>	how hard a miss is punished
κ	<code>penalty_reduction</code>	the hardest push allowed, and where it starts
a	<code>mu_adapt</code>	how fast that push is retuned
θ	<code>penalty_reduction</code>	how hard a failed inner loop raises a penalty
α	<code>alpha</code>	how fast a missed bar loosens
β	<code>beta</code>	how fast a met bar tightens
μ_0	<code>mu_in</code>	the first penalty, and the ceiling over every later one
μ_{eq}	<code>mu_eq</code>	the same, for the equality penalties
μ_{min}	<code>mu_min</code>	the floor under every penalty
$\bar{w}_0$	<code>eps_outer_init</code>	the bar every row starts at
$\varepsilon_{N,0}$	<code>eps_newton_init</code>	the Newton tolerance every outer starts at
ϱ_p	<code>rho_p_outer</code>	how fast the proximal slack falls
N	<code>safe_guard</code>	how many outers the proximal centre may stand still for
ε	<code>eps_abs</code>	the floor under the bars
$\bar{w}_{min}$	<code>smallest_tolerance</code>	the floor under the Newton tolerance

Two floors sit under the tolerances and they are deliberately different. The bars stop at the accuracy the solve is asked for, since a row held tighter than that buys nothing. The Newton tolerance stops far below it, because the inner residual is not the quantity being certified: the subproblem has to be driven well past the target for the residual that is certified to reach it. A Newton tolerance floored at ε instead lets the inner loop declare an iterate solved that the outer loop cannot improve, and the outer loop then repeats it unchanged.

Algorithm 19.2 κ , retuned on the observed efficiency

Input: the primal residual and the number of inner steps taken

- 1: **if** an earlier outer exists **and** at least one inner step was taken **then**
 - 2: $\eta \leftarrow \log_{10}(\text{previous residual}/\text{residual})/\text{inner steps taken}$ $\triangleright$ digits per step
 - 3: **if** η fell short of the previous η **then**
 - 4: reverse the direction
 - 5: **end if**
 - 6: $\kappa \leftarrow \kappa a^{\text{direction}}$, kept clear of zero and of one
 - 7: **end if**
 - 8: keep the residual as the previous one
-

Algorithm 19.3 gbc1, one outer update

Input: the violation per row, the primal residual, whether the inner loop converged, the number of inner steps taken

```

1: for every row  $i$  with  $\text{violation}_i \leq \bar{w}_i$  do
2:   move that row's slack, dual and equality centres
3: end for
4: if the inner loop did not converge then
5:    $\mu_{\text{floor}} \leftarrow \max(\mu_{\text{floor}}, \min(\mu_{\text{max}}/\theta, \mu_0))$  ▷ ratcheted, never undone
6:   for every row  $i$  with  $\text{violation}_i > \bar{w}_i$  do
7:      $\mu_i \leftarrow \min(\mu_i/\theta, \mu_0), \quad \mu_{s,i} \leftarrow \min(\mu_{s,i}/\theta, \mu_{\text{eq}})$ 
8:   end for
9: else
10:  retune  $\kappa$ 
11:  for every row  $i$  do
12:     $f_i \leftarrow \min(1, \max(\kappa, (\text{violation}_i/\bar{w}_i)^{-e}))$ 
13:  end for
14:  flatten  $f$  on every curved block, by the minimum ▷ admissibility
15:   $\mu \leftarrow \max(\max(\mu_{\text{floor}}, \mu_{\text{min}}), \mu \odot f), \quad \mu_s \leftarrow \max(\mu_{\text{min}}, \mu_s \odot f)$ 
16: end if
17: for every row  $i$  do
18:    $\bar{w}_i \leftarrow \begin{cases} \max(\bar{w}_i \mu_i^\beta, \varepsilon) & \text{if } \text{violation}_i \leq \bar{w}_i, \\ \bar{w}_0 \mu_i^\alpha & \text{otherwise} \end{cases}$  ▷ the bar loosens on a miss
19: end for
20:  $\mu_{\text{max}} \leftarrow \max_i \mu_i$ 
21: if the primal residual met  $b$  or  $N$  outers have passed then
22:   move the proximal centre onto the iterate
23:    $\varepsilon_N \leftarrow \max(\varepsilon_N \mu_{\text{max}}, \bar{w}_{\text{min}}), \quad b \leftarrow \max(b \mu_{\text{max}}^\beta, \varepsilon)$ 
24: else
25:    $\varepsilon_N \leftarrow \max(\varepsilon_N \mu_{\text{max}}, \bar{w}_{\text{min}})$  if no inner step was taken, otherwise  $\varepsilon_{N,0} \mu_{\text{max}}$ 
26:    $b \leftarrow \bar{w}_0 \mu_{\text{max}}^\alpha$ 
27: end if
28:  $\rho_p \leftarrow \max(\rho_{p,\text{min}}, \varrho_p \rho_p)$ 

```

Three things separate this from BCL beyond the per row decision. A failed inner loop raises the penalties of the rows that missed, rather than leaving them, and ratchets a floor under every penalty that no later reduction may cross; the floor is what stops a penalty from being driven back down into the region that produced the failure. The push κ is not a constant: each converged outer measures how many digits of violation the inner loop bought per step, and moves κ in whichever direction that measurement last improved. And a row that misses its bar has the bar loosened rather than tightened, so a row that cannot be met does not drag the whole schedule to a floor it will never reach.

The proximal centre is on its own clock. It follows the iterate only when the primal residual meets the scalar bar b , or when N outers have passed without that happening, which bounds how long the centre may stand still. Between those moves the Newton tolerance is reset from $\varepsilon_{N,0}$, except when the previous inner loop took no step at all: there the tolerance is tightened instead, since resetting it would ask for work already done.

19.6 interior, for the barrier road

The barrier road has no penalties to schedule, its cone term being carried by τ and the centring parameter, which the smoothing manages itself. What remains is ρ and ρ_p , and the schedule treats them separately, each with its own stall counter.

Its parameters are its own, shared with neither projection schedule.

symbol	setting	controls
ι	improvement	how much a residual must fall to count as progress
ς	stalled_rate	how much of the rate a stalled iteration still gets
φ_{coarse}	reg_floor	the floor under ρ and ρ_p at first
φ_{fine}	reg_fine	the floor after the ratchet
N_{stall}	stall_before_fine	how many stalls at the floor before it drops
N_{early}	early_outers	how long a stalled iteration is still indulged
β_{prox}	infeasibility_threshold	the bar the proximal contribution is judged against
ε_{reg}	eps_reg	below which a residual counts as converged outright

Algorithm 19.4 interior, one outer update

- 1: $r \leftarrow$ the relative decrease of the duality measure
 - 2: $\text{dual_prox} \leftarrow \rho \|x - x_c\|_\infty$, $\text{primal_prox} \leftarrow \rho_p \|s - s_c\|_\infty$ $\triangleright$ how much of each residual the proximal term itself contributes
 - 3: **if** dual residual $< \iota \cdot$ previous, or it is already below ε_{reg} , or the floor is fine and $\text{dual_prox} < \beta_{\text{prox}}$ **then**
 - 4: move the x centre, $\rho \leftarrow \max(\varphi, (1 - r)\rho)$ $\triangleright \varphi$ the current floor
 - 5: **else**
 - 6: count a dual stall; reduce ρ only at the damped rate $1 - \varsigma r$, and only while taken $< N_{\text{early}}$ or $\text{dual_prox} < \beta_{\text{prox}}$
 - 7: **end if**
 - 8: the same, symmetrically, for the primal residual, ρ_p and the slack and dual centres
 - 9: **if** either has stalled more than N_{stall} times at its floor, and both proximal parts are below β_{prox} **then**
 - 10: lower the floor from φ_{coarse} to φ_{fine}
 - 11: **end if**
-

Chapter 20

The cone kernel

20.1 Where it happens

The kernel is the largest directory in the solver, 1957 lines over ten files, because it carries six independent cones behind one interface. The interface is a set of free functions in `cone/kernel.hpp`, dispatched on the cone type by a variant visit, and every cone provides the same twenty. Adding a cone is implementing those twenty; nothing else changes.

function	what it computes
<code>rows, degree</code>	the block size and the barrier parameter ν
<code>is_curved</code>	whether admissibility forces $M = cI$ on the block
<code>has_interior</code>	whether $\text{int } K \neq \emptyset$
<code>contains, margin</code>	membership, and Definition 14.3
<code>evaluate_projection</code>	$\Pi_{K^*}(v)$
<code>build_projection_scaling</code>	the cache F is read from, at σ
<code>build_barrier_scaling</code>	the scaling point, at (s, z)
<code>materialise_operator</code>	the block of G_ϕ
<code>widest_operator_kind</code>	the widest block shape the cone will return
<code>apply_residual_scaling</code>	F , or its barrier analogue, applied
<code>complementarity_residual</code>	the centrality residual
<code>second_order_correction</code>	the Mehrotra corrector
<code>largest_feasible_step</code>	the $\bar{\alpha}$ of Definition 7.37
<code>interior_direction</code>	a point of $\text{int } K$, for the cold start

Table 20.1: The kernel interface. A cone that cannot support a function raises, and the solver never reaches it.

cone	K^*	symmetric	ν	curved	G block
<code>Zero</code>	$\mathbb{R}^d$	no	0	no	diagonal
<code>Nonneg</code>	itself	yes	d	no	diagonal
<code>SecondOrder</code>	itself	yes	2	yes	diagonal, rank two
<code>PSDTriangle</code>	itself	yes	k	yes	dense
<code>Exponential</code>	not itself	no	3	yes	dense
<code>Power</code>	not itself	no	3	yes	dense

Each cone below is given in the same order: membership, projection, the selected derivative, the barrier scaling, the cone operator, the boundary step, and cost.

20.2 The zero cone

$K = \{0\} \subseteq \mathbb{R}^d$ and $K^* = \mathbb{R}^d$, so the block is an equality constraint. It is closed and convex, not solid, and not self dual.

The barrier is $B \equiv 0$ with $\nu = 0$, as fixed in the theory part: the relative boundary of $\{0\}$ is empty, so every condition holds vacuously. A zero block contributes nothing to τ and has no centrality condition.

The projection is the identity, $F = I$ at every point, and

$$G = \rho_p I + M^{-1} = \rho_p + \frac{1}{\mu},$$

diagonal and independent of the iterate. There is no case analysis and no algorithm to give.

The implementation declines the barrier road here, and the restriction is its own rather than the theory's: the machinery wants a strictly feasible point, a scaling built from it and a fraction to the boundary, none of which exists when $\text{int } K = \emptyset$. Since `has_interior` returns false, the interior road routes such a block through the equality penalty, exactly as the projection road does.

20.3 The nonnegative orthant

$K = K^* = \mathbb{R}_+^d$, symmetric of rank d , with $\nu = d$; the canonical inner product is the Euclidean one, so the constant C of the centrality discussion is one. The barrier is $B(s) = -\sum_i \log s_i$, giving $\nabla B = -s^{-1}$ and $\nabla^2 B = \text{diag}(s^{-2})$, elementwise.

Everything on this cone is elementwise, and the formula is the algorithm:

$$\Pi_{K^*}(v) = \max(v, 0), \quad F = \text{diag}(\mathbf{1}_{\sigma > 0}), \quad G = \rho_p + \frac{\mathbf{1}_{\sigma > 0}}{\mu}.$$

The Jacobian is a 0/1 mask, and it is idempotent, $F^2 = F$, which the orthant shares with no other cone in the kernel. An inactive row contributes ρ_p alone, which is how the projection road identifies the active set: a row whose σ is negative drops out of the system entirely.

The barrier scaling is closed form, two square roots per row,

$$w = \sqrt{s/z}, \quad \lambda = \sqrt{s \circ z}, \quad G = \rho_p + \frac{z}{s},$$

The centrality residual is $\lambda \circ \lambda - \tau \mathbf{1}$, so $\langle s, z \rangle = \sum_i \lambda_i^2 = d\tau = \nu\tau$; the corrector is $\Delta s \circ \Delta z$, which is exactly the omitted quadratic term rather than an estimate of it. The step is $\min\{-v_i/\Delta v_i : \Delta v_i < 0\}$ and the margin is $\min_i v_i$, both $O(d)$.

20.4 The second order cone

$$\mathcal{Q}^d = \{(t, u) \in \mathbb{R} \times \mathbb{R}^{d-1} : \|u\| \leq t\},$$

self dual and symmetric, with Jordan product $x \circ y = (\langle x, y \rangle, x_0 y_1 + y_0 x_1)$, identity $e = (1, 0, \dots, 0)$, determinant $\det v = t^2 - \|u\|^2$ and rank two. The barrier is $B = -\log(t^2 - \|u\|^2)$ with $\nabla_{\text{Euc}} B = -2v^{-1}$ and $\nu = 2$; the canonical trace inner product is twice the Euclidean one, so $C = 2$ here against $C = 1$ on the orthant and the semidefinite cone.

Proposition 20.1 (Projection onto the second order cone). *Let $v = (t, u)$ and $K = \mathcal{Q}^d$. Then*

$$\Pi_K(v) = \begin{cases} v & \|u\| \leq t, \\ 0 & \|u\| \leq -t, \\ c(\|u\|, u) & \text{otherwise,} \end{cases} \quad c = \frac{1}{2} \left(1 + \frac{t}{\|u\|}\right) \in (0, 1). \quad (20.1)$$

Proof. The first two cases are $v \in K$ and $v \in K^\circ = -K$. Otherwise $|t| < \|u\|$, so $c \in (0, 1)$ and $p = c(\|u\|, u)$ lies on the boundary, its apex equalling its tail norm. Moreau asks three things. First $p \in K$, with equality. Second $v - p \in K^\circ$: from $c\|u\| = \frac{1}{2}(\|u\| + t)$,

$$t - c\|u\| = -\frac{1}{2}(\|u\| - t), \quad (1 - c)\|u\| = \frac{1}{2}(\|u\| - t),$$

so the apex of $v - p$ is the negative of its tail norm. Third $\langle p, v - p \rangle = c\|u\|(t + \|u\| - 2c\|u\|) = 0$. Uniqueness completes the proof. $\square$

Algorithm 20.1 Projection onto the second order cone

Input: $v = (t, u)$
1: $n \leftarrow \|u\|$
2: **if** $n \leq t$ **then**
3: **return** v $\triangleright$ already in the cone
4: **else if** $n \leq -t$ **then**
5: **return** 0 $\triangleright$ in the polar cone
6: **else**
7: $c \leftarrow \frac{1}{2}(1 + t/n)$
8: **return** (cn, cu)
9: **end if**

The tests use $\leq$, so the three regions meet exactly on the boundary and no point falls between them.

Proposition 20.2 (The Jacobian is a rank two correction). *In the boundary case, with $\hat{u} = u/\|u\|$, $\hat{e} = (0, \hat{u})$ and e_0 the apex direction,*

$$F = cI + [\hat{e} \ e_0] T [\hat{e} \ e_0]^\top, \quad T = \begin{pmatrix} -\frac{1}{2}t/\|u\| & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} - c \end{pmatrix}. \quad (20.2)$$

Proof. In that region c is smooth and $\nabla_u \|u\| = \hat{u}$, so

$$\nabla c = \frac{1}{2\|u\|} \left(1, -\frac{t}{\|u\|} \hat{u}\right).$$

Differentiating the tail cu gives $cI_{d-1} + u(\nabla c)^\top$, rank one along $\hat{u}$ in its second term; differentiating the apex $c\|u\|$ gives $\|u\|(\nabla c)^\top + c\hat{u}^\top$, one further row. Both lie in the span of $\hat{e}$ and e_0 , and matching coefficients gives T . $\square$

Away from the boundary the Jacobian is not that at all, and the code distinguishes four regions rather than three. The extra one is numerical: when u vanishes to machine precision the boundary formula would divide by $\|u\|$, so that case is decided by the sign of t alone.

Algorithm 20.2 Applying $F \in \partial\Pi_{\mathcal{Q}^d}(\cdot)(\sigma)$

Input: $\sigma = (t, u)$ and a vector $v = (v_0, v_1)$

```

1:  $n \leftarrow \|u\|$ 
2: if  $n \leq \varepsilon_{\text{tiny}}$  then
3:   return  $v$  if  $t \geq 0$ , else 0 ▷  $u$  vanishes; the tail carries no direction
4: else if  $n < t$  then
5:   return  $v$  ▷ interior,  $F = I$ 
6: else if  $n < -t$  then
7:   return 0 ▷ polar interior,  $F = 0$ 
8: else
9:    $c \leftarrow \frac{1}{2}(1 + t/n), \quad \hat{u} \leftarrow u/n$ 
10:   $\pi \leftarrow \langle \hat{u}, v_1 \rangle, \quad \kappa \leftarrow \frac{1}{2}(v_0 - (t/n)\pi)/n$ 
11:  return  $(\frac{1}{2}\pi + \frac{1}{2}v_0, \quad c v_1 + \kappa n \hat{u})$ 
12: end if

```

The block $G = \rho_p I + M^{-1}F$ inherits those four regions. In three of them F is a multiple of the identity and the block is a plain diagonal shift, $\rho_p + 1/\mu$ inside the cone and ρ_p inside the polar cone; only in the boundary region does it become diagonal plus rank two. This is why `widest_operator_kind` answers `DiagPlusLowRank` for this cone although three of its four regions produce a diagonal: the pattern must accommodate the widest case.

Algorithm 20.3 Nesterov and Todd scaling for the second order cone

Input: $s, z \in \text{int } \mathcal{Q}^d$

```

1:  $\rho_s \leftarrow \sqrt{\det s}, \quad \rho_z \leftarrow \sqrt{\det z}$ 
2:  $\hat{s} \leftarrow s/\rho_s, \quad \hat{z} \leftarrow z/\rho_z$  ▷ both now of unit determinant
3:  $\gamma \leftarrow \sqrt{(1 + \langle \hat{s}, \hat{z} \rangle)/2}$ 
4:  $w_0 \leftarrow (\hat{z}_0 + \hat{s}_0)/(2\gamma), \quad w_1 \leftarrow (\hat{z}_1 - \hat{s}_1)/(2\gamma)$  ▷ sum on the apex, difference on the tail
5:  $f \leftarrow \sqrt{\rho_z/\rho_s}, \quad \lambda \leftarrow f W^{1/2} s$  ▷ applied by the arrow formula,  $O(d)$ 

```

Reflecting the tail of w gives $W^{-1/2}$, which is why the corrector reflects rather than inverting anything. On the barrier road, with $J = 2e_0 e_0^\top - I$,

$$G = \rho_p I + f^2(2ww^\top - J) = (\rho_p + f^2)I + 2f^2ww^\top - 2f^2e_0e_0^\top,$$

and here there is no case analysis, the barrier road visiting only the interior.

The centrality residual is $\lambda \circ \lambda - 2\tau e$, the factor two being $C = 2$; the corrector is the Jordan product of the scaled steps. The step is the largest α with $\det(v + \alpha\Delta v) \geq 0$ and $v_0 + \alpha\Delta v_0 \geq 0$, and since the determinant is a quadratic in α this is one discriminant and a choice between two roots, $O(d)$. The margin is $v_0 - \|v_1\|$ and the interior direction is e_0 .

20.5 The semidefinite cone

The cone is $\mathcal{S}_+^k$, self dual and symmetric of rank k , so $\nu = k$, carried in a packed form of $d = k(k+1)/2$ rows. The packing is not the naive one.

Proposition 20.3 (*svec* is an isometry). *Let svec take the lower triangle in column order, multiplying every off diagonal entry by $\sqrt{2}$ and leaving the diagonal alone. Then $\langle \text{svec } X, \text{svec } Y \rangle = \text{tr}(XY)$ for all $X, Y \in \mathcal{S}^k$, and svec carries $\mathcal{S}_+^k$ onto a closed convex self dual cone.*

Proof. Each off diagonal pair is counted twice in the trace, so

$$\text{tr}(XY) = \sum_i X_{ii}Y_{ii} + 2 \sum_{i>j} X_{ij}Y_{ij}$$

. The packed vectors contribute the same diagonal sum and

$$\sum_{i>j} (\sqrt{2}X_{ij})(\sqrt{2}Y_{ij}) = 2 \sum_{i>j} X_{ij}Y_{ij}$$

from the rest. Self duality is inherited, a linear isometry carrying a self dual cone to a self dual cone. $\square$

Without the $\sqrt{2}$ the map is still a bijection but no longer preserves inner products, and every pairing in the solver, the duality gap among them, would be wrong on semidefinite blocks by a factor depending on how much mass sits off the diagonal. The scaling appears in two functions only, `matrix_from_svec` and `svec_from_matrix`.

The barrier is $B(X) = -\log \det X$ with $\nabla B = -X^{-1}$, and $\det(\lambda X) = \lambda^k \det X$ gives $\nu = k$; the canonical inner product is Frobenius, so $C = 1$.

Proposition 20.4 (Projection onto the semidefinite cone). *For $X = U\Lambda U^\top$, one has $\Pi_{\mathcal{S}_+^k}(X) = U \max(\Lambda, 0) U^\top$.*

Proof. Set $P = U \max(\Lambda, 0) U^\top \succeq 0$. Then $X - P = U \min(\Lambda, 0) U^\top$ lies in $-\mathcal{S}_+^k$, and their product is $U \max(\Lambda, 0) \min(\Lambda, 0) U^\top = 0$, the two diagonal factors having disjoint support, so $\langle P, X - P \rangle = 0$. Moreau's characterisation applies. $\square$

Algorithm 20.4 Projection onto the semidefinite cone

Input: the packed v

- 1: $X \leftarrow \text{smat}(v)$ $\triangleright$ unpack, dividing off diagonals by $\sqrt{2}$
 - 2: $(U, \Lambda) \leftarrow$ symmetric eigendecomposition of X $\triangleright O(k^3)$
 - 3: **return** $\text{svec}(U \max(\Lambda, 0) U^\top)$
-

There is no case analysis: the clamp handles every sign pattern uniformly, and the interior, boundary and polar cases of the other cones appear here only as the number of nonpositive eigenvalues.

Proposition 20.5 (The Jacobian is a Löwner operator). *At an X with eigenvalues λ_i and eigenvectors U ,*

$$D\Pi_{\mathcal{S}_+^k}(\cdot)(X)[H] = U \left(\Omega \circ (U^\top H U) \right) U^\top, \quad \Omega_{ij} = \begin{cases} \frac{\lambda_i^+ - \lambda_j^+}{\lambda_i - \lambda_j} & \lambda_i \neq \lambda_j, \\ \mathbf{1}_{\lambda_i > 0} & \lambda_i = \lambda_j, \end{cases} \quad (20.3)$$

with $\circ$ the entrywise product and $\lambda^+ = \max(\lambda, 0)$.

The two branches of Ω are the only case analysis on this cone, and the test is relative rather than exact: two eigenvalues count as equal when their gap falls below machine epsilon times the larger of their magnitudes and one.

Algorithm 20.5 Applying the Löwner Jacobian

Input: the cached U and Ω , a packed v

- 1: $H \leftarrow \text{smat}(v)$
 - 2: $\tilde{H} \leftarrow \Omega \circ (U^\top H U)$ ▷ into the eigenbasis, entrywise, back out
 - 3: **return** $\text{svec}(U \tilde{H} U^\top)$ ▷ two congruences, $O(k^3)$
-

Algorithm 20.6 Materialising the G block

Input: the scaling at σ , the weights ρ_p and μ

- 1: **for** $j = 1, \dots, d$ **do**
 - 2: column j of $G \leftarrow$ the Löwner Jacobian applied to e_j , divided by μ ▷ one call above, $O(k^3)$
 - 3: **end for**
 - 4: add ρ_p to the diagonal
-

Proposition 20.6 (Cost of materialising a semidefinite block). *Forming the dense G block of a semidefinite cone of side k costs $O(k^5)$.*

Proof. The loop runs $d = k(k+1)/2$ times, and each pass applies the Löwner Jacobian, two matrix products and an entrywise product, $O(k^3)$. The total is $O(dk^3) = O(k^5)$. $\square$

This dominates any problem with a large semidefinite block and is why `random_psd` is the slowest of the generated sets. The remedy is not to materialise: a road that only applies G to a vector needs one congruence per application, $O(k^3)$. Both roads materialise because both place the block in a sparse matrix.

Algorithm 20.7 Nesterov and Todd scaling for the semidefinite cone

Input: s, z packed, with $X = \text{smat}(s) \succ 0$ and $Z = \text{smat}(z) \succ 0$

- 1: $(X^{1/2}, X^{-1/2}) \leftarrow$ symmetric square roots of X ▷ one eigendecomposition, eigenvalues floored
 - 2: $Y \leftarrow (X^{1/2} Z X^{1/2})^{1/2}$ ▷ a second
 - 3: $W \leftarrow X^{-1/2} Y X^{-1/2}$, $(Q, \Theta) \leftarrow$ its eigendecomposition ▷ a third
 - 4: $W^{\pm 1/2} \leftarrow Q \Theta^{\pm 1/2} Q^\top$
 - 5: $L \leftarrow W^{1/2} X W^{1/2}$, symmetrised; $(U, \lambda) \leftarrow$ its eigendecomposition ▷ a fourth
-

Every square root floors its eigenvalues at a relative 10^{-14} , so a block that has drifted to the boundary yields a large but finite scaling rather than a division by zero. The symmetrisation of L is not cosmetic: a product of two symmetric matrices is not symmetric, and passing an asymmetric matrix to a symmetric eigensolver silently discards half of it.

The centrality residual is $\lambda_i^2 - \tau$ on the k eigenvalues, reassembled by congruence, so $\langle X, Z \rangle = k\tau = \nu\tau$; the corrector is the symmetrised product of the congruence scaled steps, symmetrised for the reason just given. The step is $-1/\lambda_{\min}(X^{-1/2} \Delta X X^{-1/2})$ when that eigenvalue is negative and infinite otherwise, $O(k^3)$; the margin is $\lambda_{\min}(X)$ and the interior direction is $\text{svec}(I)$. Membership is tested by attempting a Cholesky of X shifted by a relative 10^{-9} , which is cheaper than an eigendecomposition and answers the same question.

20.6 The exponential and power cones

Neither has a closed form projection, and both reduce the dual projection to the primal one the same way.

Proposition 20.7 (Reduction by Moreau). *For any closed convex cone K and any v , one has $\Pi_{K^*}(v) = v + \Pi_K(-v)$.*

Both cones therefore implement the projection onto K itself, in the single line `out = point + project_onto_cone(-point)`.

20.6.1 The exponential cone

$$\mathcal{K}_{\text{exp}} = \overline{\{(x, y, z) : y > 0, y e^{x/y} \leq z\}},$$

not self dual, not symmetric, no Jordan product, block size always three, $\nu = 3$. With the slack $\ell = y \log(z/y) - x$ the barrier is $B = -\log \ell - \log y - \log z$, and its gradient and Hessian are in closed form by the quotient rule.

A point outside $\mathcal{K}_{\text{exp}}$ and outside its polar projects either onto a face or onto the smooth part of the boundary. On the smooth part the projection is determined by a single scalar, and the following says which.

Proposition 20.8 (The exponential boundary equation). *Let $g(a, b, c) = b e^{a/b} - c$, so that the smooth part of $\partial \mathcal{K}_{\text{exp}}$ is $\{g = 0, b > 0\}$, and parametrise it by*

$$p = (rb, b, b e^r), \quad r = a/b \in \mathbb{R}, \quad b > 0. \quad (20.4)$$

If $v = (x, y, z)$ projects onto that part at p , then

$$\nabla g(p) = (e^r, e^r(1-r), -1), \quad v - p = \mu \nabla g(p) \text{ for some } \mu \geq 0, \quad (20.5)$$

and eliminating b and μ leaves one equation in r alone.

Proof. The gradient is a direct computation: $\partial_a g = e^{a/b}$, $\partial_b g = e^{a/b}(1 - a/b)$ and $\partial_c g = -1$, which at p read as displayed. The projection onto a smooth surface has $v - p$ normal to it, hence parallel to $\nabla g(p)$, with a nonnegative multiplier because v lies outside.

Writing (20.5) in components,

$$x - rb = \mu e^r, \quad y - b = \mu e^r(1 - r), \quad z - b e^r = -\mu.$$

Dividing the second by the first removes μe^r and gives $y - b = (1 - r)(x - rb)$, that is

$$b = \frac{y - (1 - r)x}{r^2 - r + 1}, \quad (20.6)$$

the denominator being positive for every real r . The first and third give μ twice, as $(x - rb)e^{-r}$ and as $b e^r - z$, and setting those equal leaves an equation in r and b ; substituting (20.6) removes b . $\square$

The implementation evaluates an algebraically equivalent rearrangement of that equation, chosen so that the exponential appears only as e^{-r} and never as e^r , which cannot overflow:

$$h(r) = \kappa(r)(e^{-2r} + 1 - r) - y + z e^{-r}, \quad \kappa(r) = \frac{x - r y}{r^2 - r + 1},$$

the denominator of κ being positive for every real r . Its roots are the roots of the equation of Proposition 20.8, and the projection is recovered from a root by (20.6) and the parametrisation.

Nothing bounds r in advance, which is why the search sweeps. The parametrisation runs over all of $\mathbb{R}$, h is not monotone, and there is no analogue of the bracket the power cone enjoys below; the grid over $[-60, 60]$ is a search for sign changes, and the interval is where e^{-r} and e^{-2r} are representable.

Algorithm 20.8 Projection onto the exponential cone

Input: $v = (x, y, z)$

```

1: if  $v \in \mathcal{K}_{\text{exp}}$  then
2:   return  $v$ 
3: else if  $-v \in \mathcal{K}_{\text{exp}}^*$  then
4:   return 0
5: end if
6:  $best \leftarrow 0, \quad d_{best} \leftarrow \|v\|^2$ 
7: consider  $(\min(x, 0), 0, \max(z, 0))$  ▷ the axis faces
8: if  $y > 0$  and  $x/y < 100$  then
9:   consider  $(x, y, \max(z, y e^{x/y}))$  ▷ a vertical drop onto the smooth boundary
10: end if
11: for 1200 values of  $r$  spanning  $[-60, 60]$  do
12:   if  $h$  changes sign across the interval then
13:     bisect, at most 200 steps, and consider the boundary point at the root
14:   end if
15: end for
16: return  $best$ 

```

consider(p): if p is finite and lies in $\mathcal{K}_{\text{exp}}$ to a tolerance, and is nearer v than $best$, keep it

Three details of that loop are deliberate. The quantity e^{-r} is carried multiplicatively along the sweep and recomputed from scratch every **kAnchorEvery** steps, costing one exponential in that many iterations instead of one per iteration while bounding the drift. A candidate is admitted only after a membership test, so a root of h that is not a boundary point is discarded rather than returned. And the initial best is the origin at distance $\|v\|$, so the search cannot return anything worse than the trivial answer.

The Jacobian is not a finite difference. Once the projection is known the boundary is a smooth manifold near it, and the Jacobian follows from the implicit function theorem.

Algorithm 20.9 The Jacobian of the projection onto the exponential cone

Input: $\sigma \in \mathbb{R}^3$

```

1: if  $\sigma \in \mathcal{K}_{\text{exp}}$  then
2:   return  $I$ 
3: else if  $-\sigma \in \mathcal{K}_{\text{exp}}^*$  then
4:   return  $0$ 
5: end if
6:  $p \leftarrow \Pi_{\mathcal{K}_{\text{exp}}}(\sigma)$ 
7: if  $p_y \leq \varepsilon_{\text{deg}}$  then ▷ the projection landed on a face
8:   return the diagonal with  $F_{11} = 1$  if  $\sigma_x \leq 0$  and  $F_{33} = 1$  if  $\sigma_z \geq 0$ 
9: end if
10:  $g \leftarrow$  the boundary gradient at  $p$ ,  $m \leftarrow \langle \sigma - p, g \rangle / \langle g, g \rangle$ ,  $H \leftarrow$  the rank one boundary Hessian
11: return the leading  $3 \times 3$  block of  $\begin{pmatrix} I + mH & g \\ g^\top & 0 \end{pmatrix}^{-1}$  ▷ solved with full pivoting

```

The four by four system is exact up to rounding, and considerably better behaved than a difference quotient. The degenerate branch is the one worth noting: when the projection lands on a face rather than on the smooth part, the manifold has no tangent plane there and the implicit construction does not apply, so the Jacobian is taken as the diagonal indicator of the coordinates that survive.

The block is dense 3×3 on both roads, $\rho_p I + F/\mu$ on the projection road and $\rho_p I + \tau_j \nabla^2 B(s)$ on the barrier road, where $\tau_j = \langle s_j, z_j \rangle / 3$ is the block's own measure. The centrality residual is $z + \tau \nabla B(s)$, the barrier form being the only one available with no Jordan product to write $s \circ z$ with; Euler's identity still gives $\langle s, z \rangle = 3\tau$ at centrality. The corrector is identically zero, there being no product in which to form the omitted quadratic term, so these blocks receive pure centring. Step and margin are bisected.

20.6.2 The power cone

$$\mathcal{K}_\alpha = \{(x, y, z) : x^\alpha y^{1-\alpha} \geq |z|, x \geq 0, y \geq 0\}, \quad \alpha \in (0, 1),$$

with α carried on the cone object, so one problem may mix blocks of different α . The scheme is the same as the exponential cone's, except that the boundary admits an explicit parametrisation by a single multiplier, so the search is a bisection on a bracket known in advance rather than a sweep for sign changes.

Algorithm 20.10 Projection onto the power cone**Input:** $v = (x, y, z)$ and α

```

1: if  $v \in \mathcal{K}_\alpha$  then
2:   return  $v$ 
3: else if  $-v \in \mathcal{K}_\alpha^*$  then
4:   return 0
5: end if
6:  $best \leftarrow 0, \quad d_{best} \leftarrow \|v\|^2$ 
7: consider  $(0, \max(y, 0), 0), (\max(x, 0), 0, 0), (\max(x, 0), \max(y, 0), 0) \triangleright$  the three faces
8: if  $|z| > 0$  and the boundary value changes sign across  $[0, |z|]$  then
9:   bisect on the multiplier, and consider the boundary point with the sign of  $z$ 
   restored
10: end if
11: return  $best$ 

```

The parametrisation is by the normal multiplier rather than by a point of the boundary, and that is what makes the bracket free.

Proposition 20.9 (The power boundary equation). *Take $z \geq 0$, the other sign being recovered by symmetry, and let $g(x, y, z) = z - x^\alpha y^{1-\alpha}$, so that the smooth part of $\partial\mathcal{K}_\alpha$ is $\{g = 0, x, y > 0\}$. If $v = (X, Y, Z)$ projects onto that part at $p = (x, y, z)$ with multiplier $m \geq 0$, then*

$$\nabla g(p) = \left(-\frac{\alpha z}{x}, -\frac{(1-\alpha)z}{y}, 1 \right), \quad v - p = m \nabla g(p), \quad (20.7)$$

and the three components determine p from m alone:

$$z(m) = Z - m, \quad x(m)^2 - Xx(m) = m\alpha z(m), \quad y(m)^2 - Yy(m) = m(1-\alpha)z(m), \quad (20.8)$$

each of the last two taken at its positive root. The projection is then a root of

$$\varphi(m) = x(m)^\alpha y(m)^{1-\alpha} - z(m). \quad (20.9)$$

Proof. On the boundary $x^\alpha y^{1-\alpha} = z$, so $\partial_x g = -\alpha x^{\alpha-1} y^{1-\alpha} = -\alpha z/x$ and likewise for y , while $\partial_z g = 1$. Writing $v - p = m \nabla g(p)$ in components,

$$X - x = -\frac{m\alpha z}{x}, \quad Y - y = -\frac{m(1-\alpha)z}{y}, \quad Z - z = m.$$

The third is $z(m) = Z - m$. Multiplying the first by x and rearranging gives $x^2 - Xx = m\alpha z$, and the second likewise. Both are quadratics with a nonnegative constant term when $mz \geq 0$, so each has exactly one nonnegative root. Finally p lies on the boundary precisely when $x^\alpha y^{1-\alpha} = z$, which is $\varphi(m) = 0$. $\square$

Proposition 20.10 (The bracket is free). *Let $v = (X, Y, Z)$ with $X, Y \geq 0$ and $Z > 0$ lie outside $\mathcal{K}_\alpha$. Then*

$$\varphi(0) < 0 \leq \varphi(Z), \quad (20.10)$$

so $[0, Z]$ brackets a root of φ and no search for one is needed.

Proof. At $m = 0$ the constant terms in (20.8) vanish, so $x(0) = \max(X, 0) = X$, $y(0) = Y$ and $z(0) = Z$, whence

$$\varphi(0) = X^\alpha Y^{1-\alpha} - Z < 0,$$

the inequality being exactly the statement that $v \notin \mathcal{K}_\alpha$.

At $m = Z$ one has $z(Z) = 0$, so both constant terms vanish again, $x(Z) = X \geq 0$ and $y(Z) = Y \geq 0$, and

$$\varphi(Z) = X^\alpha Y^{1-\alpha} - 0 \geq 0.$$

φ is continuous on $[0, Z]$, so it has a root there. $\square$

This is the whole difference between the two non symmetric cones. The exponential cone is parametrised by a ratio ranging over all of $\mathbb{R}$, with no a priori bound, so its roots must be hunted by a sweep; the power cone is parametrised by a multiplier that is pinned between 0 and Z by Proposition 20.10, so one bisection on a known bracket suffices and the whole projection is $O(1)$.

In the implementation, the two quadratics are solved by `posroot( $\ell, q$ )`, the positive root of $t^2 - \ell t - q = 0$, taken in whichever of its two algebraic forms avoids subtracting nearly equal numbers, and φ is evaluated as $\exp(\alpha \log x + (1 - \alpha) \log y) - z$ so that the power is formed in logarithms rather than by two exponentiations.

Everything else is identical in form to the exponential cone: the same four regions with the same degenerate branch, the same implicit four by four construction with the power boundary function in place of the exponential one, a dense 3×3 block, the residual $z + \tau \nabla B(s)$, no corrector, and bisected step and margin.

Remark 20.11 (The block shape may not change between iterations.). The pattern is planned once, and `widest_operator_kind` declares the widest representation a cone will ever return so that the plan accommodates every iteration. This is why the second order cone answers `DiagPlusLowRank` although three of its four regions produce a diagonal. A cone returning a diagonal block on one iteration and a low rank block on the next would corrupt the factorisation silently, the numeric refill writing into slots that no longer mean what they meant when the pattern was built.

Remark 20.12 (Every scaling is built at a point and consumed at the same point.). `build_projection_scaling` caches quantities derived from σ , and `materialise_operator`, `apply_residual_scaling` and `complementarity_residual` all read that cache. Calling any of them after the iterate has moved returns a stale F , which is a descent direction for the wrong problem and shows up only as a slow solve.

Remark 20.13 (The metric is uniform on a curved block, and the kernel assumes it.). The kernel reads `scaling.penalty(0)` for a whole curved block rather than one entry per row, because admissibility gives $M = cI$ there. The schedule and the equilibration both guarantee it by flattening. A change that broke the guarantee would leave the kernel reading a single entry of a vector whose other entries it silently ignores.

Remark 20.14 (The orthant is the wrong cone to generalise from.). Its F is a mask, so every quantity on it is elementwise, its block is diagonal on both roads, and $F^2 = F$ makes several identities degenerate that are not degenerate elsewhere. Code that assumes a diagonal G , or indexes a block one row at a time, passes every test on the orthant and fails on the first second order block.

Chapter 21

The bindings and the Python interface

21.1 Where it happens

The interface is in two pieces. `bindings/bindings.cpp`, 265 lines, exposes the C++ solver to Python through NANOBIND and does nothing else; `python/proxgqp/`, 430 lines over five modules, is the layer a caller actually uses.

module	lines	what it holds
<code>cones.py</code>	107	the six cone classes and <code>cone_rows</code>
<code>problem.py</code>	52	<code>Problem</code> , and the validation
<code>settings.py</code>	114	the knob names, the presets, <code>make_settings</code>
<code>solver.py</code>	131	<code>solve</code> , <code>solve_qp</code> , <code>Solution</code>
<code>__init__.py</code>	26	the public names

The division of labour is strict, and it is the reason the binding is as short as it is: *nothing* in C++ knows about Python types beyond arrays and scalars, and nothing in Python knows about the solver beyond one function call.

21.2 The binding is one function

The whole solver is exposed as a single `solve`, taking fifteen arguments, all of them plain arrays and scalars:

<code>sizes</code>	<code>columns, rows</code>
<code>P</code>	<code>objective_start, objective_row, objective_value</code>
<code>q</code>	<code>linear</code>
<code>E</code>	<code>constraint_start, constraint_row, constraint_value</code>
<code>b</code>	<code>offset</code>
<code>K</code>	<code>cone_kind, cone_size, cone_exponent</code>
the rest	<code>settings, warm</code>

The two matrices cross as three arrays each, in compressed sparse column form, and are rebuilt on the C++ side; the cone list crosses as three parallel arrays, one integer kind, one size and one exponent per block, and is rebuilt by `build_cones`. No structure is shared, and no Python object is held past the call.

The warm start is the one argument with any shape to it: an optional triple (x, s, z) , which the binding unpacks into a **Results** and passes by pointer, or a null pointer when absent. That is the whole mechanism behind the sequential sets of the benchmark.

Note what the binding does *not* do. It does not validate, it does not convert types, and it does not name its arguments after anything in the solver: the constraint offset is **offset** here and b inside. Whatever arrives is mapped into Eigen views and handed to **solve**. Everything that could go wrong has already been prevented one level up.

21.3 The cones

Each cone is a small class carrying a kind, a size and an exponent, the last used only by the power cone.

class	kind	size means	rows
Zero (d)	0	the dimension	d
Nonneg (d)	1	the dimension	d
SecondOrder (d)	2	the dimension	d
PSDTriangle (k)	3	the <i>side</i> k	$k(k+1)/2$
Exponential ()	4	fixed	3
Power (α)	5	fixed	3

Table 21.1: The six cone classes. Only the semidefinite one has a size that is not its row count, and **cone_rows** is what converts a list of them into the row count of E .

The constraint is written as $Ex + s = b$ with $s \in K$, and K is the product of the cones in the order given. There is no separate way to say which rows belong to which block: the list determines it, and **cone_rows** computes the total that E and b must match.

21.4 Problem, which is where the checking is

Problem(P , q , E , b , **cones**) is a validating constructor and nothing more. It fixes the sizes from q and from the cone list, converts both matrices to compressed sparse column form with sorted indices, checks that every shape agrees, fills in zeros for a missing P or b , and flattens the cone list into the three arrays the binding wants.

Two of those are worth naming.

sort_indices is called on both matrices. Every downstream assumption about the pattern, and **validate_pattern** in the backend in particular, requires the row indices within a column to increase, and SCIPY does not guarantee it.

And the shape checks are done here rather than in C++ because this is the only place where a helpful message can be produced. A caller who passes an E with the wrong number of rows learns that, with both numbers, rather than reaching a pattern validation error inside a backend.

21.5 Settings, and the knob names

The C++ settings are a nested structure: one block per schedule, **bcl**, **gbcl**, **semismooth** and **interior**, the last two inheriting a shared part, together with the tolerances, the three iteration limits and the road and backend choice. Python flattens that into dotted names and a dictionary of presets.

```
make_settings(preset=None, **knobs)
```

Every knob is either a top level name such as `eps_abs`, or a dotted one such as `interior.mehrotra` or `gbcl.mu_adapt`, the prefix naming the schedule the knob belongs to. The enumerated ones are given as strings, resolved through the five tables `METHOD`, `LINE_SEARCH`, `PENALTY`, `ROAD` and `BACKEND`. So `method="interior"` rather than an enumerator, and an unknown string raises with the list of what is accepted.

`PRESETS` names twelve combinations, of which the useful ones are `"interior"` and `"semismooth"` to force a road, `"accurate"` and `"loose"` to move both tolerances together, and `"no_equilibration"`, `"no_mehrotra"` and `"schur"` to turn off one mechanism at a time. The last three exist for isolating a question rather than for solving faster.

`knob_names()` lists everything settable and `describe` returns the current value of each, which together are how a caller finds out what exists without reading the C++ headers.

21.6 Solving

Two entry points, and the difference between them is the only concession the interface makes to convenience.

```
solve(problem, warm_start=None, settings=None, **knobs)
```

takes a `Problem` and returns a `Solution`. The warm start may be a previous `Solution`, a dictionary with keys `x`, `s`, `z`, or any triple; all three become the same tuple before crossing.

```
solve_qp(P, q, A=None, b=None, G=None, h=None, ...)
```

takes the classical quadratic programming form, with $Ax = b$ and $Gx \leq h$, and assembles the cone problem itself: it stacks A above G , stacks b above h , and builds the cone list as a `Zero` block for the equalities followed by a `Nonneg` block for the inequalities. It exists so that a caller with an ordinary quadratic program never has to learn what a cone list is.

`Solution` carries the three vectors, the four residuals of the termination test together with the two relative residuals, the objective, the two iteration counts and the elapsed time, with `solved` and `residuals` as conveniences. The status is a string rather than an enumerator, mapped through `STATUS`.

Remark 21.1 (The slack that comes back is not the slack that was used.). `Solution.s` is the recomputed $b - Ex$ of the termination chapter, not the iterate's own s . Feeding a `Solution` straight back as a warm start therefore hands the solver a slack consistent with x by construction, which is the right thing, but it is not the point the previous solve was actually sitting at.

Chapter 22

Building it

22.1 What has to be present

The solver is C++17 with two external dependencies and one vendored one.

dependency	what for	how obtained
Eigen	dense linear algebra, sparse storage	found, or fetched
SUITESPARSE	CHOLMOD, the default backend	found, or fetched
NANOBIND	the Python bindings	found
QDLDL	the dependency free backend	vendored in <code>third_party/</code>
a BLAS	the dense kernels and the supernodal path	found

QDLDL is the only one carried in the tree, and it is carried so that the solver can be built and its tests run with no sparse library installed at all.

22.2 The environment

`environment.yml` pins everything, including the four other solvers the benchmark compares against, and is meant to be used with `micromamba` or `conda`:

```
micromamba env create -f environment.yml
micromamba activate proxqp
```

Two things about it are deliberate.

Everything comes from `conda-forge`, including `SUITESPARSE`, which is what makes `CHOLMOD` available without building it. That matters for more than convenience: `CHOLMOD` is not redistributed with this work, it is a dependency the environment installs, so nothing here ships code under its licence.

And the competitors are pinned in the same file. `Clarabel`, `PIQP`, `SCS`, `ECOS` and `OSQP` come from `conda-forge`; `HIGHS` and `PROXQP` from `pip`, those not being packaged there. A benchmark whose comparison depends on which versions were installed is not reproducible, and this is the file that fixes them.

22.3 Building

```
cmake -S solver -B build -G Ninja
cmake --build build -j
```

```
ctest --test-dir build
```

Three targets come out of it. `_core` is the Python extension module, built by NANOBIND and installed beside the pure Python package; `proxgqp_core` is a static library holding the solver without the bindings, which is what the tests link against; and each test is an executable registered with CTEST.

The split between the two libraries is what lets the tests exercise the solver without Python in the loop. A test failure is then a C++ failure, and nothing about the bindings can produce or mask one.

Threads while building. The build fetches nothing by default and compiles in a few minutes, but `-j` with no argument will use every core. On a workstation being used for anything else, ask for fewer.

22.4 Building without the dependencies present

```
cmake -S solver -B build -DPROXGQP_VENDOR_DEPENDENCIES=ON
```

fetches Eigen and SUITESPARSE at configure time and builds them in tree, pinned to the tags in the CMAKE file. It exists for the case where the environment cannot be used, and it is slower by a large factor, SUITESPARSE being a substantial build of its own.

Two settings are forced in that path and are worth knowing about. FORTRAN is disabled, SUITESPARSE otherwise requiring a compiler for it that a minimal environment may not have; and CHOLMOD is linked statically, so that the resulting extension module does not depend on a shared library that will not be present.

The supported path is the environment. The vendoring path is a fallback, and it is the one where the licence question is sharper: fetching and statically linking CHOLMOD means shipping it, where installing it from `conda-forge` does not.

22.5 Installing

`cmake --install` places the extension module and the Python package together, so that `import proxgqp` works from the install prefix. The package is pure Python except for `_core`, and the two must stay beside each other; nothing looks for the extension anywhere else.

Remark 22.1 (The build has no configuration for the solver itself.). Nothing about the method is a compile time choice. The road, the backend, the smoothing, the line search and the schedule are all settings resolved at run time, and the same binary runs every configuration in this document. The only compile time option is where the dependencies come from.

Part IV

Benchmark

Chapter 23

The harness

23.1 What it is

The benchmark is a directory of its own, 3739 lines over 63 files, and it is a separate program from the solver: it runs every solver as a child process, grades all of them by one rule it implements itself, and never links against any of them.

piece	what it holds	how many
<code>utils/</code>	the shared machinery, the verifier, the aggregates, the run store	13 files
<code>solvers/</code>	one adapter per solver	11 files
one directory per set	how to build or read that set's problems	15 directories
<code>prepare.py</code>	fetching, generating and checksumming the data	232 lines
<code>benchmark_report.py</code>	the driver and the report	219 lines
<code>run_set.py</code>	one set, from the command line	36 lines

The configuration lives in one place, at the top of `benchmark_report.py`: the five solvers, the fifteen sets, which of them are sequential, the tolerance 10^{-9} , the time limit of twenty seconds, the worker count, which is the machine's core count less four.

23.2 Getting the data

Three of the fifteen sets are collected archives rather than generated problems, and one more reads market data. `prepare.py` handles all of it and is the first thing to run.

command	what it does
<code>python prepare.py generate</code>	builds the six planted sets
<code>python prepare.py fetch</code>	downloads Maros, Netlib and CBLIB
<code>python prepare.py manifest</code>	writes the checksums of what is present
<code>python prepare.py verify</code>	checks every file against the manifest

The manifest holds a SHA256 for every problem file and for every market data file, so a set that has been silently altered is detected rather than benchmarked. The generated sets are checksummed too, on the problem itself rather than the file, so that a change to the generator shows up as a mismatch even though the file is written afresh each time.

23.3 Running it

command	what it does
<code>python benchmark_report.py</code>	every solver on every set, then the report
<code>python run_set.py <i>name</i></code>	one set, same defaults
<code>python run_set.py <i>name</i> --solvers a b</code>	one set, chosen solvers
<code>python run_set.py <i>name</i> --out-of-the-box</code>	one set, every solver on its defaults
<code>python run_set.py <i>name</i> --run <i>id</i></code>	one set, appended to an existing run
<code>python tune.py</code>	a small gate, for checking a change quickly

Every solve is a separate process. `run_one.py` takes a set, a problem, a solver and a tolerance, runs exactly one solve, and prints one JSON record. That is the unit of the whole harness, and it has three consequences worth naming.

A solver that crashes takes down one process and produces one failed record, rather than ending the run. A solver that hangs is killed by the parent's timeout, which is how the twenty second limit is enforced regardless of whether the solver honours a limit of its own. And no solver can affect another through global state, a threading library or a memory allocator, because none of them is ever loaded into the same process.

23.3.1 One core per solve

Each child is launched under `taskset` on a core drawn from a pool, and its environment is set so that every threading library it might link sees a single thread:

```
OMP_NUM_THREADS,  OPENBLAS_NUM_THREADS,  MKL_NUM_THREADS,  VECLIB_MAXIMUM_THREADS.
```

Both halves are needed and neither is sufficient. Without the environment a solver linked against a threaded BLAS would use every core it could find, and the timings of whatever ran beside it would be meaningless. Without the pinning the operating system would migrate the children between cores and the same solve would time differently on successive runs.

23.3.2 Sequential sets are different

Two sets measure warm starting, and they cannot be run problem by problem. Their problems form chains, each chain a sequence of neighbouring problems, and each solve is warm started from the previous one's answer.

The driver therefore has a second path. It groups the problems into chains by a key the set supplies, and gives each pair of a chain and a solver to one worker, which runs that chain in order, carrying the answer forward in a scratch file. Parallelism is across chains and solvers rather than across problems, which is what preserves the order within a chain while still using the machine.

23.4 Where a run goes

A run owns a directory, and writes into nothing else:

```
result/runs/<timestamp>-<uid>/
```

holding one JSON file per solver and set, named `<solver>_<set>.json`, and one `meta.json` describing the run itself: the identifier and the uid, when it started and finished in UTC, how long it took, which sets and solvers it covered, the settings it ran under, the versions of the rival solvers, and whether it finished at all. `result/runs/latest` points at the newest.

`meta.json` records the processor, the core count, the Python version and the thread environment variables, and deliberately nothing else about the machine: no host name, no paths, no repository or branch.

23.5 Grading

No solver's own verdict is used. Every returned point is graded by `utils/verify.py`, which recomputes the four residuals from the original problem data:

$$\begin{aligned} \text{primal} &= \|\Pi_{K^*}(-\hat{s})\|_\infty, & \text{dual} &= \|Px + q + E^\top z\|_\infty, & \hat{s} &= b - Ex, \\ \text{cone} &= \|z - \Pi_{K^*}(z)\|_\infty, & \text{gap} &= |\langle \hat{s}, z \rangle|, \end{aligned}$$

each against $\varepsilon_{\text{abs}} + \varepsilon_{\text{rel}}$ times its own scale, with $\varepsilon_{\text{abs}} = \varepsilon_{\text{rel}} = 10^{-9}$. A solver reporting success whose point fails this is recorded as `solved` but not `verified`, and only `verified` counts.

The regrading is a check on the answer, not a substitute for each solver's own stopping rule. Every solver is configured to stop on *its own* criterion at the same accuracy: each adapter sets that solver's absolute tolerance to 10^{-9} , its relative tolerance to 10^{-9} , and, where the solver distinguishes one, its duality gap tolerance to the same. Each therefore decides for itself when it has finished, by its own definition of finished, at a target no looser than ours.

Each is also given an unbounded iteration budget, ours included. A solver stopped by an iteration limit would be reported as failing on problems it could have solved, and every solver in the comparison does better without one, so none is given one; the twenty second limit is the only budget any of them faces.

What the regrading adds is that a claim of success is checked rather than taken. A solver that reports `solved` at a point whose residuals do not meet the bar is recorded as `unverified`, and no solver's own verdict is allowed to stand unexamined, ours least of all.

23.5.1 What a solver is not blamed for

A set whose cones a solver cannot express is not counted against it. The adapter returns `unsupported`, those rows are dropped from that solver's totals rather than recorded as failures, and its score is reported out of what it could express.

23.6 Statistics

Definition 23.1 (Shifted geometric mean). For values $v_1, \dots, v_N \geq 0$ and a shift $\varsigma > 0$,

$$\text{sgm}_\varsigma(v) = \exp\left(\frac{1}{N} \sum_{i=1}^N \log(v_i + \varsigma)\right) - \varsigma. \quad (23.1)$$

The geometric mean is used because a solver is judged on ratios: being twice as fast on a fast problem should count as much as being twice as fast on a slow one, which an

arithmetic mean does not do. The shift keeps a single very small value from dominating, and the harness takes ς equal to one millisecond.

A failed solve is scored at the cap rather than dropped, because dropping it would reward a solver for giving up, and excluding it would compare solvers on different problem sets. The cap is the twenty second limit, which is meaningful: a timed out solve genuinely consumed twenty seconds.

Chapter 24

The benchmark sets

Fifteen sets, 1509 instances, of three kinds: six generated with a planted solution, three collected archives, and six built from portfolio optimisation. This chapter says what each contains, in what form, and why the instance counts are what they are.

set	kind	instances	cones
random_qp	generated	90	zero, orthant
random_socp	generated	90	zero, orthant, second order
random_psd	generated	60	zero, orthant, semidefinite
random_exp	generated	60	zero, orthant, exponential
random_pow	generated	60	zero, orthant, power
random_mixed	generated	60	all six
maros	collected	138	zero, orthant
netlib	collected	109	zero, orthant
cbllib	collected	134	exponential, power, second order
skfolio_real_qp	portfolio	96	zero, orthant
skfolio_real_curved	portfolio	192	second order, exponential
skfolio_random_qp	portfolio	60	zero, orthant
skfolio_random_curved	portfolio	120	second order, exponential
seq_rebalance	portfolio, warm	120	zero, orthant
seq_rebalance_curved	portfolio, warm	120	second order, exponential

24.1 The generated sets

Six sets sweep one cone at a time over sizes from $n = 30$ to $n = 3000$. They exist because the collected archives contain almost no curved cones: without them the second order, semidefinite, exponential and power paths would be exercised only by CBLIB and by the portfolio sets, and never in isolation.

Every instance is built *backwards* from a solution, which is what makes the answer known in advance rather than agreed on by majority vote among solvers.

Algorithm 24.1 Generating one instance with a planted solution

Input: the variable count n , a cone specification, a seed

Output: (P, q, E, b, K) and the pair (x^*, z^*) solving it

```

1: for each cone block in the specification do
2:   draw its regime: active, inactive or degenerate, with probabilities  $\pi_{\text{act}}, \pi_{\text{inact}}, \pi_{\text{deg}}$ 
3:   draw a complementary pair  $(s_j, z_j)$  in that regime  $\triangleright s_j \in K_j, z_j \in K_j^*,$ 
      $\langle s_j, z_j \rangle = 0$ 
4: end for
5:  $s \leftarrow$  the concatenation,  $z^* \leftarrow$  the concatenation,  $m \leftarrow \dim s$ 
6:  $E \leftarrow$  a sparse  $m \times n$  matrix, ten nonzeros per row, normal entries
7:  $x^* \leftarrow$  a normal vector of length  $n$ 
8:  $b \leftarrow Ex^* + s$   $\triangleright$  so  $(x^*, s)$  is feasible by construction
9:  $P \leftarrow Q\Lambda Q^\top$  with  $Q$  a random orthogonal matrix and  $\Lambda$  logarithmically spaced over
   the requested condition number and rank
10:  $q \leftarrow -Px^* - E^\top z^*$   $\triangleright$  so stationarity holds at  $x^*$ 
11: return the problem, together with  $x^*, z^*$  and the objective value at  $x^*$ 

```

The last two lines are the whole construction. Feasibility is arranged by defining b from a chosen x^* and a chosen cone-feasible slack; stationarity is arranged by defining q so that $Px^* + q + E^\top z^* = 0$; and complementarity holds because each block's pair was drawn complementary. The three conditions of the theory part are therefore satisfied by construction, and the instance carries its own optimal objective value.

The three regimes. How a complementary pair is drawn is what the regime selects, and the three are chosen to exercise different behaviour.

regime	the pair	what it tests
active	s_j on the boundary, $z_j \neq 0$ normal to it	the curved parts of F
inactive	s_j strictly inside, $z_j = 0$	the identity or zero branches of F
degenerate	$s_j = z_j = 0$	the vertex, where F is set valued

On the second order cone, for instance, an active block takes $s_j = (\|u\|, u)$ on the boundary and z_j a positive multiple of $(\|u\|, -u)$, which is the outward normal there; on the exponential cone it takes s_j on the smooth boundary and z_j a negative multiple of the gradient $(e^r, e^r(1-r), -1)$ of Proposition 20.8. The three probabilities are 0.6, 0.3 and 0.1. The degenerate regime, at one block in ten, is the one that puts a block exactly at the point where the Clarke Jacobian is a set rather than a matrix.

The sweep. Each set is the product of five sizes, three condition numbers, two objective ranks and three seeds, giving 90, or a subset of that for the more expensive cones. The condition number enters through Λ , whose entries are logarithmically spaced from 1 down to 10^{-c} , so a problem with $c = 6$ has an objective spanning six orders of magnitude. The rank parameter chooses between a full rank P and $P = 0$, the second being a linear program with cones.

24.2 The collected archives

Three archives of problems from the literature, none generated here : they are graded, like everything else, by the harness recomputing the residuals.

24.2.1 Maros and Mészáros, 138 instances

The standard repository of convex quadratic programs,

$$\min_x \frac{1}{2} \langle x, Px \rangle + \langle q, x \rangle \quad \text{subject to} \quad Ax = b, \quad l \leq Cx \leq u, \quad \ell \leq x \leq v,$$

converted to the split form by turning each two sided constraint into a pair of one sided rows and each equality into a zero cone block. It is the reference set for quadratic programming and the reason PROQP and PIQP appear in the comparison at all.

24.2.2 Netlib, 109 instances

The classical linear programming archive, so $P = 0$ throughout, in the form

$$\min_x \langle q, x \rangle \quad \text{subject to} \quad Ax = b, \quad x \geq 0.$$

It is included because a linear program is the hardest case for a proximal method: with $P = 0$ the only strong convexity in the subproblem is the proximal term itself, so the schedule has nothing else holding the system together as ρ falls. The semismooth road's weakest result in the whole benchmark is on this set.

24.2.3 cblib, 134 instances of 934

The Conic Benchmark Library, and the only archive here with curved cones. The instance count needs explaining, since the library holds 934 files and only 134 are used.

The library is assembled for conic solvers generally, and most of its contents are outside what this solver addresses:

- the great majority are *mixed integer* conic problems, carrying integrality constraints on some variables. Nothing in this work addresses integrality, and a continuous solver run on such an instance answers a different question, namely the continuous relaxation;
- a further group are known to be primal or dual *infeasible*, and are in the library precisely to test infeasibility detection rather than solution;
- a smaller group are *weakly* infeasible or unbounded, where no certificate of finite size exists, and where every solver in the comparison would be reporting a judgement rather than a result.

What remains, after taking the continuous, feasible and bounded members, is 134 instances. That is the set used, and the filter is applied once when the archive is prepared rather than at run time, so every solver receives the same 134.

The cone mix is what makes it valuable: 42 instances carry exponential cones, 12 carry power cones, and the rest are second order or mixed. It is also by a wide margin the hardest set in the benchmark.

24.3 The portfolio sets

Six sets built from `skfolio`, in six formulations, on real and on synthetic return data. They are here because they are what the solver was asked to do: the exponential cone entered this work because benchmarking against `skfolio` showed it was needed.

formulation	cones	in
mean variance	zero, orthant	the qp sets
mean CVAR	zero, orthant	the qp sets
minimum deviation	second order	the curved sets
mean EVAR	exponential	the curved sets
risk budgeting	exponential	the curved sets
maximum diversification	second order	the curved sets

Table 24.1: The six formulations. Two are quadratic programs and four are not, which is why the portfolio sets are split.

Throughout, $R \in \mathbb{R}^{T \times d}$ is the matrix of returns, T observations of d assets, Σ its covariance, μ its column means, and w the weights. Every formulation carries the budget and long only constraints

$$\mathbf{1}^\top w = 1, \quad w \geq 0,$$

a zero cone row and an orthant block, and differs only in what it adds.

Mean variance, a quadratic program.

$$\min_w \frac{1}{2} \langle w, \Sigma w \rangle - \gamma \langle \mu, w \rangle.$$

Nothing beyond the budget and the sign constraints, so $P = \Sigma$ and the problem is an ordinary quadratic program.

Mean cvar, a linear program. With β the confidence level, and introducing a value at risk level τ and one excess variable u_t per observation,

$$\min_{w, \tau, u} -\gamma \langle \mu, w \rangle + \tau + \frac{1}{T(1-\beta)} \sum_{t=1}^T u_t \quad \text{subject to} \quad u_t \geq -\langle R_t, w \rangle - \tau, \quad u \geq 0.$$

The conditional value at risk is written by its Rockafellar and Uryasev linearisation, so $P = 0$ and the two families of constraints are orthant blocks of T rows each. It is a linear program with $d + 1 + T$ variables, and it is the larger of the two quadratic sets by a wide margin.

Minimum deviation, a second order cone. Writing $\Sigma = FF^\top$ for a factor F ,

$$\min_{w, t} t \quad \text{subject to} \quad (t, F^\top w) \in \mathcal{Q}^{d+1},$$

that is $t \geq \|F^\top w\| = \sqrt{\langle w, \Sigma w \rangle}$. Minimising the standard deviation rather than the variance, which is the same optimiser but a conic rather than a quadratic problem, and the reason the second order cone appears in a portfolio set at all.

Maximum diversification, a second order cone. With σ the vector of individual standard deviations,

$$\min_{w,t} t \quad \text{subject to} \quad \langle \sigma, w \rangle = 1, \quad (t, F^\top w) \in \mathcal{Q}^{d+1}, \quad w \geq 0.$$

The budget row is replaced by the normalisation $\langle \sigma, w \rangle = 1$, and minimising the portfolio deviation subject to it maximises the diversification ratio $\langle \sigma, w \rangle / \sqrt{\langle w, \Sigma w \rangle}$.

Mean evar, exponential cones. The entropic value at risk is a log sum exponential, and it is written with one exponential cone per observation. With a scale z , a width v and u_t one variable per observation,

$$\min_{w,z,v,u} -\gamma \langle \mu, w \rangle + z + v \log \frac{1}{T(1-\beta)} \quad \text{subject to} \quad \sum_{t=1}^T u_t \leq v, \quad (\langle R_t, w \rangle + z, v, u_t) \in \mathcal{K}_{\text{exp}}.$$

Each cone membership is $v e^{(\langle R_t, w \rangle + z)/v} \leq u_t$, which is the epigraph form of one term of the exponential moment, and summing the u_t under v is the constraint that makes the whole a log sum exponential bound. This formulation, and the next, are why the exponential cone was needed.

Risk budgeting, exponential cones. With a prescribed budget vector c summing to one, and y the logarithms of the weights,

$$\min_{w,y} \frac{1}{2} \langle w, \Sigma w \rangle \quad \text{subject to} \quad \langle c, y \rangle \geq 0, \quad w \geq 0, \quad (y_i, 1, w_i) \in \mathcal{K}_{\text{exp}}.$$

The cone membership $1 \cdot e^{y_i} \leq w_i$ is $y_i \leq \log w_i$, so the constraint $\langle c, y \rangle \geq 0$ is the log barrier $\sum_i c_i \log w_i \geq 0$ that produces a risk budgeted portfolio. There are d exponential blocks, one per asset, which makes this the formulation with the most curved blocks of the six.

Why the split. Only two of the six formulations are quadratic programs. Before the split, a single `skfolio` set mixed all six, and PIQP and PROXQP could express only a third of it while being scored on the whole; the two halves are now separate sets so that every solver is compared on problems it can state.

Real and synthetic. The *real* sets use market data, swept over datasets, estimation windows and eras; the *random* sets generate returns instead, which removes the dependence on the data files and lets the same formulations be run at sizes the market data does not offer.

24.4 The sequential sets

Two sets, and the only ones that measure warm starting. Each is a family of chains, and within a chain each problem is a small perturbation of the last:

family	what moves along the chain
rolling	the estimation window slides forward one period at a time
frontier	the risk aversion γ is swept along the efficient frontier
beta	a market exposure constraint is tightened by steps

Three families, forty steps each, and one problem per step, giving 120 instances per set. `seq_rebalance` uses the two quadratic formulations and `seq_rebalance_curved` the four curved ones. .

Chapter 25

Results

25.1 The conditions

Every number below comes from one run of `benchmark_report.py` on a single machine: 1509 instances over fifteen sets, five solvers, tolerance 10^{-9} absolute and relative, a twenty second limit, one pinned core per solve, and every point regraded by the harness. Rows a solver cannot express are excluded from its totals rather than counted against it. It was taken on 2026-09-05, started 12:46 UTC and finished 13:57 UTC, seventy minutes for the 1509 instances, on an `x86_64` machine with twelve cores, eight of them carrying the harness, one thread per solve, under Python 3.14, against Clarabel 0.11.1, PIQP 0.6.3 and PROXQP 0.7.3, the last as shipped in `proxsuite` 0.7.3. Every record it produced, one line per instance and solver, is kept beside the code in

`benchmark/result/runs/20260905T124650Z-db22480c/`

with a `meta.json` recording the machine, the settings and the duration. The identifier is what the harness assigns each run, so a later run lands beside this one rather than over it, and the tables here can always be checked against the records they came from.

These numbers move as the solver does. They describe the build of that day and are meant to be replaced whenever the solver changes enough to shift them; the comparison is worth repeating rather than quoting, and the harness exists so that repeating it costs one command.

25.2 The quadratic sets

Six of the fifteen sets are quadratic programs, and on those all five solvers can be compared on identical data: `random_qp`, `maros`, `netlib`, `skfolio_real_qp`, `skfolio_random_qp` and `seq_rebalance`, 613 instances in all.

set	gqp interior	gqp semismooth	Clarabel	PIQP	ProxQP
random_qp (90)	72	72	72	67	68
maros (138)	131	94	107	121	80
netlib (109)	87	38	66	85	30
skfolio_real_qp (96)	96	95	95	94	94
skfolio_random_qp (60)	60	53	49	55	55
seq_rebalance (120)	120	120	118	115	115
all (613)	566	472	507	537	442

Table 25.1: Quadratic sets: instances verified. Every solver can express every problem here, so all five are scored out of the same 613.

set	gqp interior	gqp semismooth	Clarabel	PIQP	ProxQP
random_qp (90)	0.146	0.245	0.176	0.202	0.341
maros (138)	0.043	0.266	0.141	0.056	0.928
netlib (109)	0.225	2.765	0.629	0.157	5.590
skfolio_real_qp (96)	0.013	0.019	0.019	0.017	0.084
skfolio_random_qp (60)	0.041	0.097	0.112	0.081	0.139
seq_rebalance (120)	0.003	0.004	0.005	0.007	0.016
all (613)	0.035	0.109	0.072	0.048	0.288

Table 25.2: Quadratic sets: shifted geometric mean of seconds, shift one millisecond, failures scored at the twenty second limit.

25.3 All fifteen sets

Only three of the five solvers carry all six cones, so the comparison over the whole benchmark is between those three.

set	gqp interior	gqp semismooth	Clarabel
random_qp (90)	72	72	72
random_socp (90)	71	69	69
random_psd (60)	54	57	45
random_exp (60)	45	45	44
random_pow (60)	56	59	51
random_mixed (60)	35	47	27
maros (138)	131	94	107
netlib (109)	87	38	66
cbllib (134)	66	42	19
skfolio_real_qp (96)	96	95	95
skfolio_real_curved (192)	192	155	110
skfolio_random_qp (60)	60	53	49
skfolio_random_curved (120)	105	72	82
seq_rebalance (120)	120	120	118
seq_rebalance_curved (120)	120	59	36
all (1509)	1310	1077	990

Table 25.3: All fifteen sets: instances verified, for the three solvers carrying all six cones.

set	gqp interior	gqp semismooth	Clarabel
random_qp (90)	0.146	0.245	0.176
random_socp (90)	0.291	0.288	0.262
random_psd (60)	0.071	0.041	0.250
random_exp (60)	0.536	0.535	0.279
random_pow (60)	0.211	0.146	0.154
random_mixed (60)	0.478	0.393	1.521
maros (138)	0.043	0.266	0.141
netlib (109)	0.225	2.765	0.629
cbplib (134)	2.435	5.542	11.248
skfolio_real_qp (96)	0.013	0.019	0.019
skfolio_real_curved (192)	0.037	0.124	0.259
skfolio_random_qp (60)	0.041	0.097	0.112
skfolio_random_curved (120)	0.139	1.174	0.272
seq_rebalance (120)	0.003	0.004	0.005
seq_rebalance_curved (120)	0.131	0.735	1.430
all (1509)	0.100	0.256	0.262

Table 25.4: All fifteen sets: shifted geometric mean of seconds. Machine dependent, unlike the counts above.

Chapter 26

Closing remark

The solver began as a study of the ProxQP paper. The first generalisation was implemented for the second order cone alone, on the semismooth road. A great deal of time then went into trying to keep the per iteration cost of the qp case, by changing the form of the problem; most of those attempts failed, and the ones that succeeded did so only in particular cases, such as a single second order block together with an orthant, where a pencil argument or a geometric condition on C applies, and even there at the price of a large constant. Other libraries, hyhound among them, were tried in the same context.

Benchmarking against skfolio showed that the exponential cone was necessary, so the codebase was restructured to support all six cones, with Claude Code serving as the coding agent. Discovering PIQP demonstrated that an interior-point version of the algorithm was feasible. This led to the development of a smoothing-point perspective, whose main advantage was that it required no changes to the underlying linear algebra. An interior-point version was subsequently planned and added.

Four possible directions remain. The cost of the linear algebra is the first and most significant. Scheduling is the second. It determines the iteration count on the semismooth path, and GBCL, obtained by writing BCL for each constraint after the generalisation, already outperforms BCL in this regard. The interior-point path is no different, and its scheduling deserves the same treatment.

Initialisation is another possible direction. Mixed methods are the fourth and least explored: one could apply an interior-point treatment to some blocks and a semismooth treatment to others, or switch between the two during the solve i.e using the interior-point path far from the solution, where it requires fewer iterations, and the proximal path near the solution, where warm-starting is advantageous.

Bibliography

- [1] R. Bhatia. *Matrix Analysis*. Graduate Texts in Mathematics 169, Springer, 1997.
- [2] J. Bolte, A. Daniilidis and A. S. Lewis. Tame functions are semismooth. *Mathematical Programming*, 117:5–19, 2009.
- [3] S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.
- [4] J. R. Bunch and L. Kaufman. Some stable methods for calculating inertia and solving symmetric linear systems. *Mathematics of Computation*, 31:163–179, 1977.
- [5] R. Chares. *Cones and Interior-Point Algorithms for Structured Convex Optimization involving Powers and Exponentials*. PhD thesis, Université catholique de Louvain, 2009.
- [6] T. A. Davis and W. W. Hager. Modifying a sparse Cholesky factorization. *SIAM Journal on Matrix Analysis and Applications*, 20:606–627, 1999.
- [7] L. C. Evans and R. F. Gariepy. *Measure Theory and Fine Properties of Functions*. Revised edition, CRC Press, 2015.
- [8] J. Faraut and A. Korányi. *Analysis on Symmetric Cones*. Oxford University Press, 1994.
- [9] P. E. Gill, G. H. Golub, W. Murray and M. A. Saunders. Methods for modifying matrix factorizations. *Mathematics of Computation*, 28:505–535, 1974.
- [10] N. J. Higham. *Accuracy and Stability of Numerical Algorithms*. Second edition, SIAM, 2002.
- [11] S. Mehrotra. On the implementation of a primal-dual interior point method. *SIAM Journal on Optimization*, 2:575–601, 1992.
- [12] G. J. Minty. Monotone (nonlinear) operators in Hilbert space. *Duke Mathematical Journal*, 29:341–346, 1962.
- [13] Y. Nesterov and A. Nemirovskii. *Interior-Point Polynomial Algorithms in Convex Programming*. SIAM, 1994.
- [14] Y. E. Nesterov and M. J. Todd. Self-scaled barriers and interior-point methods for convex programming. *Mathematics of Operations Research*, 22:1–42, 1997.
- [15] A. Pazy. Asymptotic behavior of contractions in Hilbert space. *Israel Journal of Mathematics*, 9:235–240, 1971.

- [16] L. Qi and J. Sun. A nonsmooth version of Newton's method. *Mathematical Programming*, 58:353–367, 1993.
- [17] R. T. Rockafellar. *Convex Analysis*. Princeton University Press, 1970.
- [18] R. T. Rockafellar. Monotone operators and the proximal point algorithm. *SIAM Journal on Control and Optimization*, 14:877–898, 1976.
- [19] R. T. Rockafellar. Augmented Lagrangians and applications of the proximal point algorithm in convex programming. *Mathematics of Operations Research*, 1:97–116, 1976.
- [20] D. Ruiz. A scaling algorithm to equilibrate both rows and columns norms in matrices. Technical Report RAL-TR-2001-034, Rutherford Appleton Laboratory, 2001.
- [21] L. van den Dries. *Tame Topology and O-minimal Structures*. London Mathematical Society Lecture Note Series 248, Cambridge University Press, 1998.
- [22] A. J. Wilkie. Model completeness results for expansions of the ordered field of real numbers by restricted Pfaffian functions and the exponential function. *Journal of the American Mathematical Society*, 9:1051–1094, 1996.